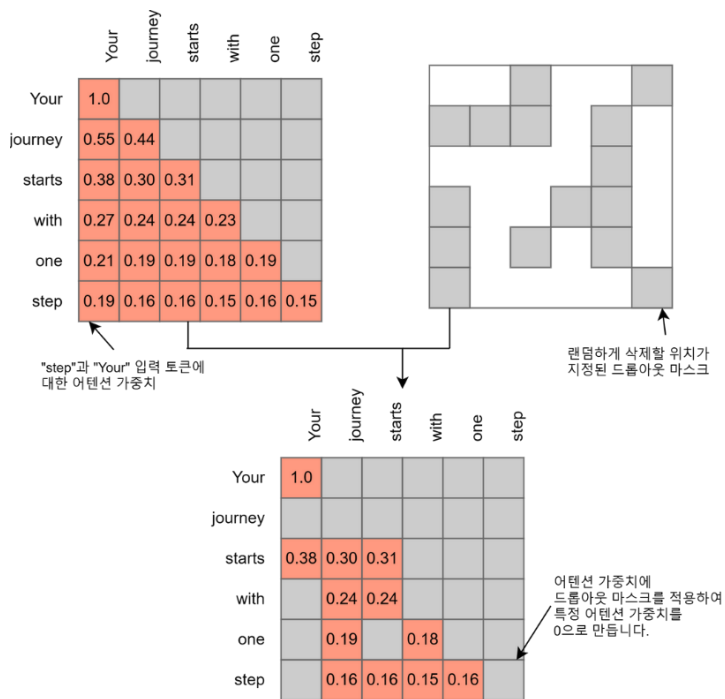


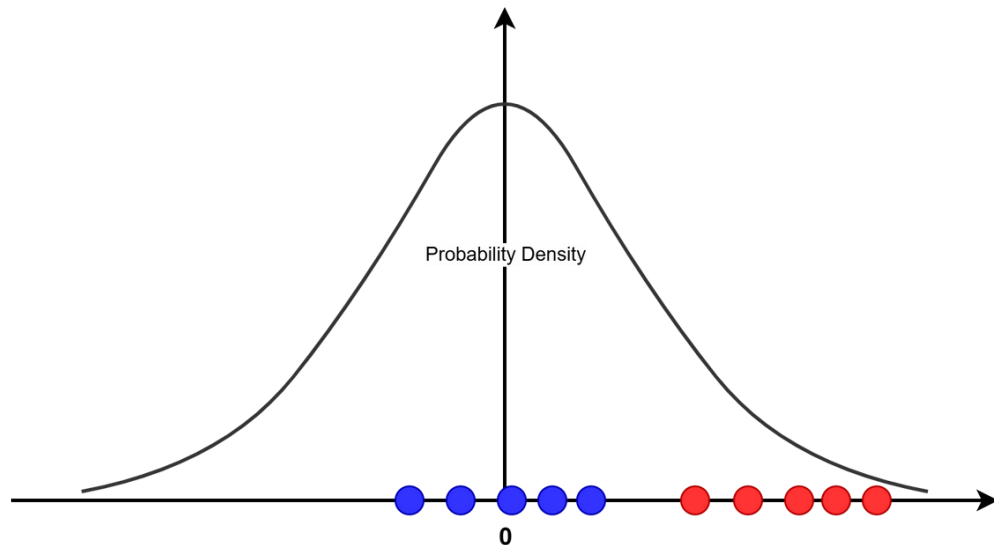
✓ Overfitting 방지

- Softmax 이후에 랜덤으로 0으로 마스킹함, 이번에 학습할 때는 여기만 집중해
- 학습 속도가 느리고, 데이터가 많아서 학습에는 잘 사용하지 않음



✓ 평균이 0이고 표준편차가 1이 되도록 변환

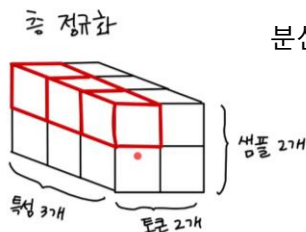
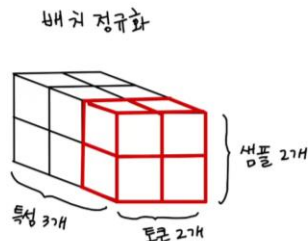
- 학습을 안정화 시킴
- 과도한 정규화는 학습 내용을 사라지게 할 수 있음



정규화: Layer Normalization

✓ LLM에서는 주로 Layer Normalization을 사용함

- 학습된 내용을 보존해서 필요한 범위만 정규화 하도록 감마(γ)와 베타(β)는 학습
- 학습하는 이유는 정규화가 항상 최선은 아니기 때문



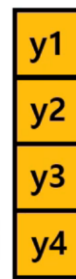
$$\text{평균계산 } \mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\text{분산계산 } \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\text{정규화 } \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

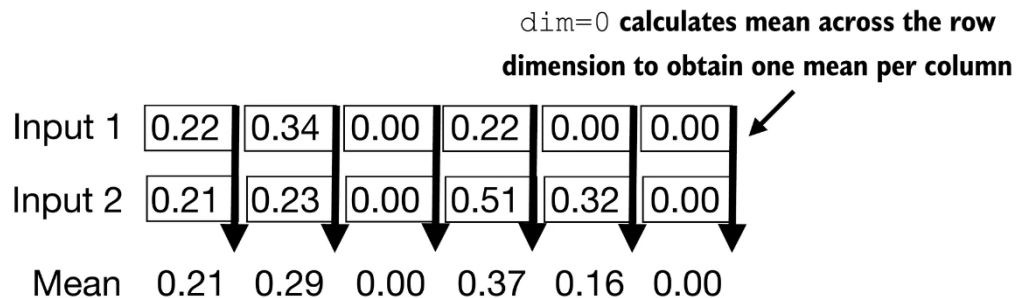
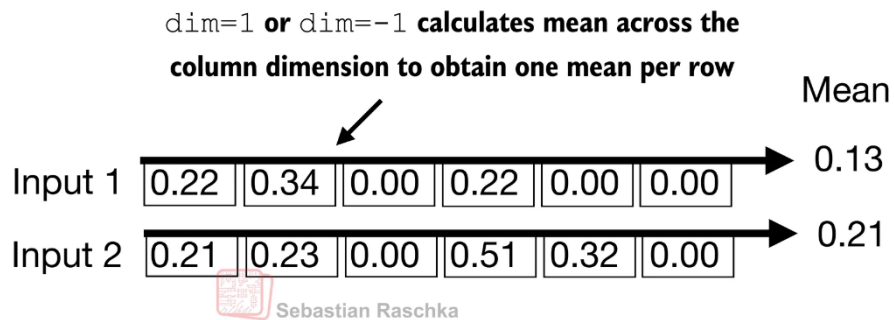
$$\text{스케일 및 시프트 } y_i = \gamma \hat{x}_i + \beta$$

layer



정규화: Normalization

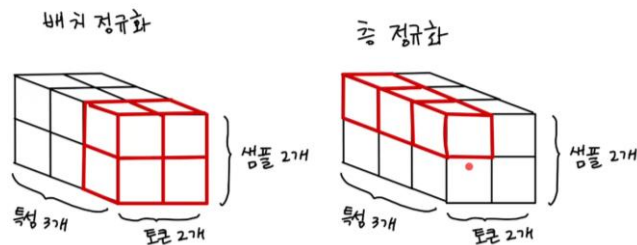
- ✓ 행렬에서 Layer, Batch Normalization



정규화: RMS Normalization

✓ 비용이 많이 드는 평균을 제외

- 정규화의 핵심 효과는 평균을 0으로 맞추는 것보다 데이터의 크기(Scale)를 일정하게 유지하는 것에서 나옴



$$\cancel{\mu_B} = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \cancel{\mu_B})^2$$

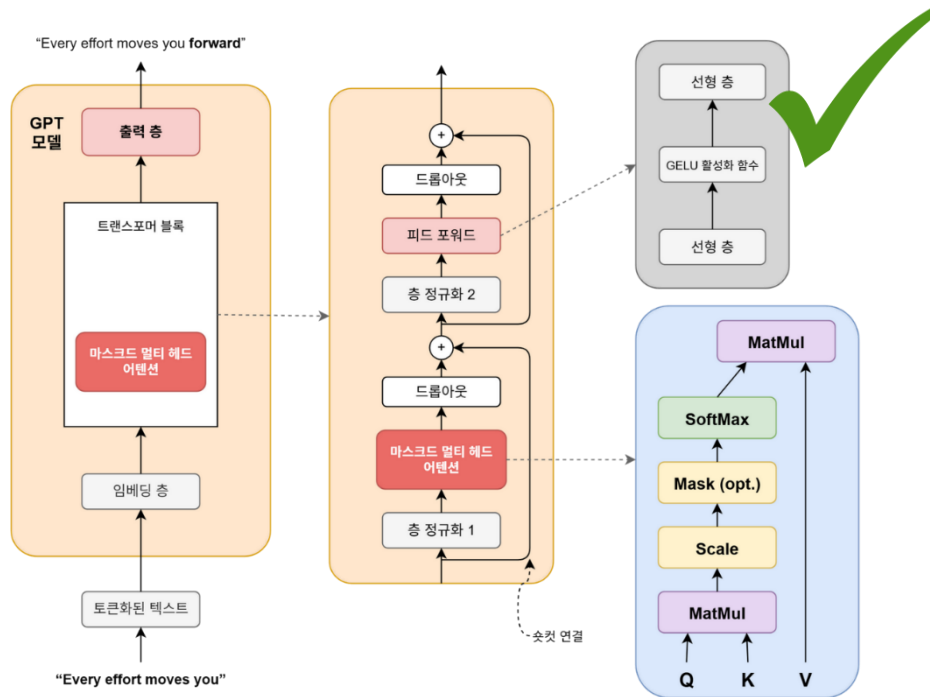
$$\hat{x}_i = \frac{x_i - \cancel{\mu_B}}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \boxed{\gamma} \hat{x}_i + \boxed{\beta}$$

Feed Forward Network

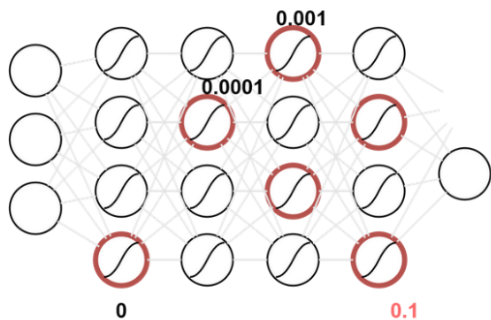
✓ 지식의 저장소

- 수집된 정보를 해석하고 처리하여 저장된 지식과 연결



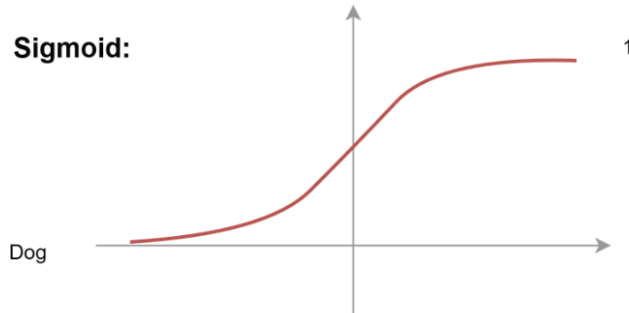
활성화 함수: Sigmoid

- ✓ 복잡한 패턴 학습 가능
- ✓ Layer가 많아질수록 Vanishing Gradient Problem 발생

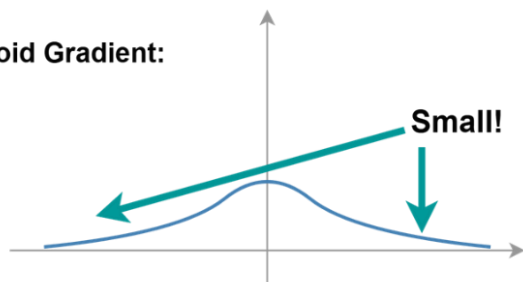


Vanishing Gradient Problem

Sigmoid:

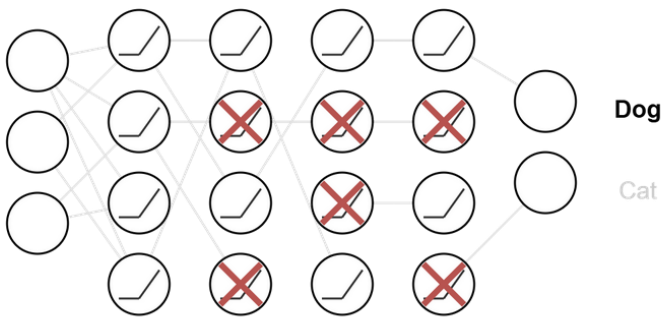


Sigmoid Gradient:



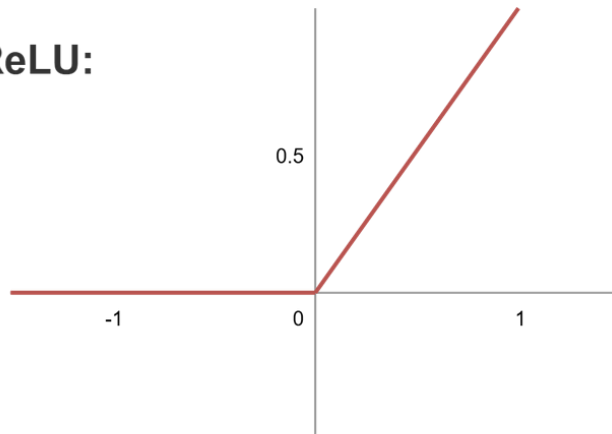
활성화 함수: ReLU

- ✓ Layer을 깊게 쌓을 수 있게 됨
- ✓ 비활성화되는 뉴런이 많아짐



Dying ReLU Problem

ReLU:

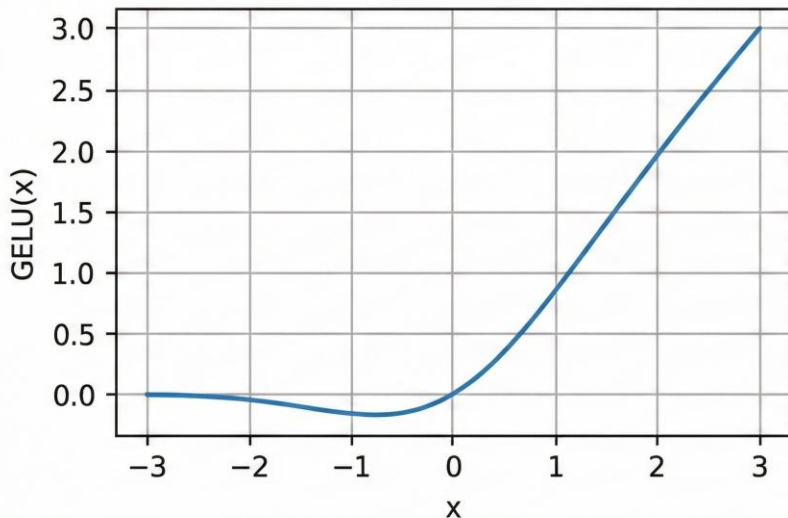


$$\begin{aligned} \text{ReLU}(Wx + b) &= 0 \\ \text{if } \max(Wx + b, 0) &= 0 \\ \text{or } Wx + b < 0 &\text{ or } b < -Wx \end{aligned}$$

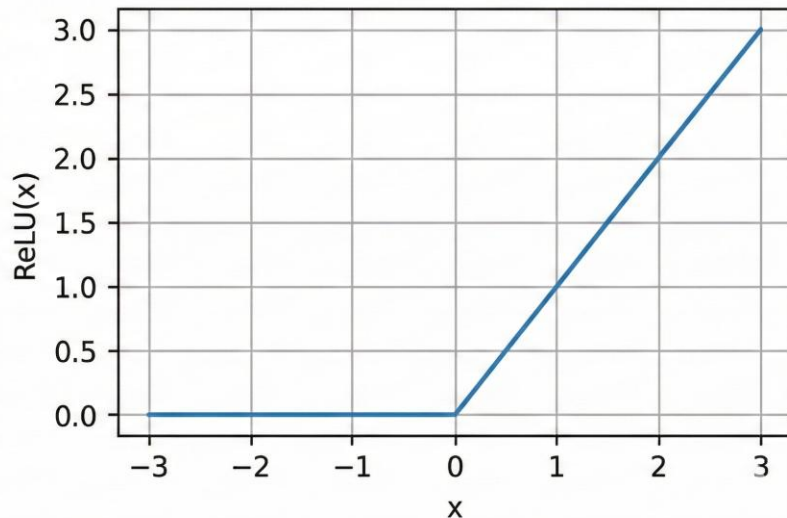
This happens with very negative bias when $W, x > 0$

- ✓ 좀 더 안정화된 학습이 가능

GELU activation function



ReLU activation function

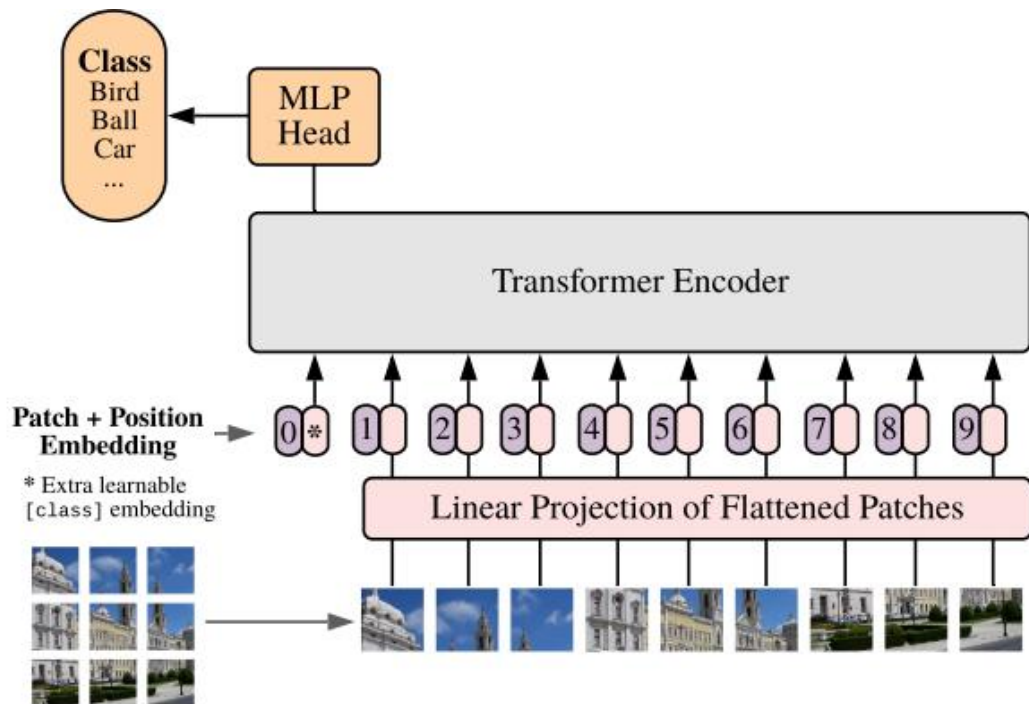


$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

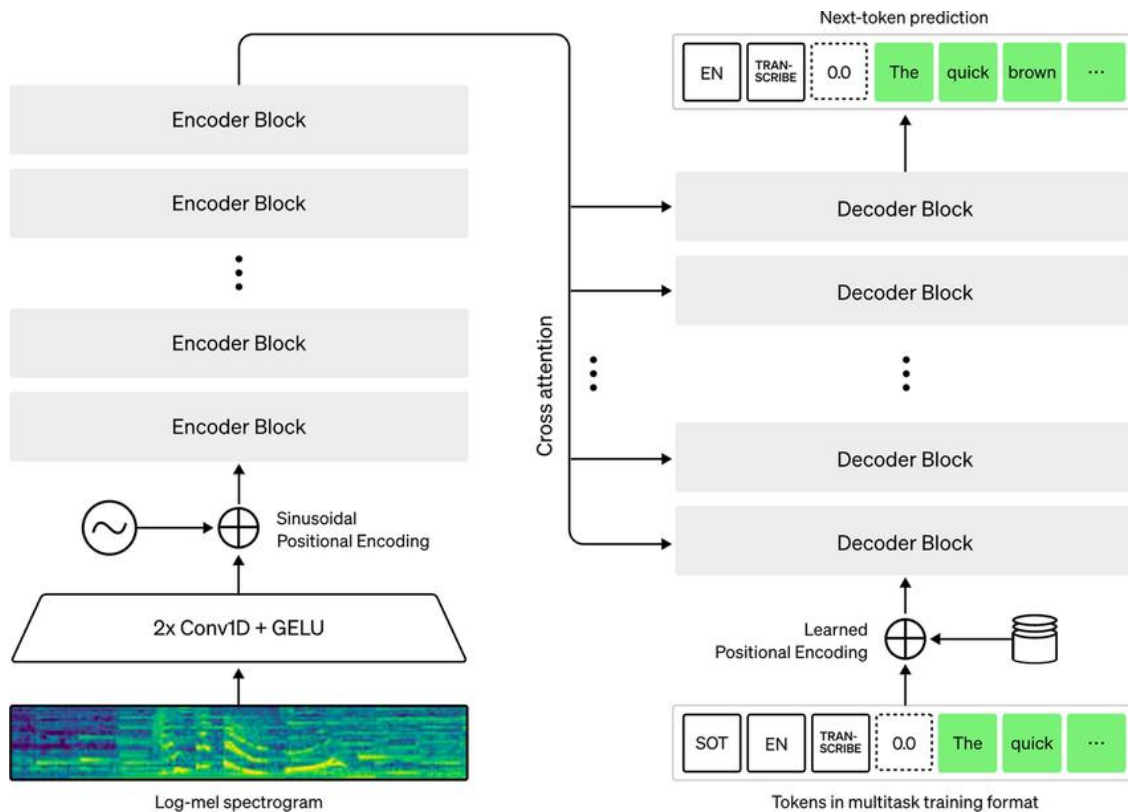
$$\text{ReLU}(x) = \max(0, x)$$

Vision Transformer

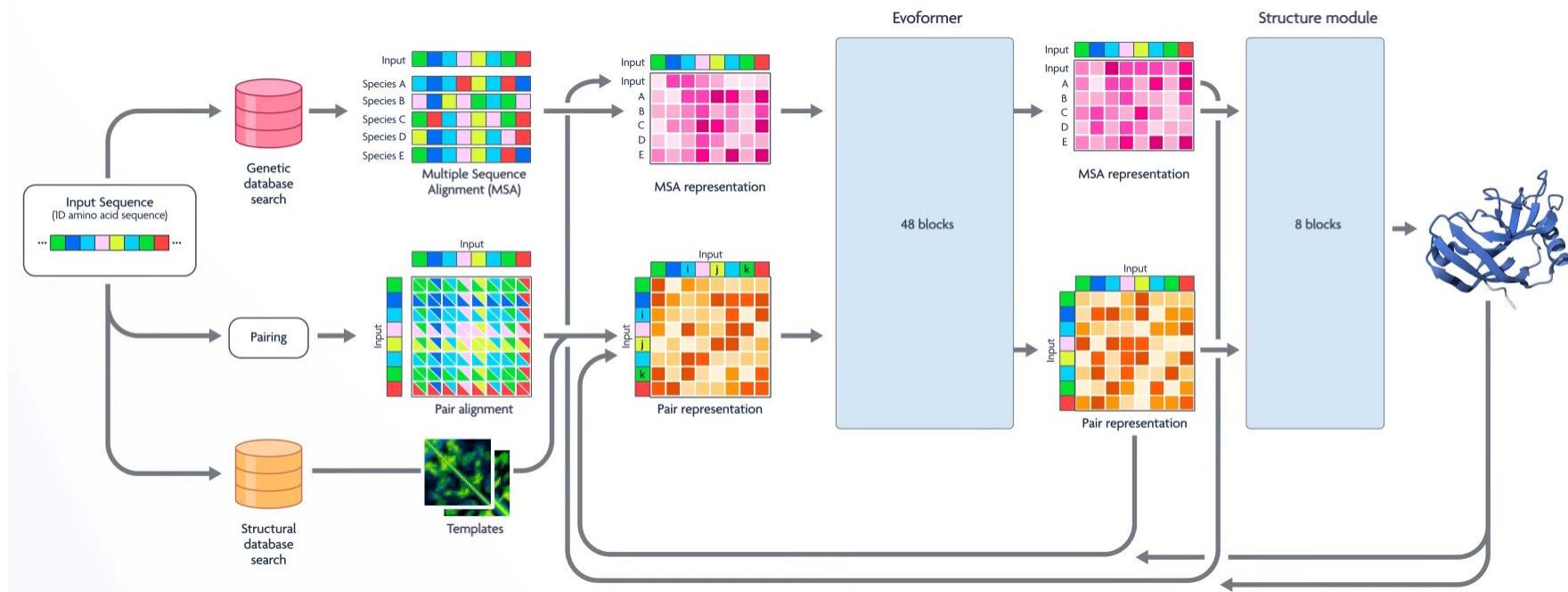
- ✓ 이미지 인식에 Transformer Encoder 사용



✓ 음성 인식의 Encoder와 Decoder에 사용

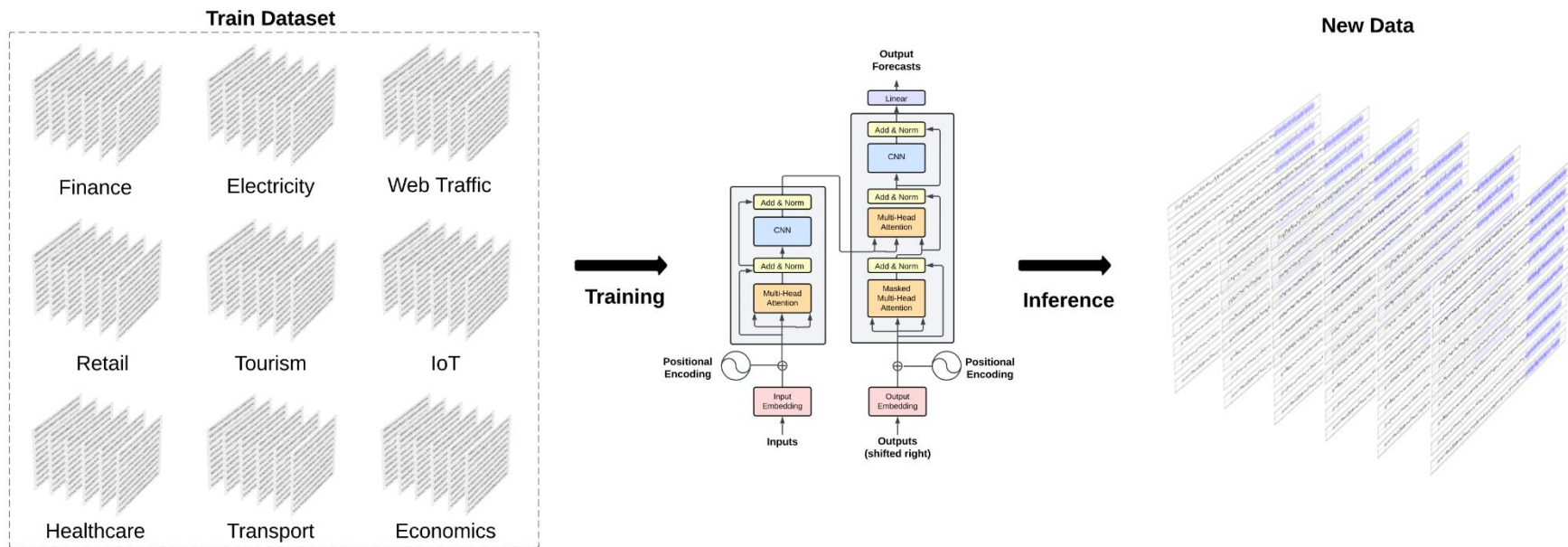


✓ 아미노산과 아미노산의 관계를 파악하는데 Transformer 사용

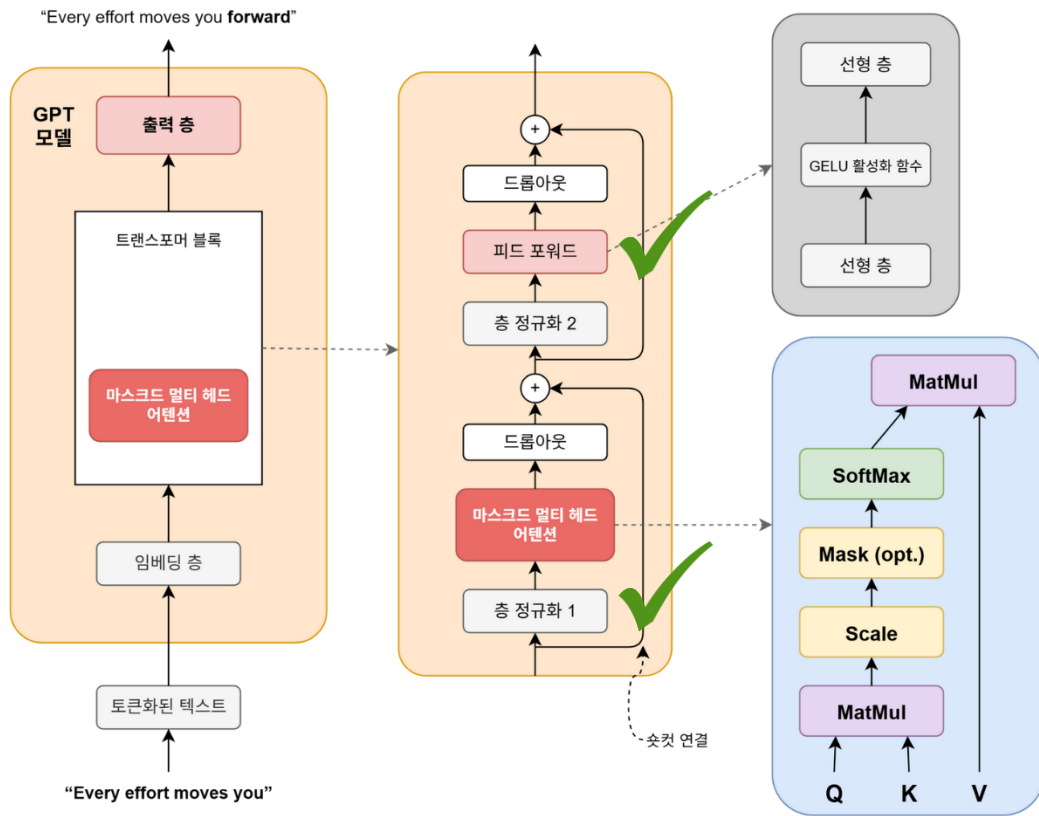


Time Series Prediction

- ✓ Token: 연속적인 실수 형태의 시계열 데이터를 미리 정해진 유한한 개수의 구간으로 나눔
- ✓ LLM 모델에 적용



언어 모델: GPT

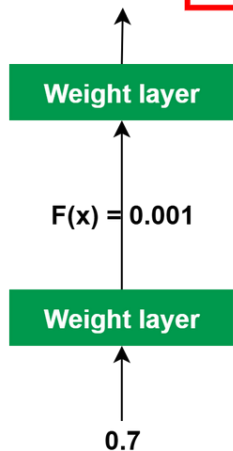


Residual connection

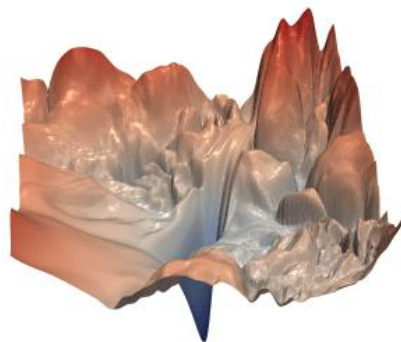
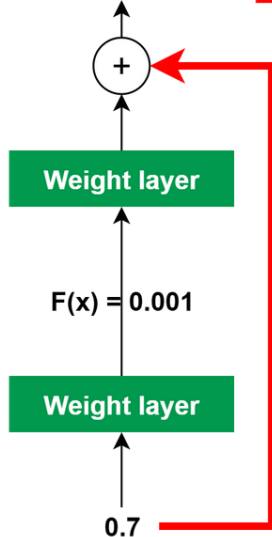
✓ 안정된 학습

- 엄청 깊다면 값의 변화가 크지 않을 것이다. 변화량만 학습하면 되니까 학습이 쉽다

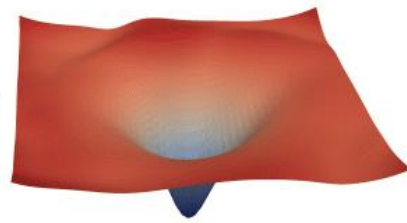
$$0.7 \times 0.001 = 0.007$$



$$0.7 \times 0.001 + 0.7 = 0.707$$



(a) without skip connections



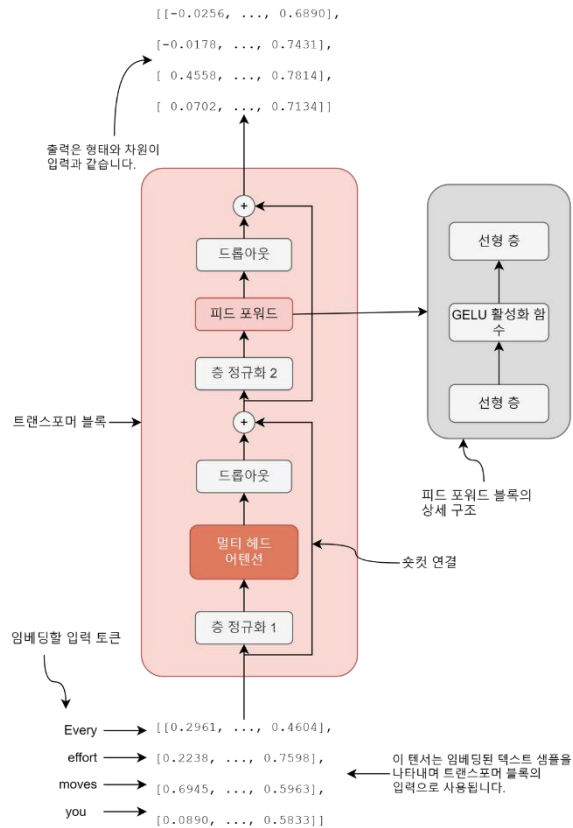
(b) with skip connections

[Intuitive Explanation of Skip Connections in Deep Learning | AI Summer](#)

GPT 모델 입출력

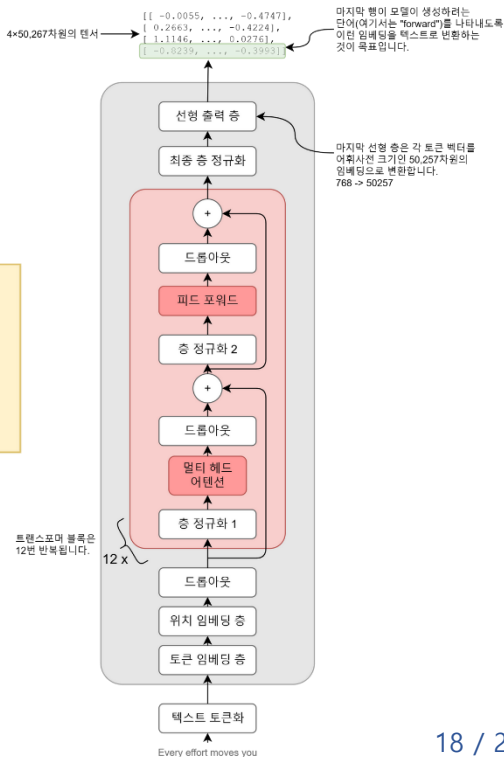
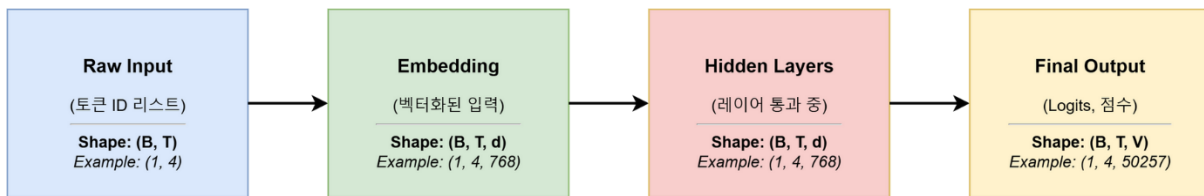
✓ Transformers 모듈의 입출력의 차원이 동일

- 모듈을 계속 쌓기 위함
- Residual connection 가능하도록 하기 위함



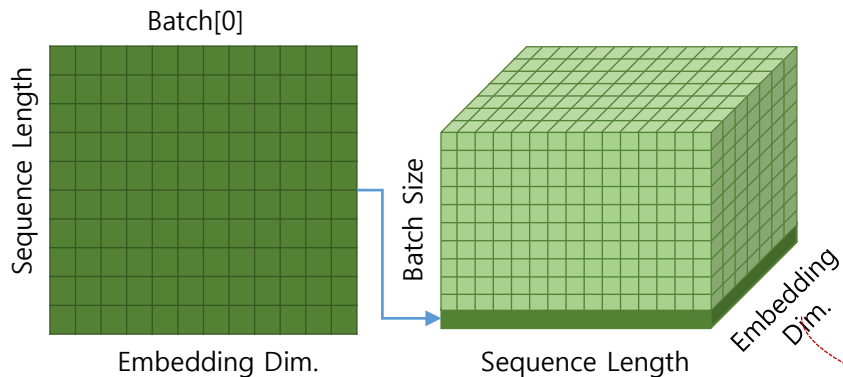
GPT 모델 입출력

- ✓ B(배치): 문장 1개 입력, T(길이): Token 4개, d (벡터 크기): 768, V (사전 크기): 50,257



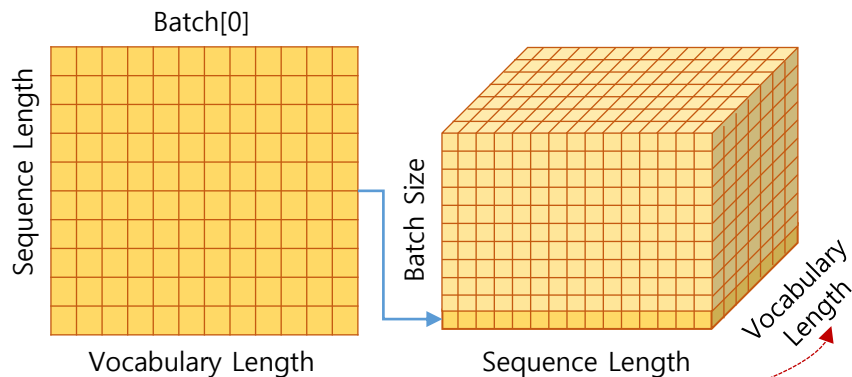
✓ Source Input Tensor

- Batch Size
- Sequence Length
- Embedding Dimension

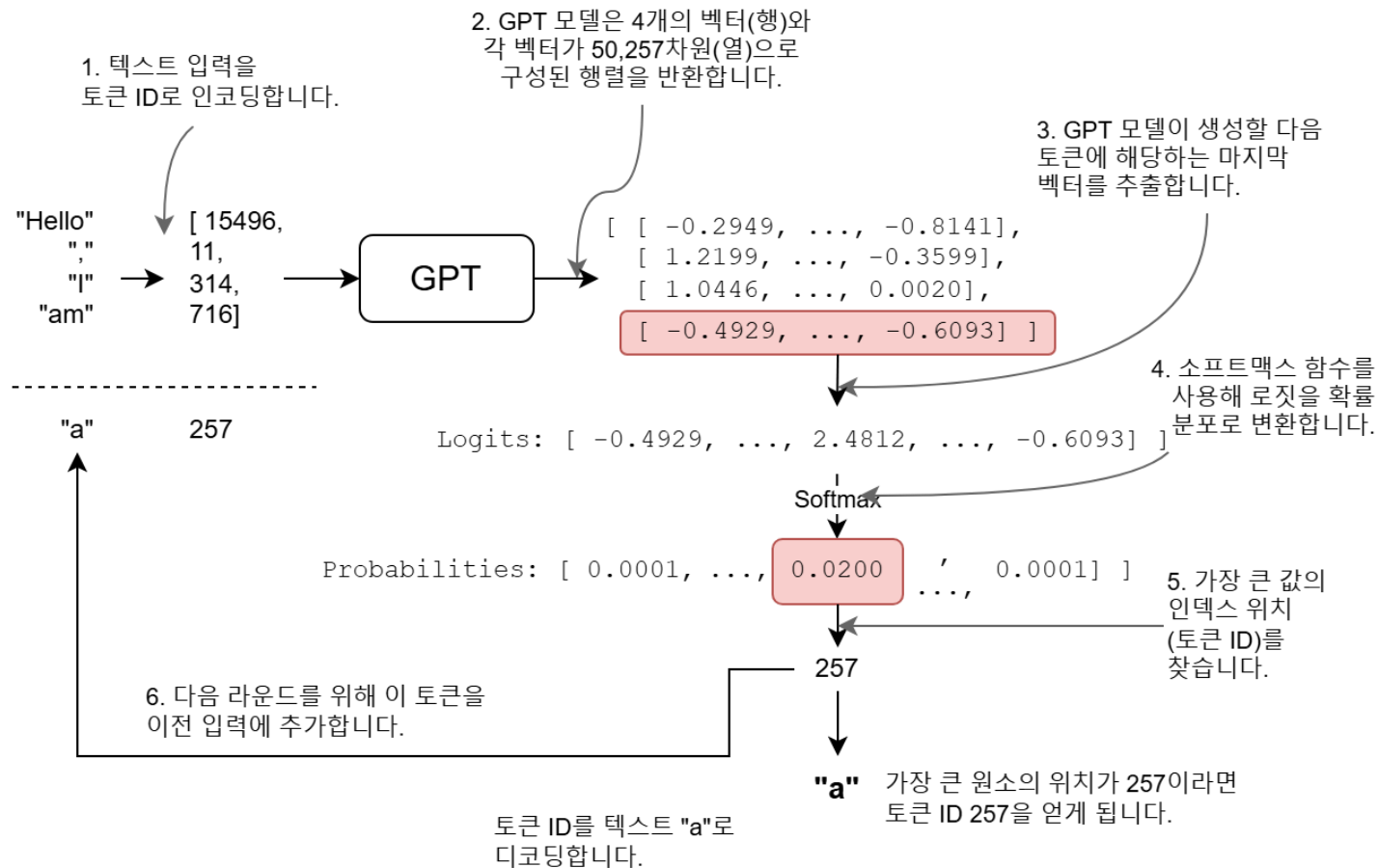


✓ Output Probabilities Tensor

- Batch Size
- Sequence Length
- Vocabulary Length.



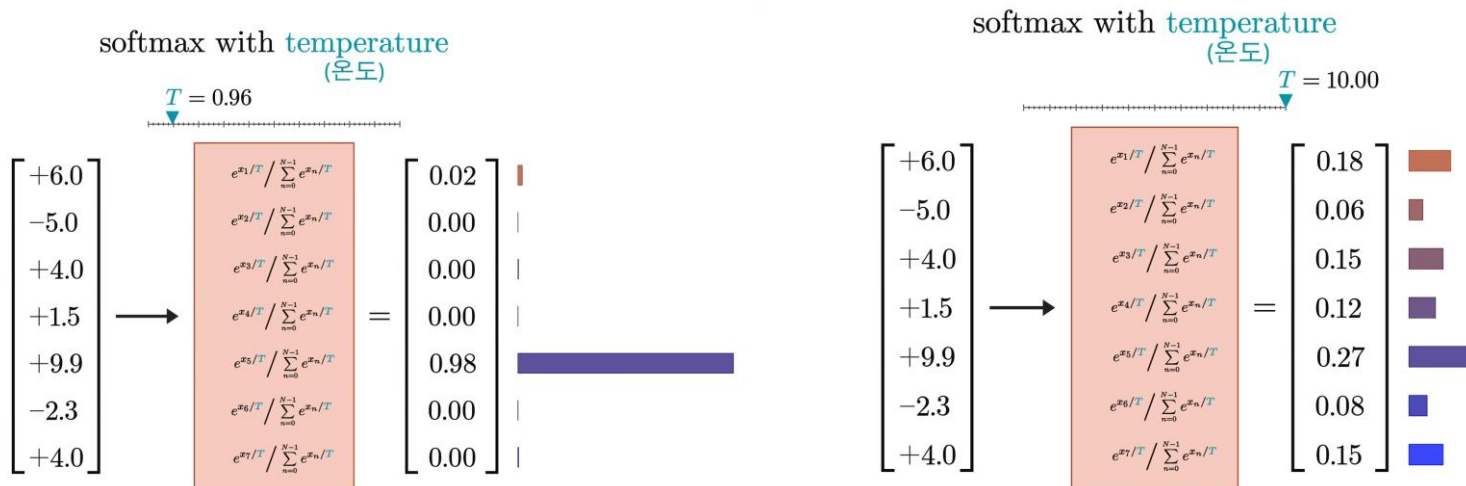
Output Token 생성 과정



Temperature(T)

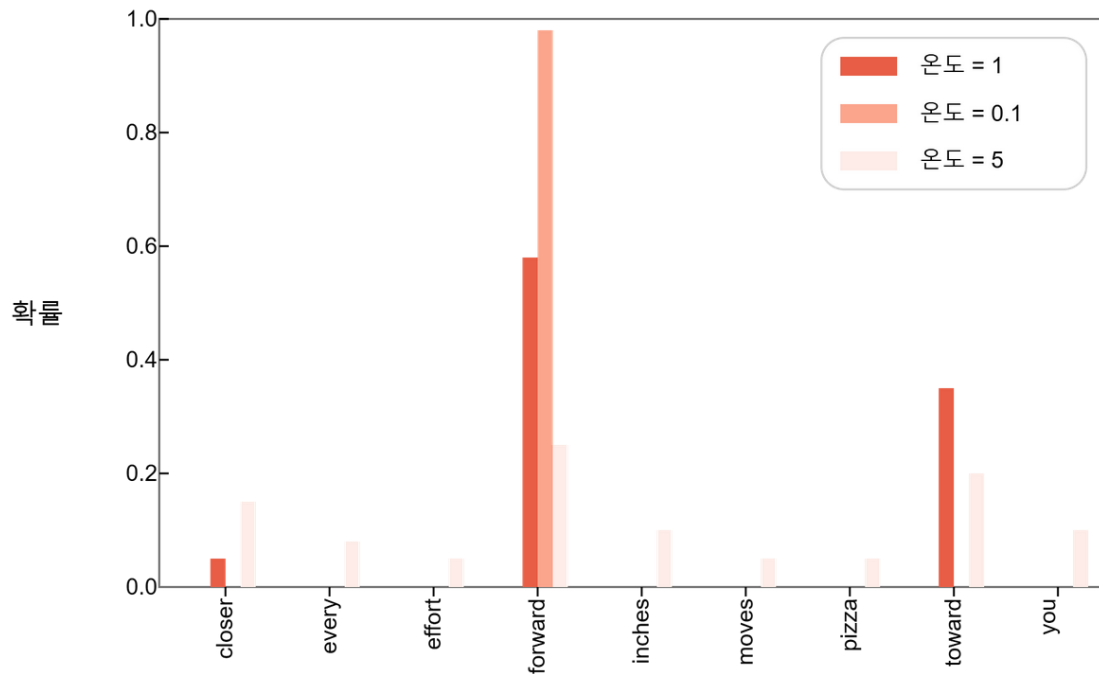
✓ T가 크면 더 작은 값들이 더 많은 비중을 차지하게 됨

- Softmax에서 T로 지수를 나눔



✓ 온도 스케일링

- 온도가 낮을수록 높은 확률이 확정적



Top-K 샘플링

- ✓ 샘플을 K만큼 고른 후 확률 기반으로 랜덤으로 선택

어휘사전:	"closer"	"every"	"effort"	"forward"	"inches"	"moves"	"pizza"	"toward"	"you"
인덱스:	0	1	2	3	4	5	6	7	8

로짓	= [4.51, 0.89, -1.90, 6.75, 1.63, -1.62, -1.89, 6.28, 1.79]								
↓									
탑-k (k = 3)	= [4.51, 0.89, -1.90, 6.75, 1.63, -1.62, -1.89, 6.28, 1.79]								
↓									
-inf 마스킹	= [4.51, -inf, -inf, 6.75, -inf, -inf, -inf, 6.28, -inf]								
↓									
소프트맥스	= [0.06, 0.00, 0.00, 0.57, 0.00, 0.00, 0.00, 0.36, 0.00]								

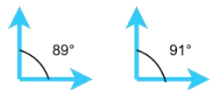


지식의 저장

FFN 지식의 저장

✓ 벡터 직교

- N차원에서 서로 직교하는 벡터는 N개
- 89-91도까지 허용하면 고차원일수록 서로 직교하는 벡터가 대다수임



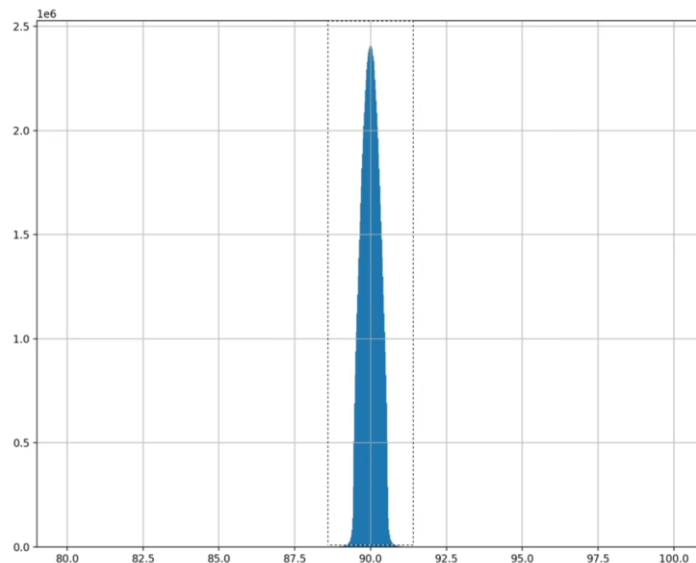
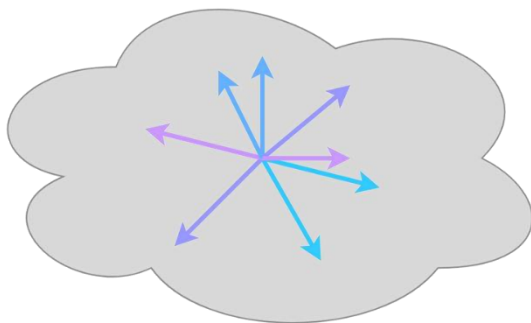
Choose multiple vectors,
each pair between 89° and 91° apart

Johnson-Lindenstrauss
Lemma



Maximum # of vectors: $\approx \exp(\epsilon \cdot N)$

N-dimensional
Space

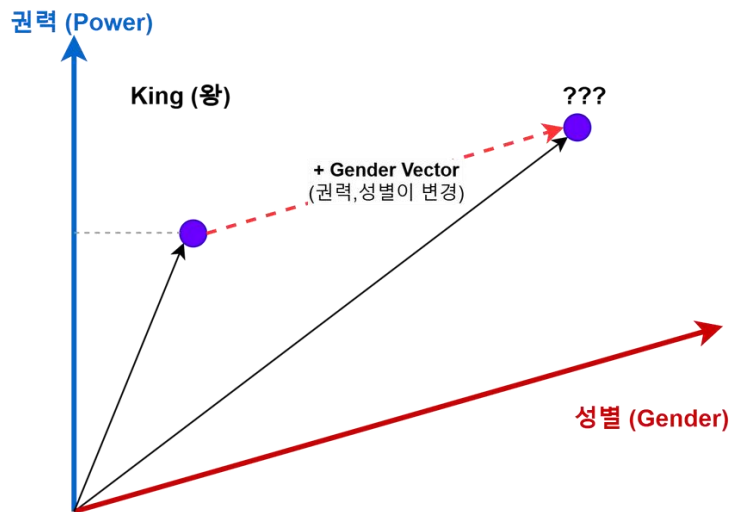
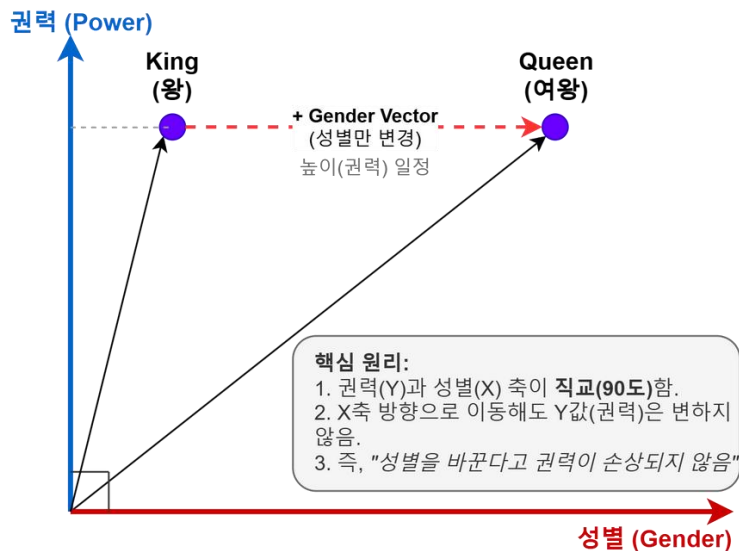


100만개 벡터를 서로
직교하는지 검사

FFN 지식의 저장

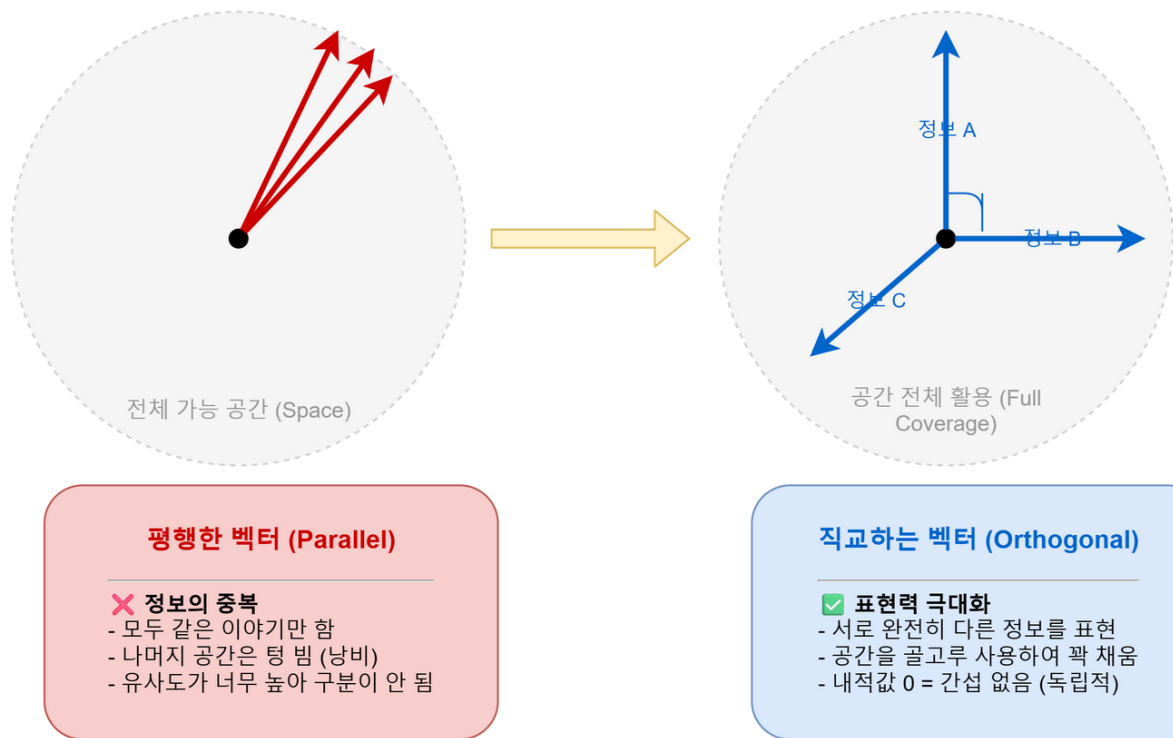
✓ 직교성(Orthogonality)과 특징 분리(Disentanglement)

- 개념들이 서로 간섭하지 않도록 함



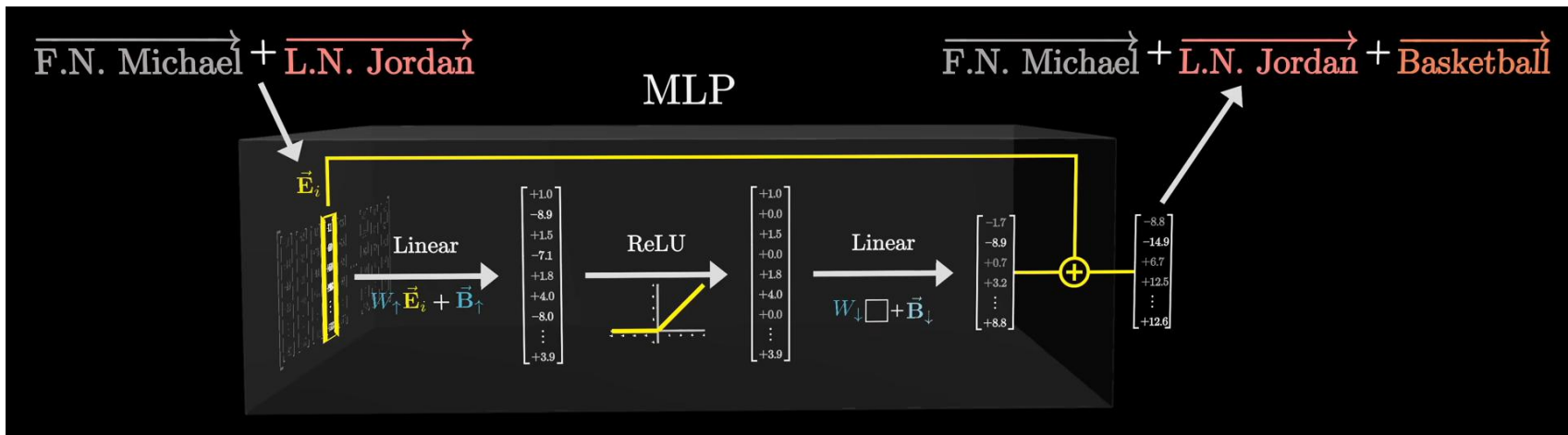
FFN 지식의 저장

✓ 정보 저장의 효율성



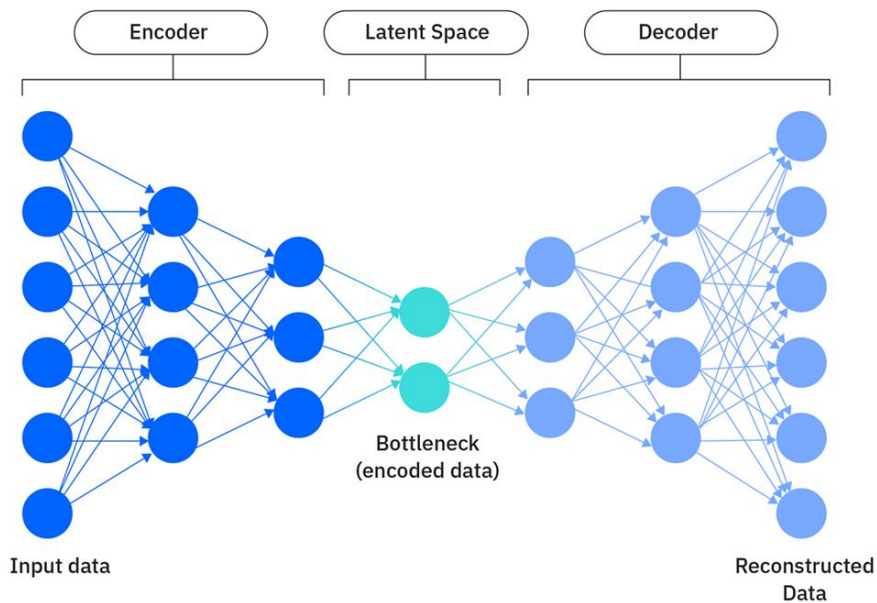
FFN 지식의 저장

- ✓ 수많은 Layer들이 정보를 저장하고 있으며, 다음 Layer로 가면서 새로운 정보를 만들



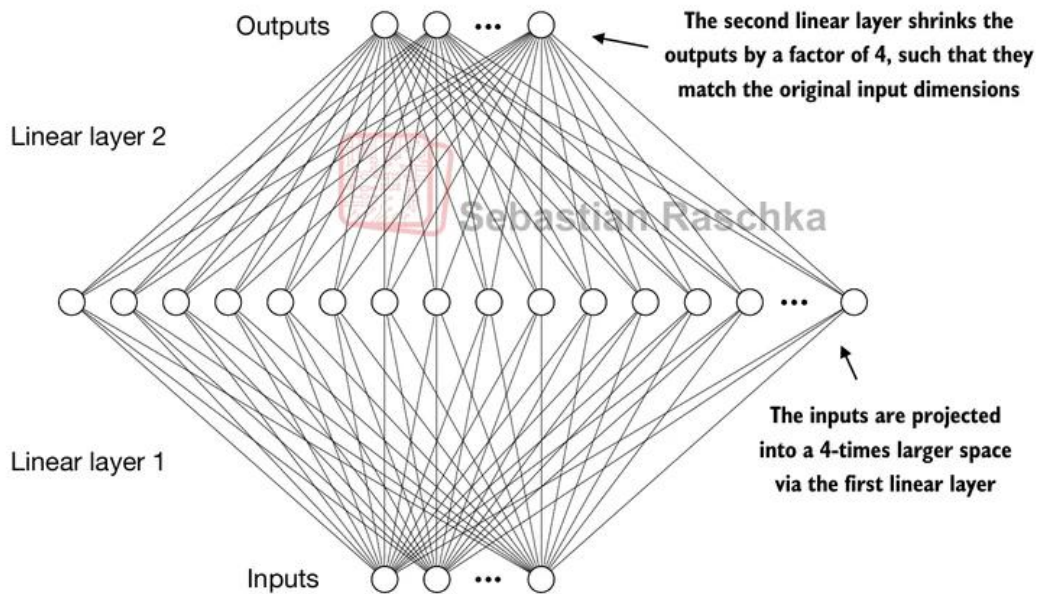
Latent Space

- ✓ 차원이 축소되어 정보가 압축되어서 저장되는 공간



Expansion Layer

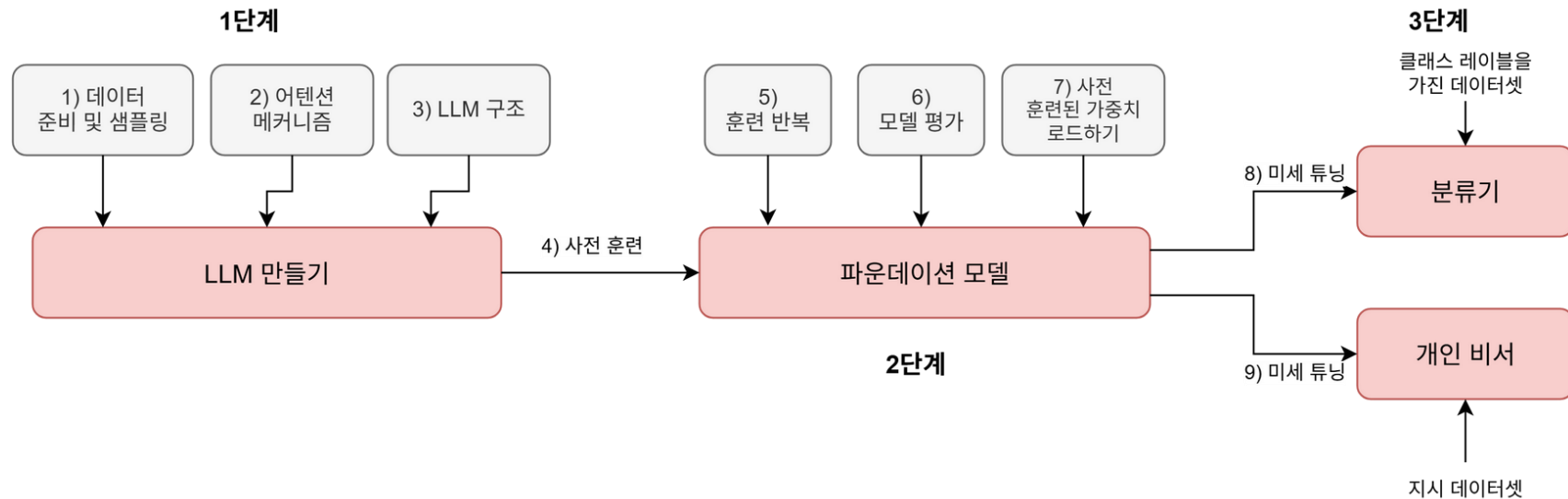
- ✓ 차원을 확대하여 특징을 세분화하여 저장, 활성화 함수에 의한 정보 손실 방지



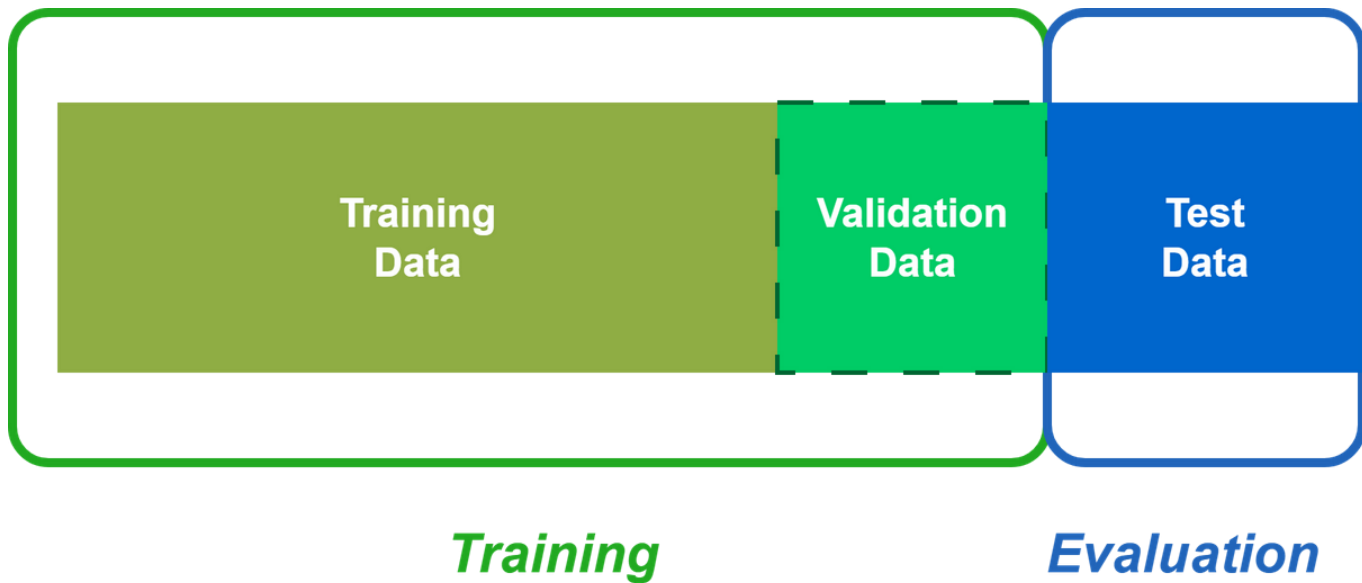
Chapter_4_Excercise_GPT.ipynb

학습

학습과정

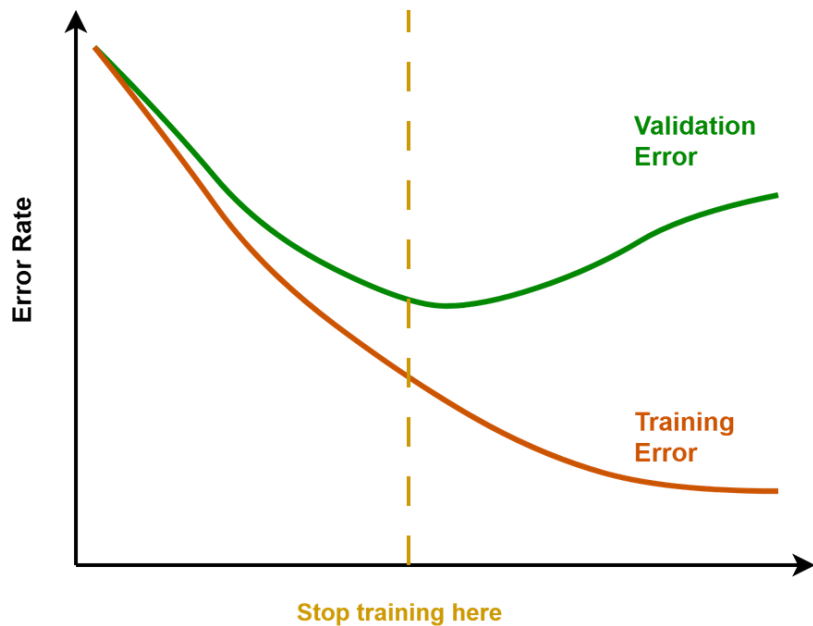


- ✓ Training과 Evaluation 분리

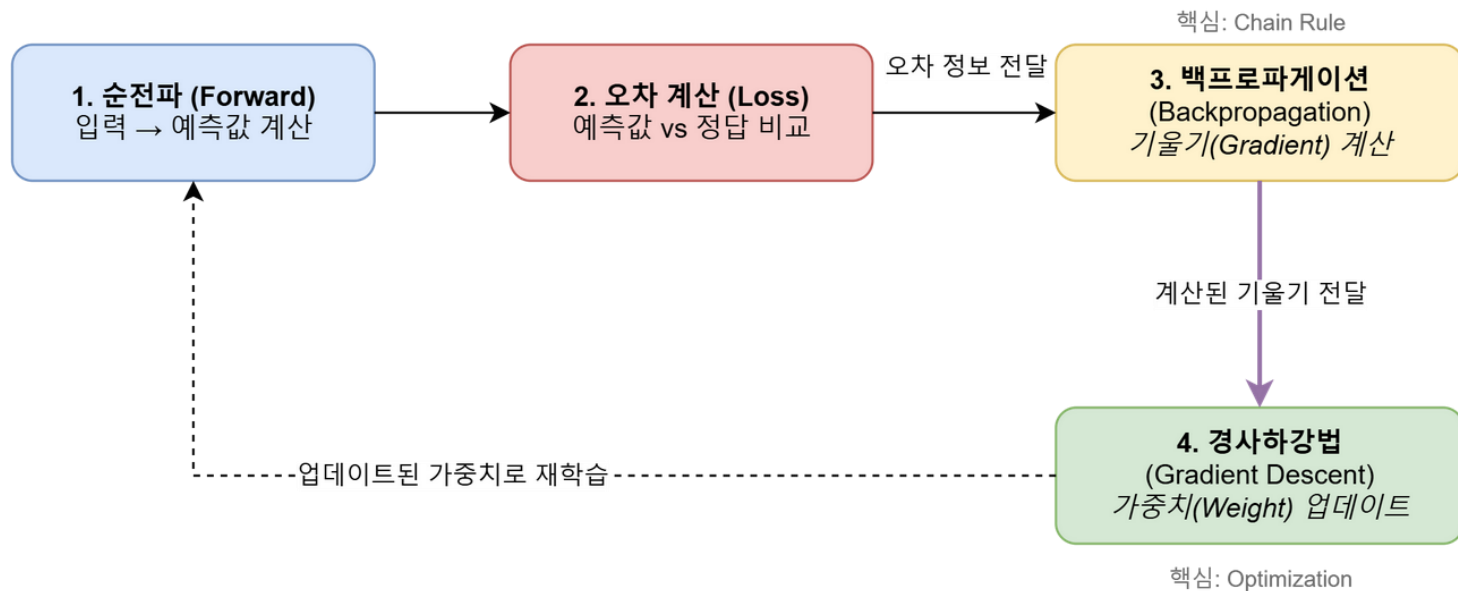


Validation Dataset

- ✓ overfitting 방지, hyperparameter tuning을 결정



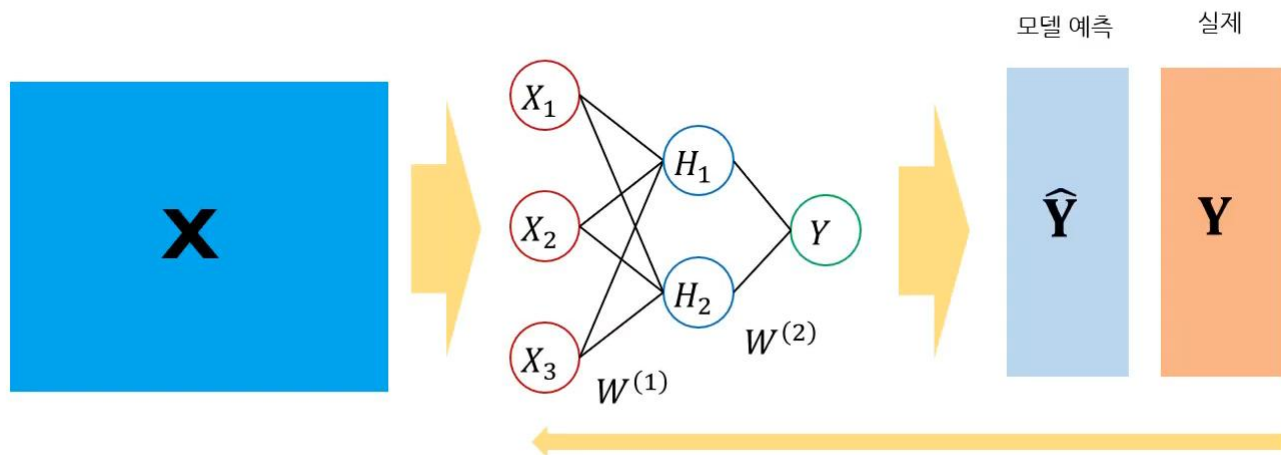
- ✓ Loss를 최소화하기 위해 가중치를 찾는 과정



비용함수

- ✓ 모델로부터 $\hat{Y}$ 값과 실제 Y 값의 차이를 최소화 하는 가중치

$$\arg \min_w \sum_i L(Y, f(X; w))$$



비용함수-수치예측(MSE)

✓ 예측한 값과 실제값의 차이를 측정

- Mean squared error (MSE) $C = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ (y_i : 실제값, $\hat{y}_i$: 예측값)

실제값	예측값	$\frac{1}{5} \sum_{i=1}^5 (y_i - \hat{y}_i)^2$
10	11	$= \frac{1}{5} [(10 - 11)^2 + (20 - 19)^2 + (30 - 30)^2 + (40 - 43)^2 + (50 - 47)^2]$ $= \frac{1}{5} (1 + 1 + 0 + 9 + 9)$ $= 4$
20	19	
30	30	
40	43	
50	47	

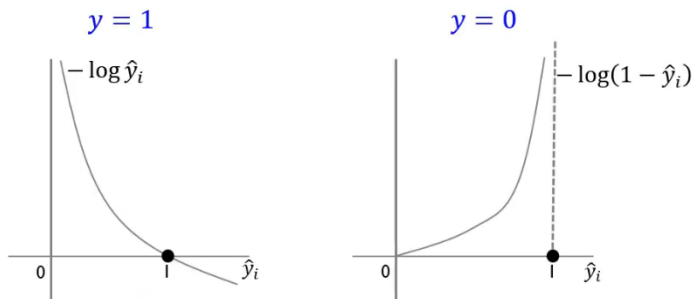
비용함수-Binary 분류(Cross Entropy)

✓ 0과 1로 분류하는 예제

0과 1은 분류하기 위한 라벨이므로 숫자로서 의미가 없음

Classification: cross entropy $L = -\sum_i y_i \log \hat{y}_i$ (y_i :실제값, $\hat{y}_i$:예측값)

$$C(y, \hat{y}_i) = \begin{cases} -\log \hat{y}_i, & y = 1 \\ -\log(1 - \hat{y}_i), & y = 0 \end{cases}$$



$$C(\pi(x), y) = -y \log \hat{y}_i - (1 - y) \log(1 - \hat{y}_i)$$

$$\min_{\beta} C(y, \hat{y}_i)$$

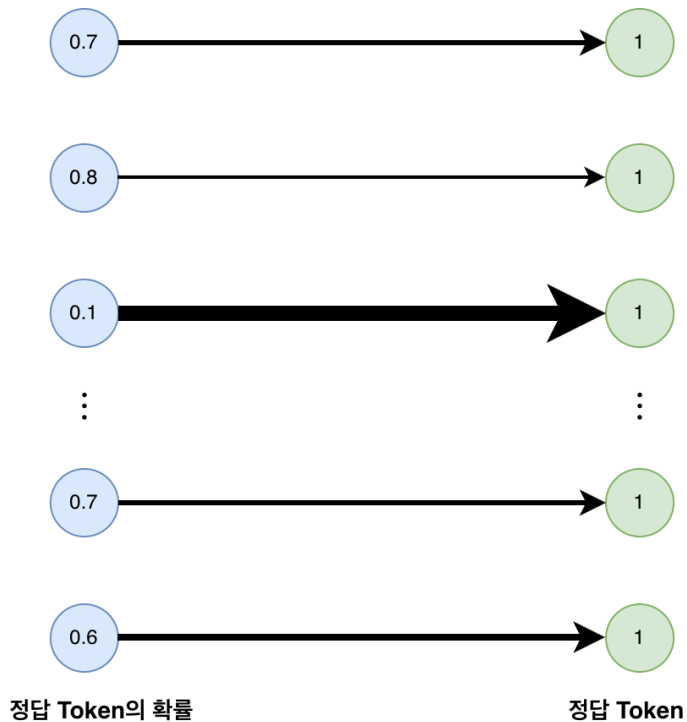
$$\mathcal{L}(Y, \hat{Y}) = \sum_{i=1}^N \{-y^{(i)} \cdot \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)})\}$$

Y		$\hat{Y}$
1	$l(y^{(1)}, \hat{y}^{(1)}) = -1 \cdot \log(0.99) \approx 0.01$	0.99
0	$l(y^{(2)}, \hat{y}^{(2)}) = -1 \cdot \log(0.89) \approx 0.12$	0.11
1	$l(y^{(3)}, \hat{y}^{(3)}) = -1 \cdot \log(0.50) \approx 0.69$	0.50
1	$l(y^{(4)}, \hat{y}^{(4)}) = -1 \cdot \log(0.05) \approx 2.99$	0.05
0	$l(y^{(5)}, \hat{y}^{(5)}) = -1 \cdot \log(0.25) \approx 1.39$	0.75

너가 제일 많이 틀렸어

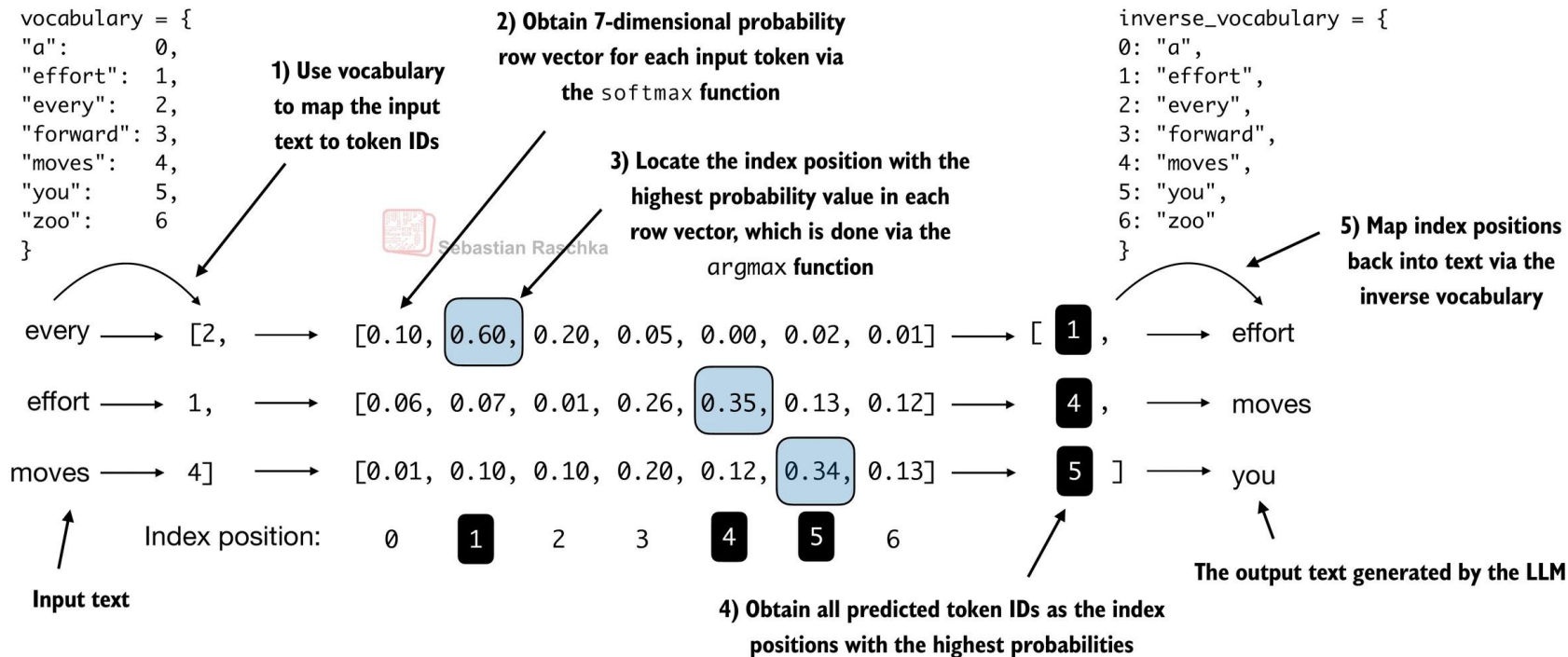
비용함수-분류(Cross Entropy)

- ✓ 정답 Token의 확률 합: 모두가 1이 되면 됨



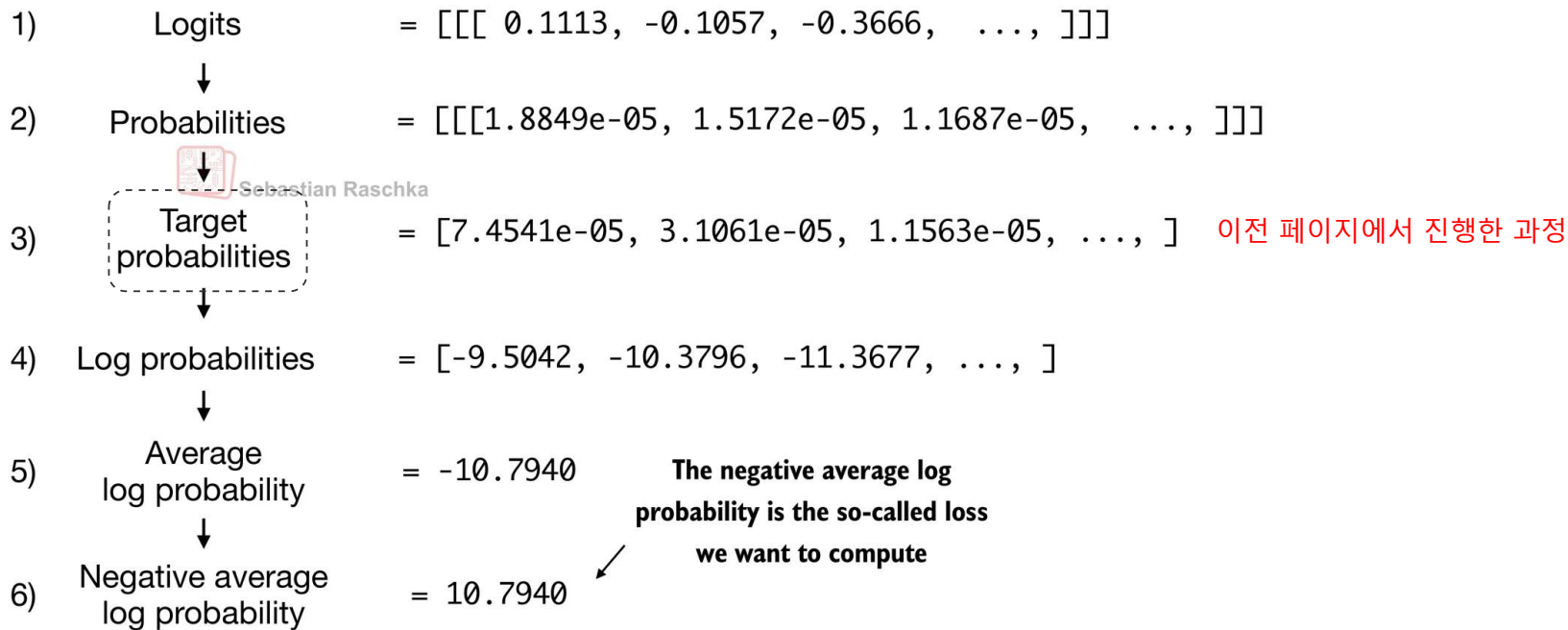
비용함수-분류(Cross Entropy)

✓ 정답 Token의 확률 합을 가져옴



비용함수-계산 과정

- ✓ 정답 Token들의 로그 확률을 구해서 평균을 구한 후 -부여



경사하강법(Gradient Descent)

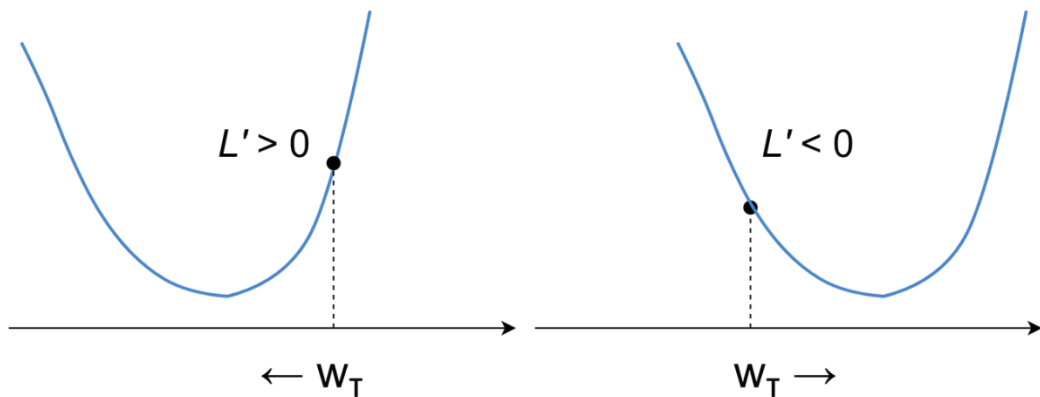
✓ Loss를 최소화하기 위해 가중치를 찾는 과정

- Optimizer가 학습 과정에서 스스로 조율해 나감

$$w_{T+1} = w_T - \alpha \cdot L'(w_T)$$

$0 < \alpha < 1$: learning rate

좀 더 섬세하고 촘촘히? **Small α**
좀 더 빠르게? **Large α**

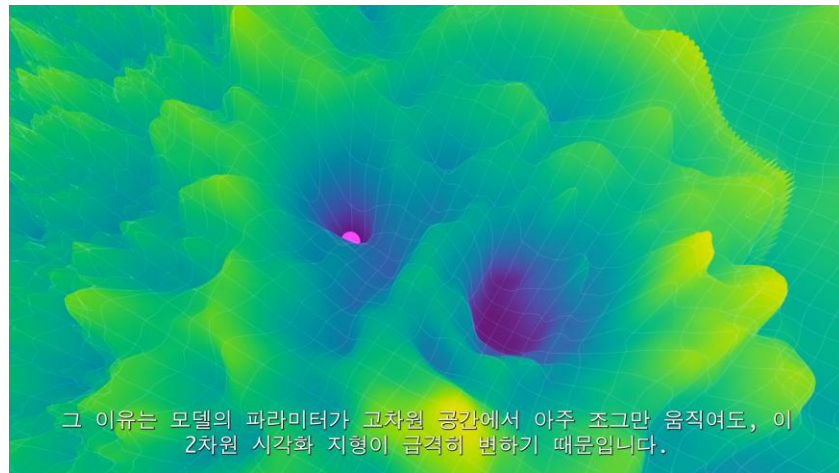
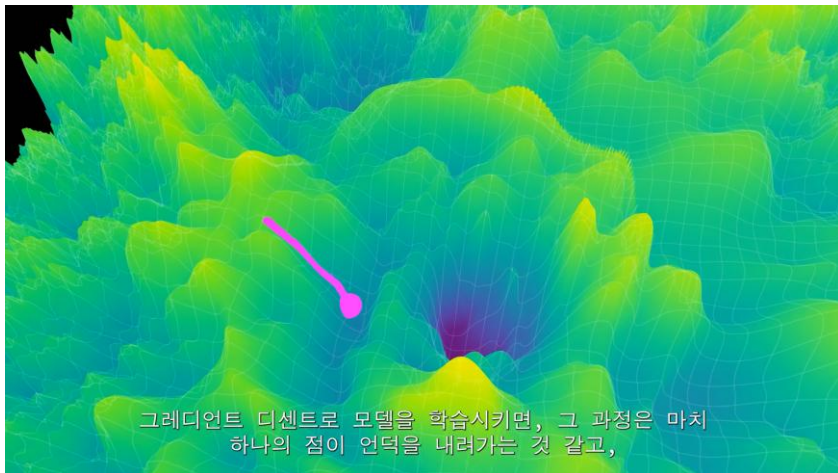


[Neural Network 9] 역전파
backpropagation 알고리즘
(수정본)

LOCAL MINIMUM

✓ 고차원에서는 발생하지 않음

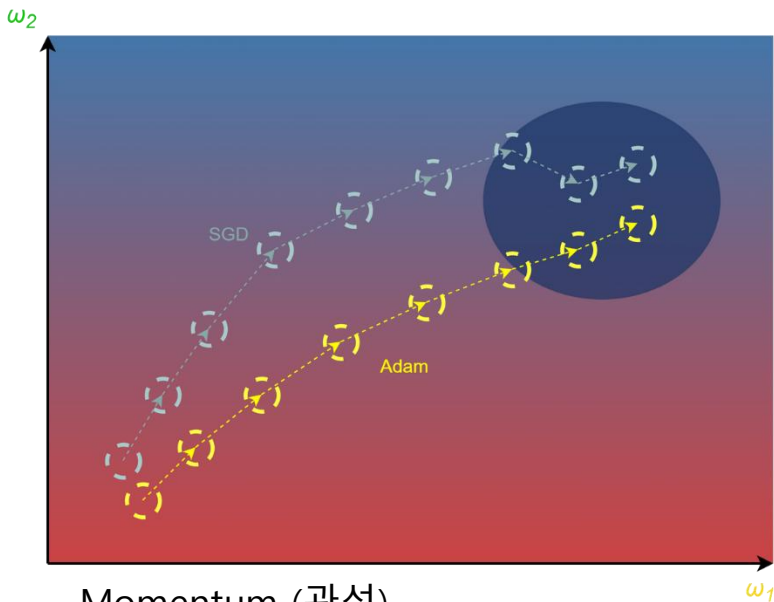
- 기울기 소실이 문제(너무 평탄해서 어디로 가야할 지 모르게 되는 경우)



[AI는 왜 아직도 '내리막길'을 걷는가? | 경사하강법과 지역 극솟값 - YouTube](#)

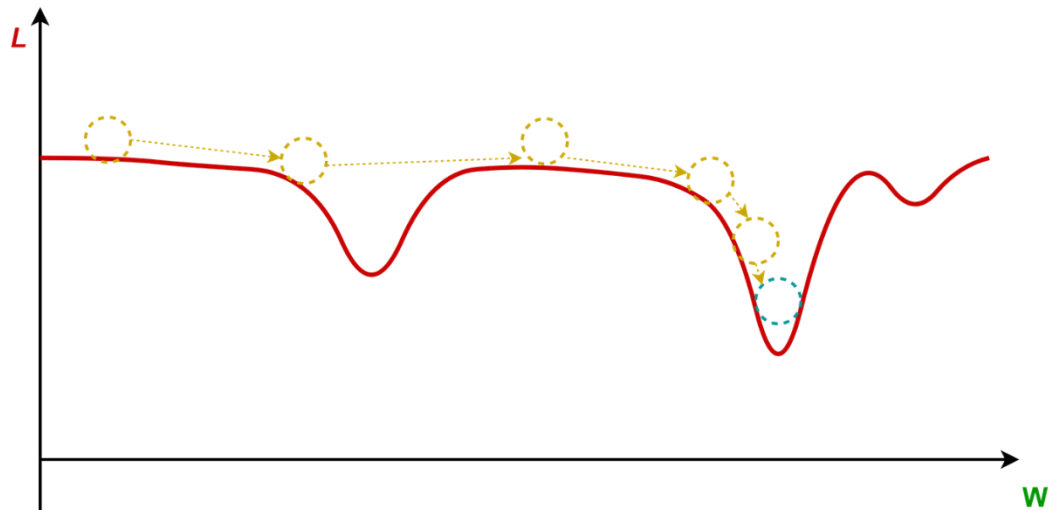
Adam Optimizer

- ✓ 과거의 기울기 변화량(Moment)을 추정하여, 각 파라미터마다 학습률을 다르게 조절



Momentum (관성)

기울기가 가리키는 방향으로 속도를 붙여서 가자



RMSprop (적응형 학습률)

보폭(Step size)을 상황에 맞춰 조절하자.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

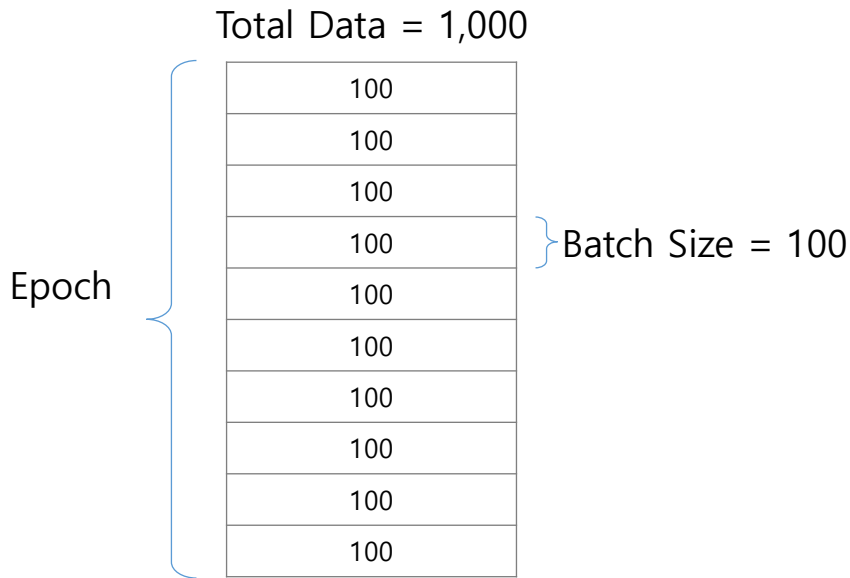
Learning Rate $\rightarrow \eta$

관성: 방향 $\rightarrow \hat{m}_t$

속도: 보폭 $\rightarrow \sqrt{\hat{v}_t}$

그래디언트가 컸던 매개변수는 $\sqrt{\hat{v}_t}$ 로 나누어 보폭을 줄이고, 작았던 매개변수는 보폭을 키워 안정적으로 수렴하게 합니다.

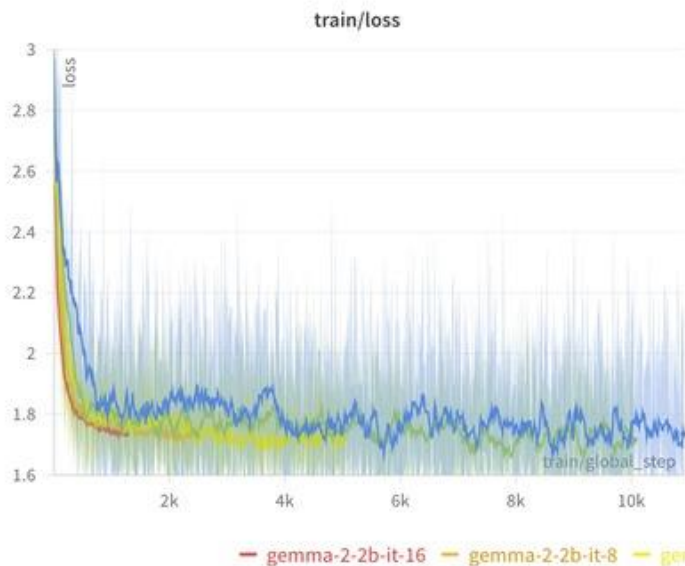
Epoch, Step, Batch Size



- **Epoch**: 전체 데이터를 1회 학습하는 것
 - **Step (or Iteration)**: 파라미터를 1회 업데이트하는 것
 - **Batch Size**: Step 1회에서 사용하는 데이터 개수
-
- 전체 데이터: 1,000
 - Batch Size: 100
 - 한 Epoch에 10번의 Step이 필요

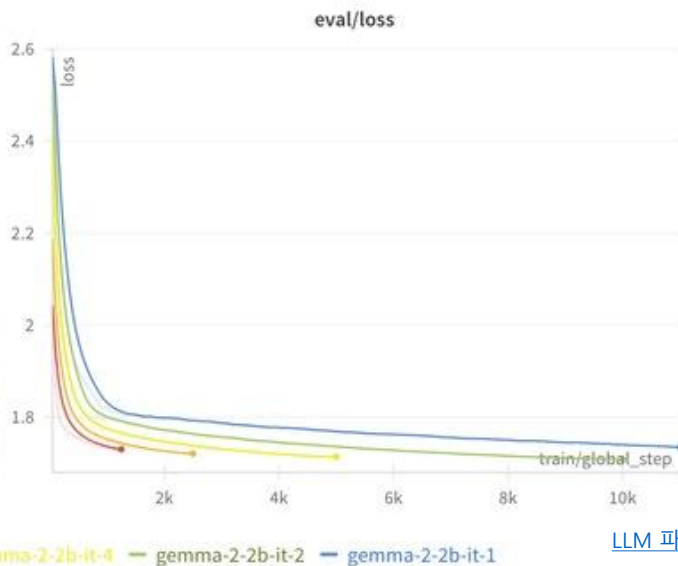
✓ 작은 배치 크기

- 학습 과정이 불안정해질 수 있지만, 일반화 성능은 더 좋아지는 경향이 있습니다.
- 메모리 사용량이 적어 제한된 자원으로 학습 가능하지만, 전체 학습 시간이 길어질 수 있습니다.



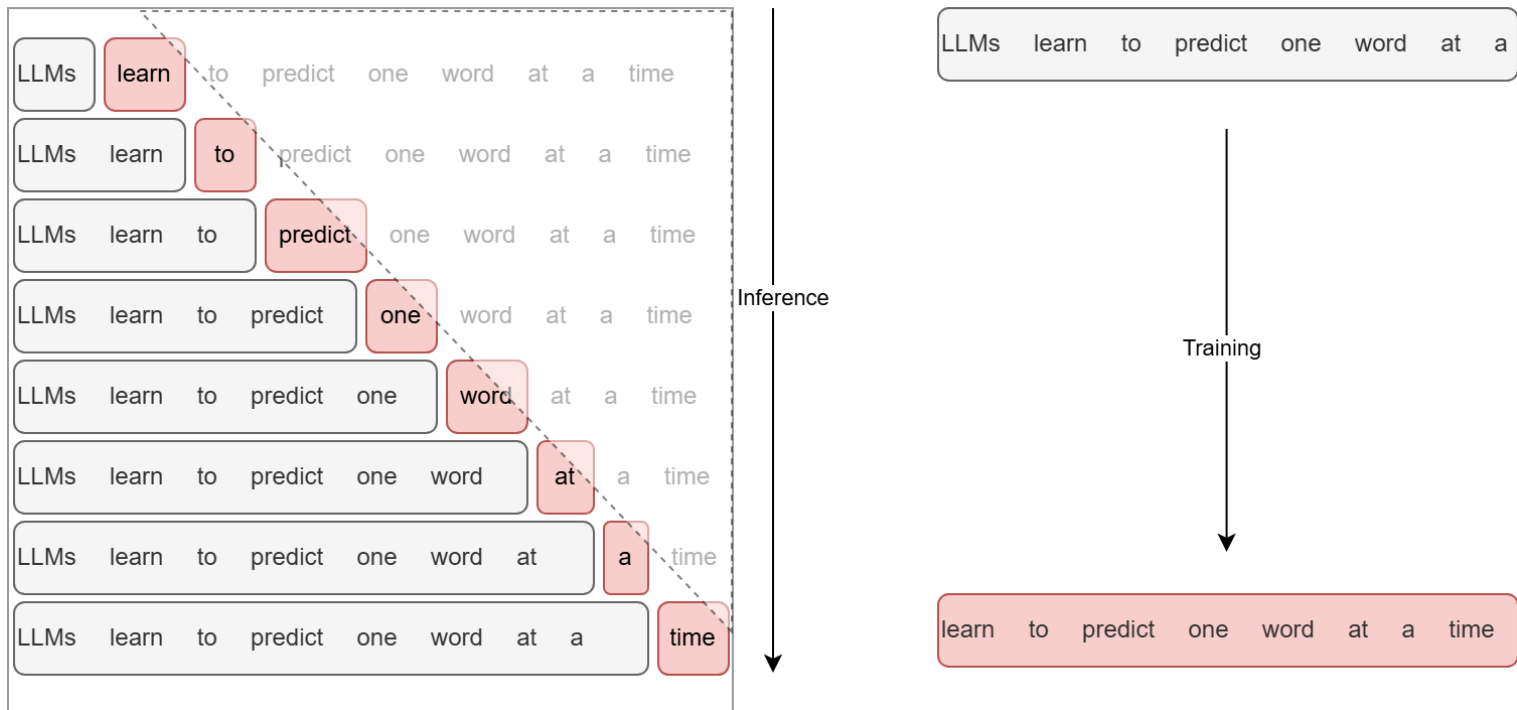
✓ 큰 배치 크기

- 학습이 안정적이지만, 세부적인 특징을 놓쳐 과적합 (Overfitting)의 위험이 있습니다.
- 더 많은 메모리가 필요하지만 학습 시간이 더 단축시킬 수 있습니다.



Inference vs Training

- ✓ Training은 한번에, Inference는 Autoregressive로(Masking으로 인해 두 과정이 동일)



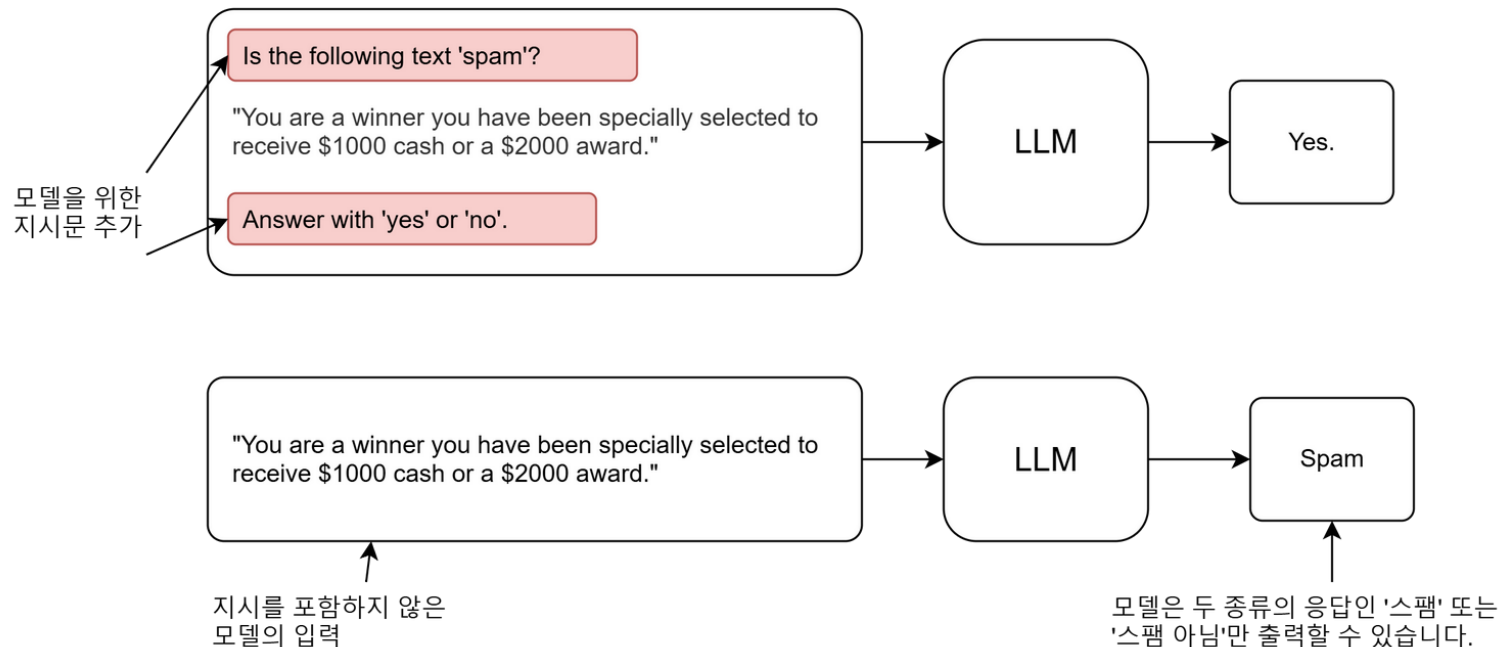
Chapter_5_Excercise_Pretraining.ipynb



Fine-tuning

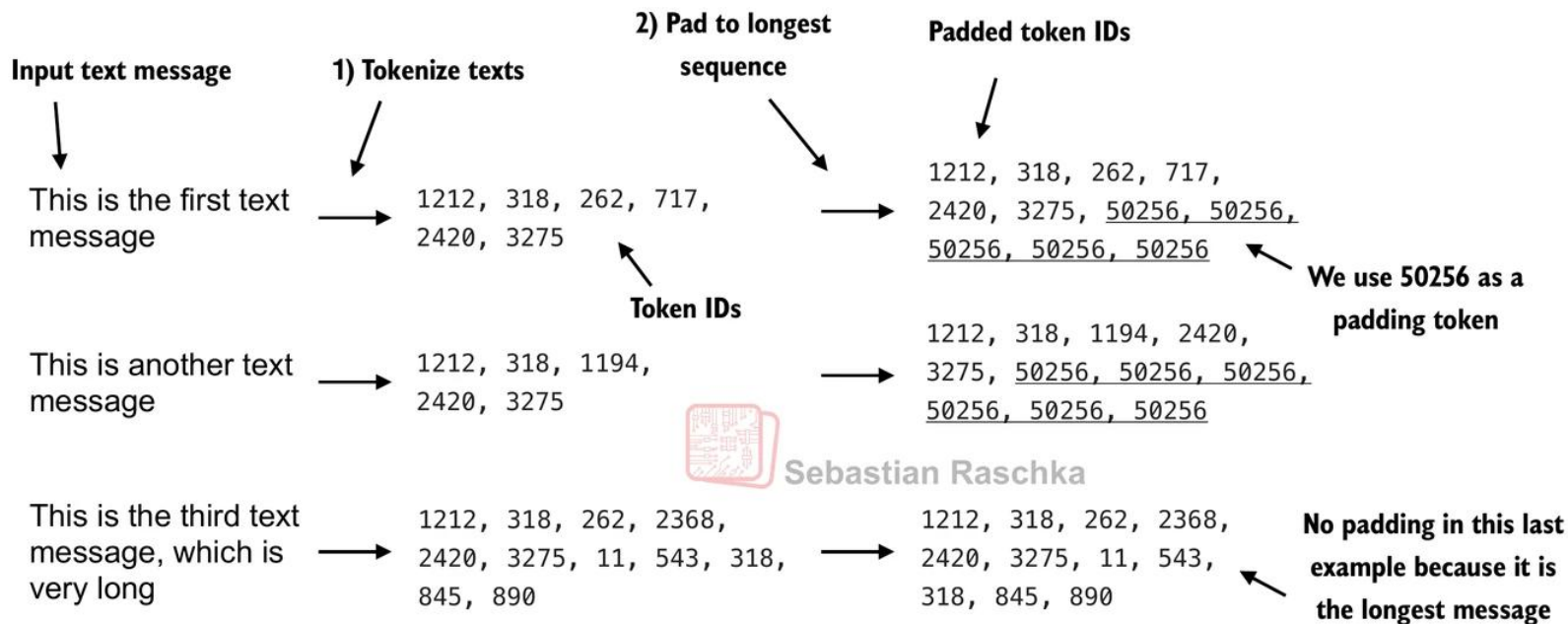
분류 미세 튜닝

- 분류를 위해 마지막 layer 수정



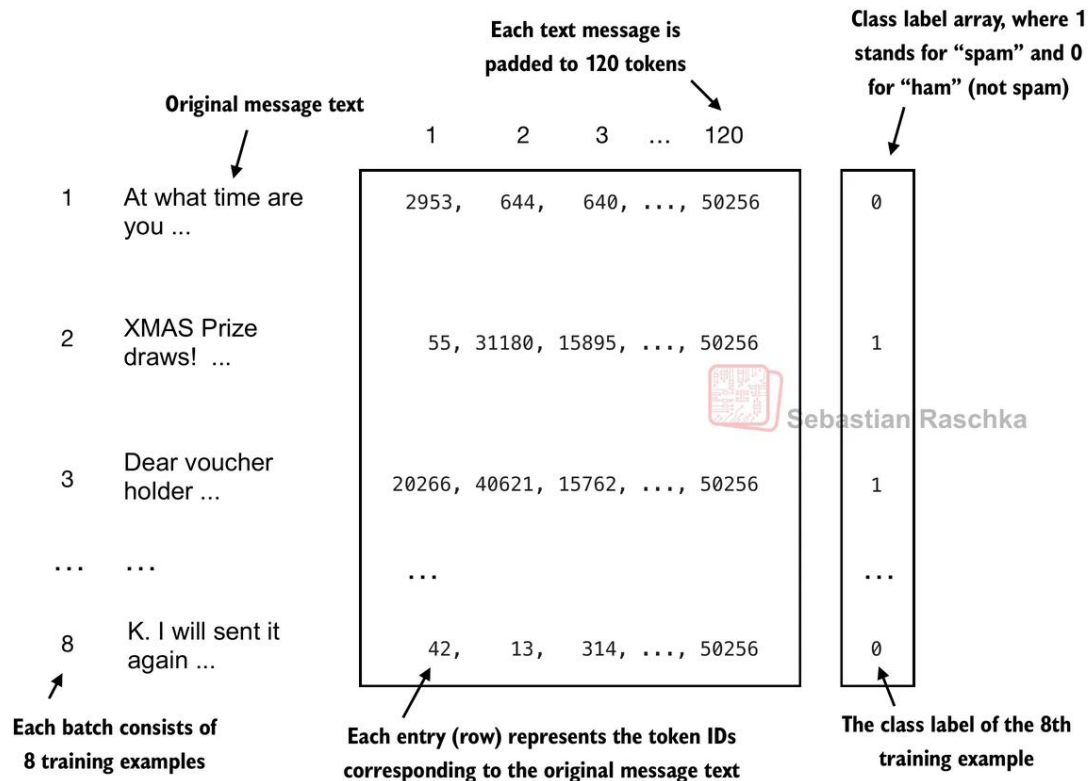
분류 미세 튜닝

- Input 데이터가 짧은 경우 Padding 필요



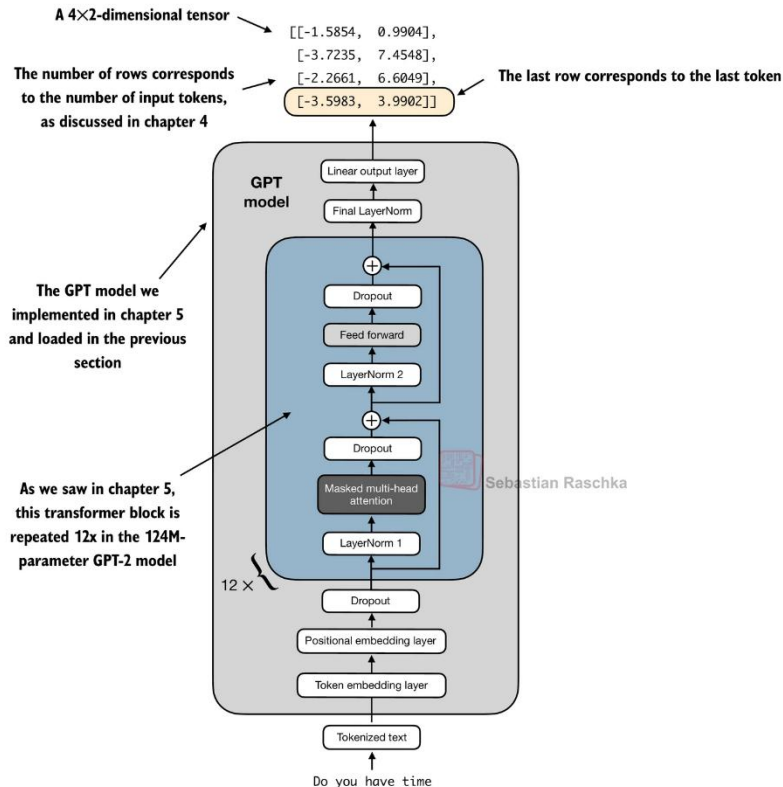
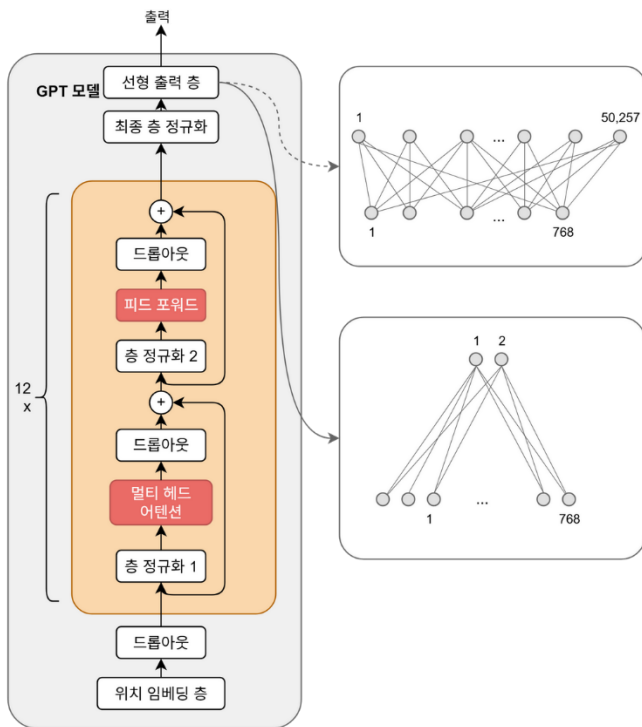
분류 미세 튜닝

- Target은 0,1로 라벨링



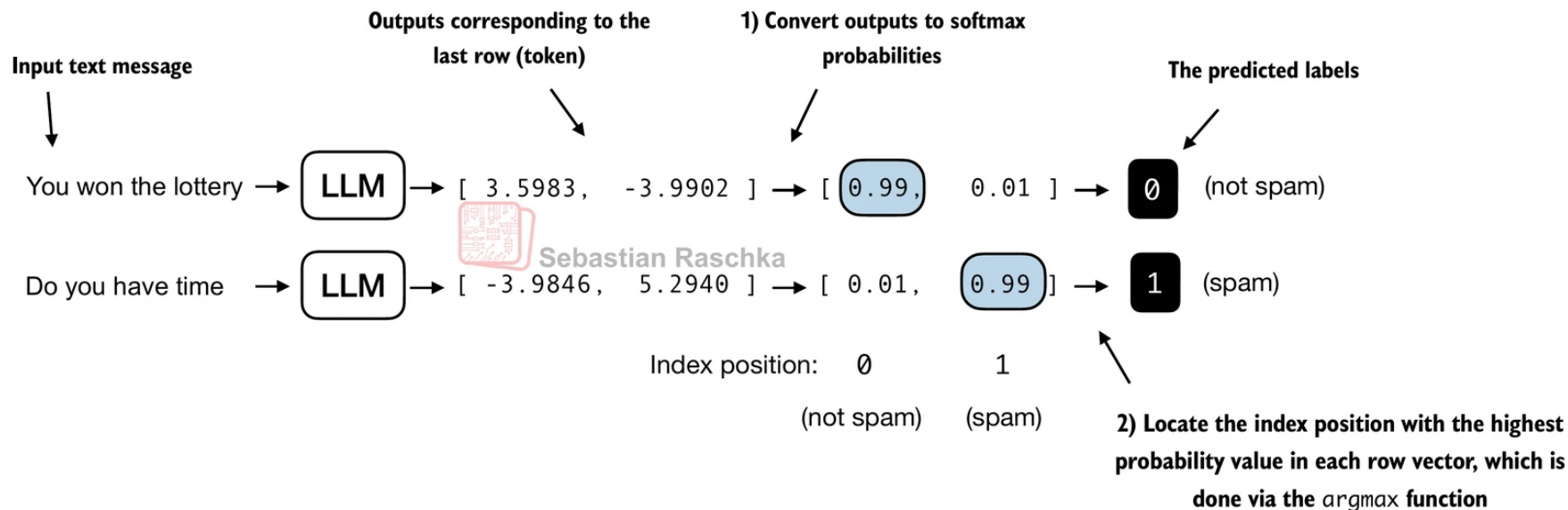
분류 미세 튜닝

- 분류를 위해 마지막 layer 수정



분류 미세 튜닝

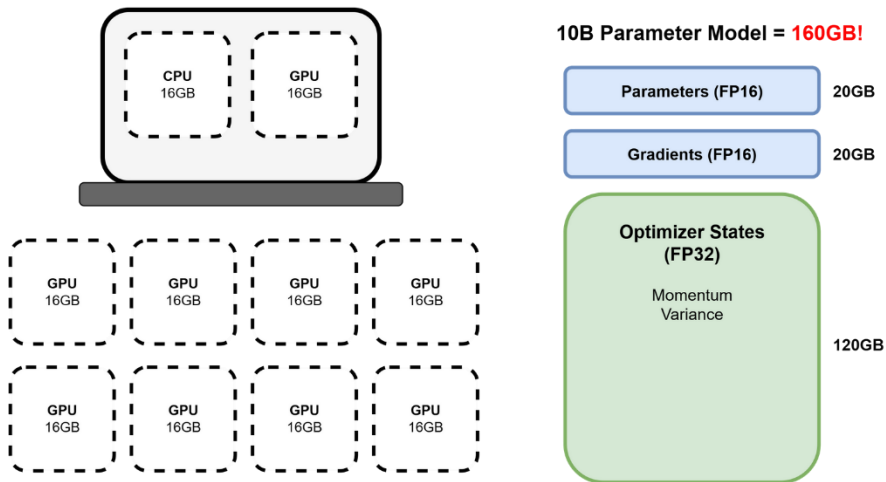
- 결과는 softmax 후 argmax로 선택



Chapter_6_Excercise_Finetuning_Classification.ipynb

✓ LLM 학습의 문제

- LLM 학습은 비싸다, 많은 메모리 요구
- 10B의 모델 학습에 160GB 메모리 필요



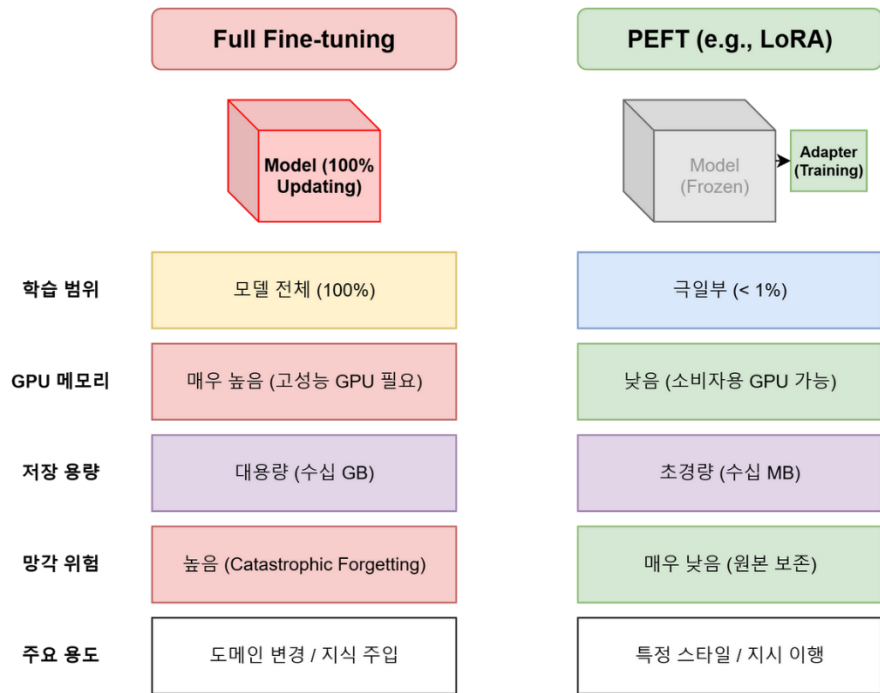
✓ LoRA: Percent of Total Parameters

- 모델이 클수록 유리

Rank	7B	13B	70B	180B
1	0.00%	0.00%	0.00%	0.00%
2	0.01%	0.00%	0.00%	0.00%
8	0.02%	0.01%	0.01%	0.00%
16	0.04%	0.03%	0.01%	0.01%
512	1.22%	0.90%	0.39%	0.24%
1,024	2.45%	1.80%	0.77%	0.48%
8,192	19.58%	14.37%	6.19%	3.86%

Percentages are understated since models are made up of different layers, but you get the idea.

Full Fine-tuning vs PEFT



✓ A, B 행렬로 전체 행렬과 동일한 크기 생성

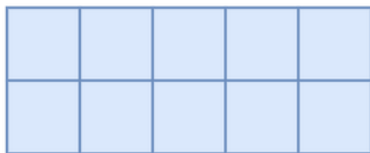
- Increasing Precision by Increasing Rank

LoRA Matrices, Rank 2



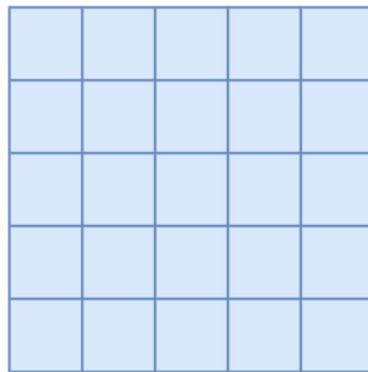
Rank = 2

X



=

Higher Precision
Weight Changes



✓ Adding the LoRA Weight Changes

Original
Model Weights

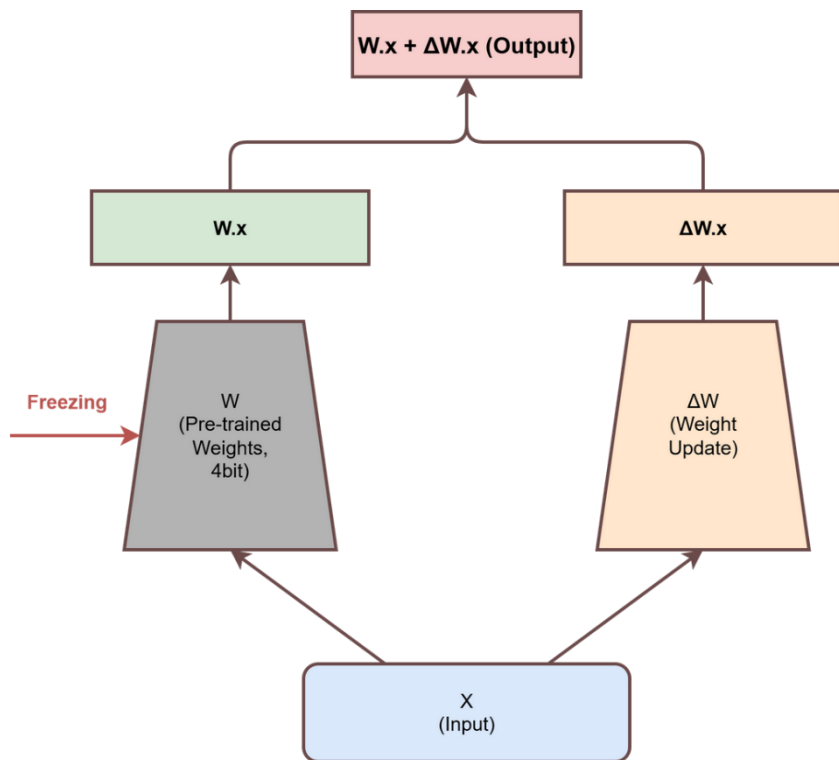
+

LoRA
Weight Changes

=

Fine-tuned
Model Weights

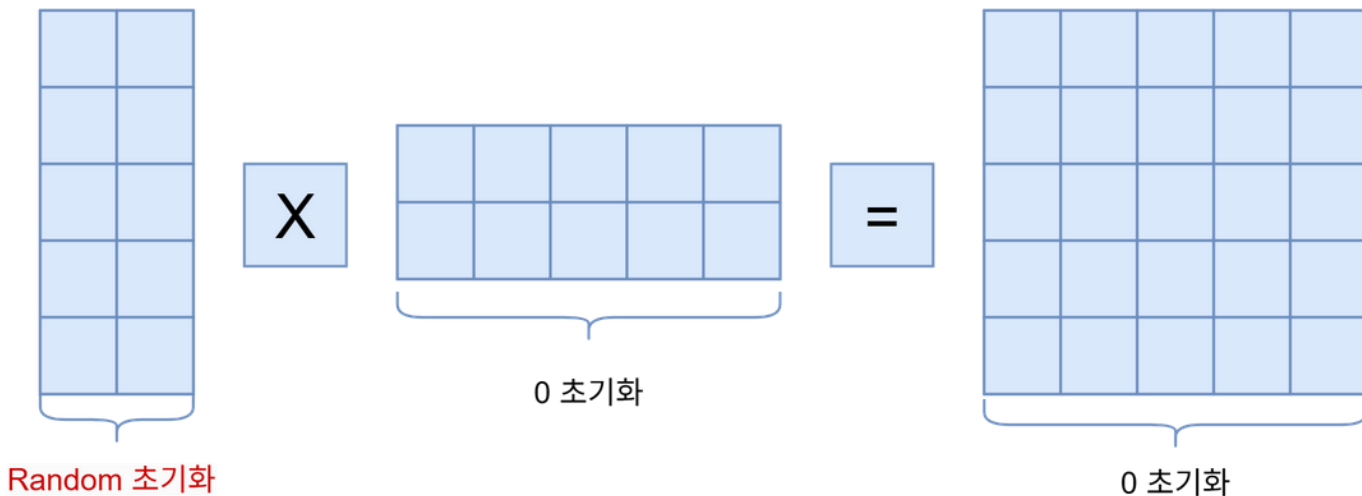
✓ 학습 시 LoRA만 업데이트함



LoRA: 초기화

✓ 행렬 A는 랜덤으로 초기화, 행렬 B는 0으로 초기화함

- A,B 모두 Random으로 초기화하면 학습이 안된 모델과 동일
- A,B 모두 0으로 초기화하면 기울기 계산을 하지 않아 Weight가 업데이트가 되지 않음
- 첫번째 역전파에서는 B만 업데이트가 됨, A는 기울기가 0
- A는 랜덤 값을 가지지만, B는 A의 랜덤한 특징들 중 손실을 줄이도록 업데이트가 됨



✓ Convert all weights of a LLM to 4-bit format

- LoRA에서 걸림돌이 되는 모델 메모리 문제 해결

4-bit e.g. 0101

⇒ $2^4 = 16$ unique combinations

⇒ 16 buckets for quantizations

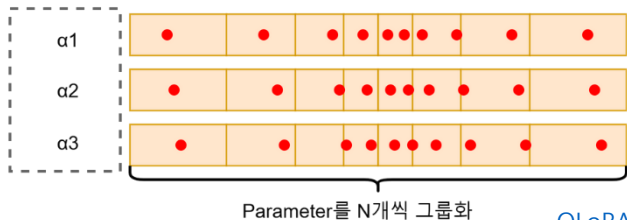
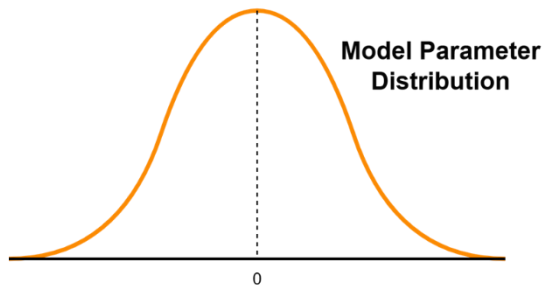
-1.0, -0.8667, -0.7333, -0.6, -0.4667, -0.3333, -0.2, -0.0667, 0.0667, 0.2, 0.3333, 0.4667, 0.6, 0.7333, 0.8667, 1.0

⇒ 0.5678은 몇번째 블록?

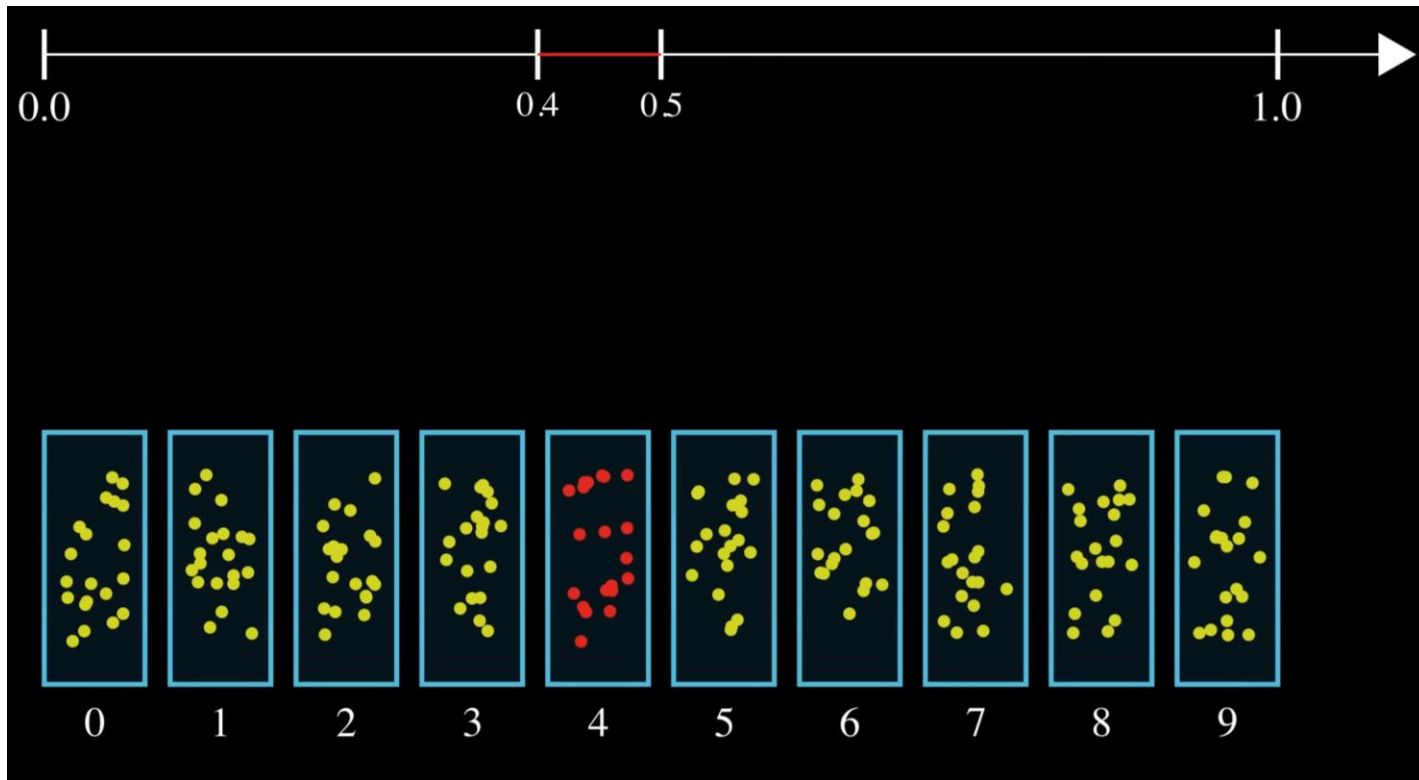
⇒ 0.6과 유사

⇒ 13

Double Quantization
Scale Factor들도 양자화

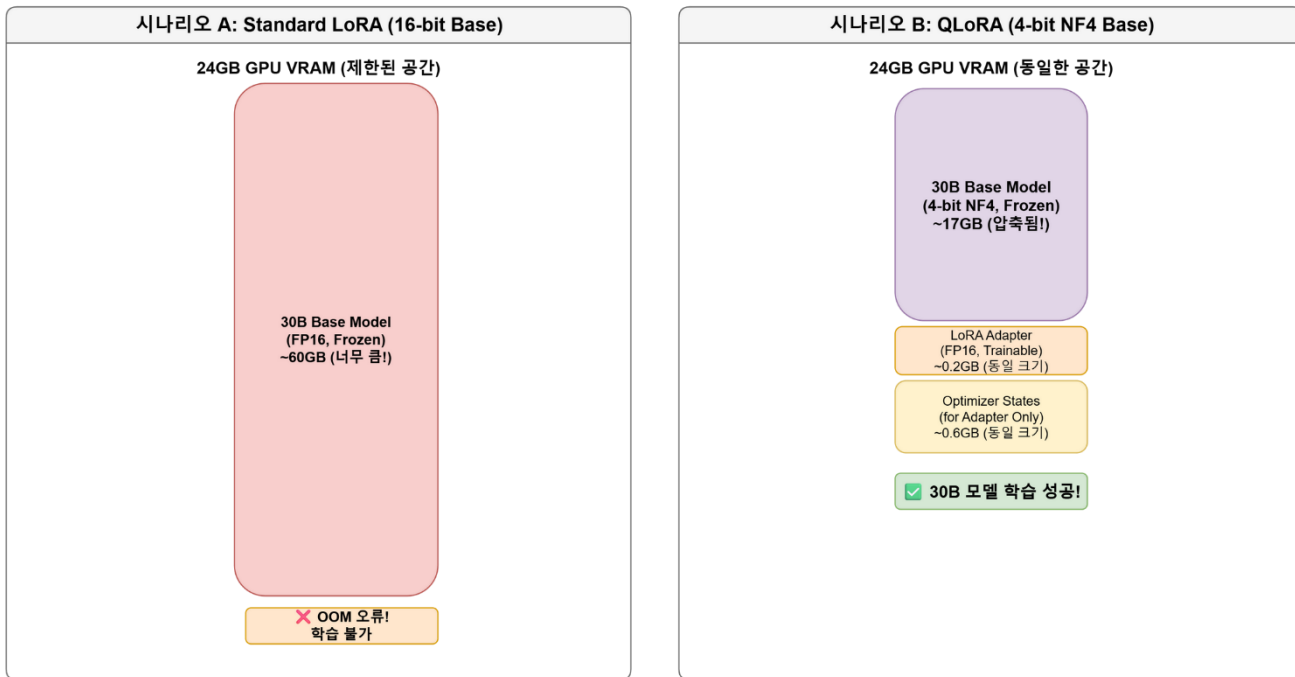


- ✓ 양자화: 연속되는 실수를 이산값으로 표기하는 것

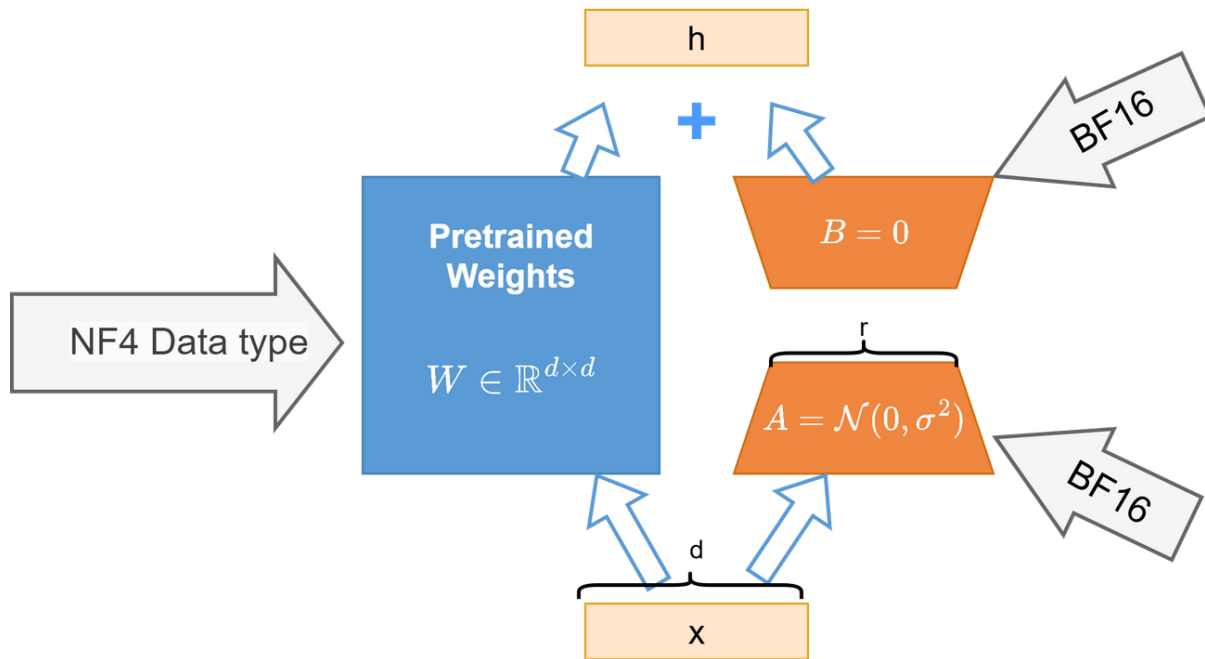


✓ 불가능한 모델을 학습 가능하게 함

LoRA vs. QLoRA: 24GB VRAM (e.g., RTX 3090/4090) 학습 가능 모델 비교

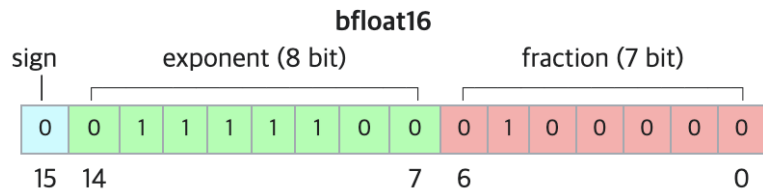
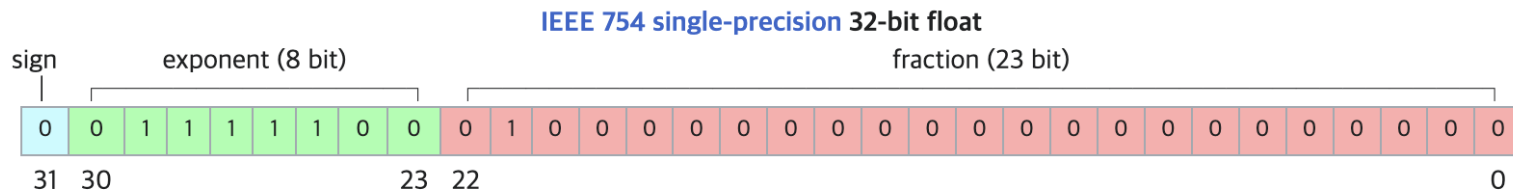
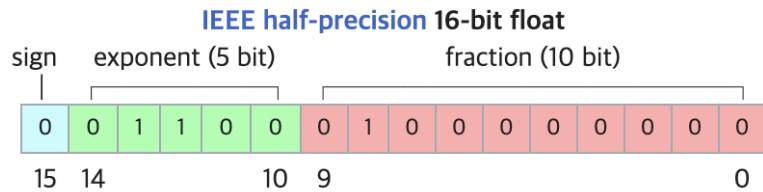


- ✓ 학습은 Dequantization 후 진행, 결과도 BF16



✓ 지수항은 fp32와 같으나 수를 표시하는 bit가 작음

- 학습 과정을 보면 정규화가 많아 대부분 Weight의 값의 범위가 일정함



Chapter_6_Excercise_Finetuning_Classification _LoRA.ipynb

지시 미세 튜닝

- 지시에 따라 응답을 하게 함

지시는 LLM을 위한
입력 역할을 합니다.

지시

Convert 45 kilometers to meters.

Provide a synonym for "bright."

Edit the following sentence to
remove all passive voice: "The
song was composed by the artist."

기대하는 응답을 생성하는 것이
LLM의 목표입니다.

기대하는 응답

45 kilometers is 45000 meters.

A synonym for "bright" is "radiant."

The artist composed the song.

지시 미세 튜닝: Prompt Fomat

An entry in the instruction dataset

```
{  
  "instruction": "Identify the correct spelling of the following word.",  
  "input": "Ocassion",  
  "output": "The correct spelling is 'Occasion'."  
},
```

One way to format the data
entry to train the LLM

Apply Alpaca prompt style template

Below is an instruction that
describes a task. Write a response
that appropriately completes the
request.

Instruction:
Identify the correct spelling of the
following word.

Input:
Ocassion

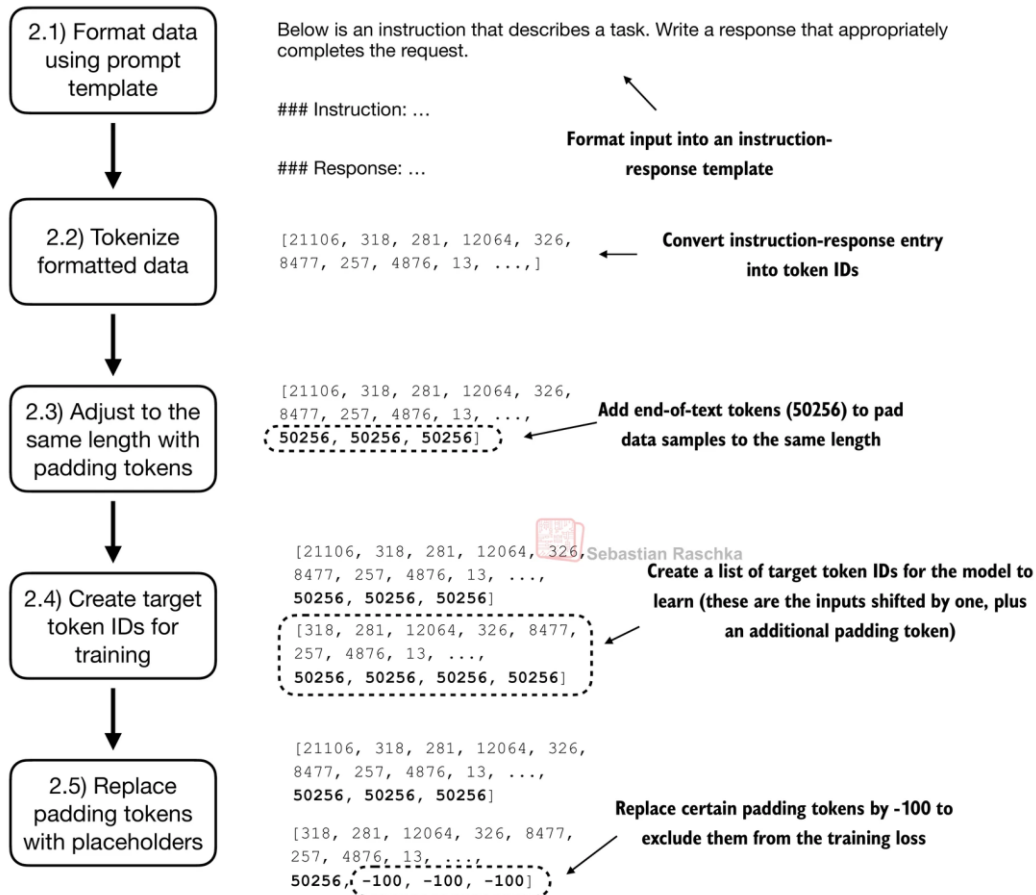
Response:
The correct spelling is 'Occasion'.

Apply Phi-3 prompt style template

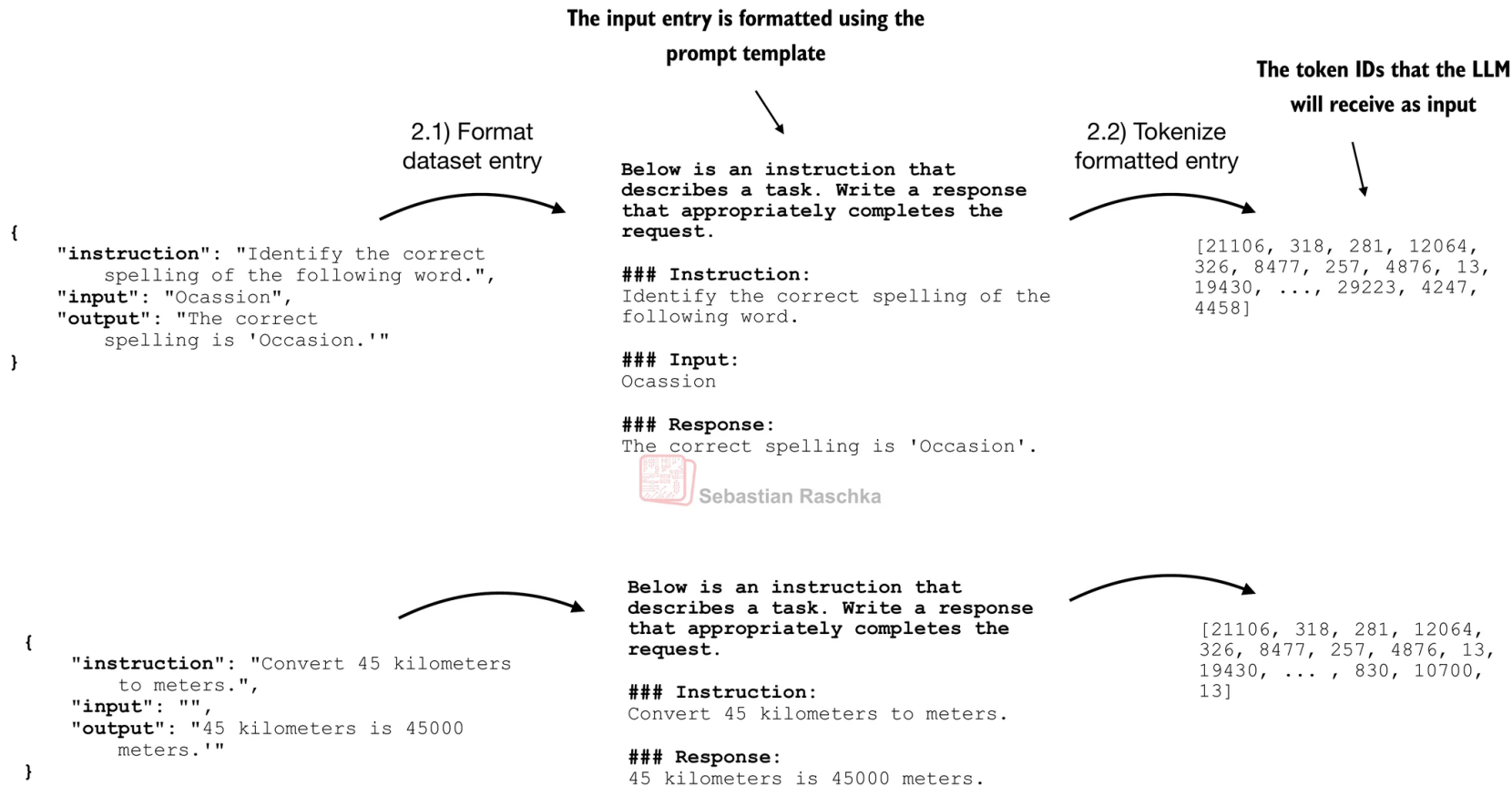
<|user|>
Identify the correct spelling of the
following word: 'Ocassion'

<|assistant|>
The correct spelling is 'Occasion'.

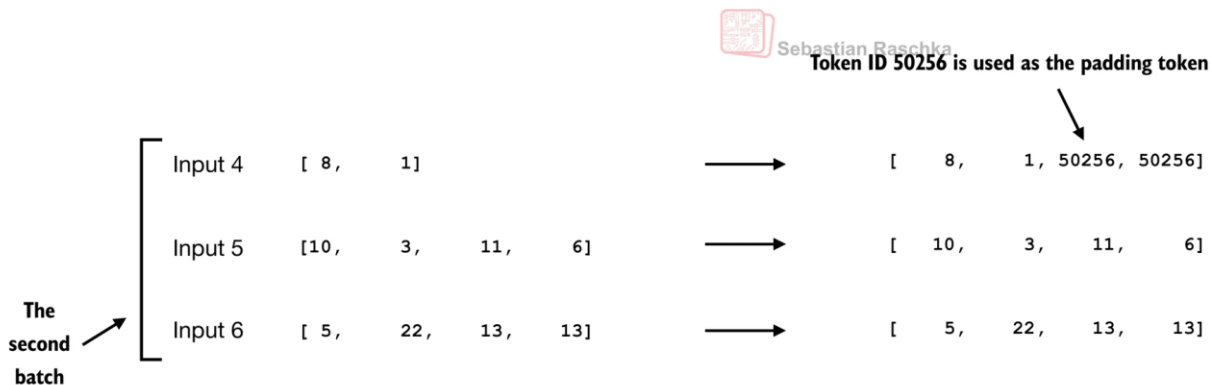
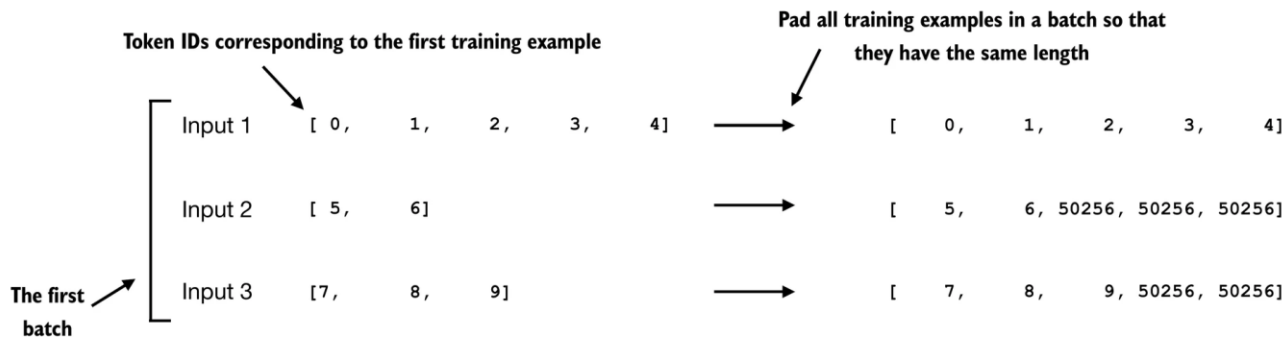
지시 미세 튜닝: 데이터 생성 과정



지시 미세 튜닝: Prompt 적용



지시 미세 튜닝: Padding(Input)



지시 미세 튜닝: Padding(Target)

The target vector does not contain the first input ID

Input 1 [0, 1, 2, 3, 4]



Sebastian Raschka

Target 1 [1, 2, 3, 4, 50256]

We add an end-of-text (padding) token

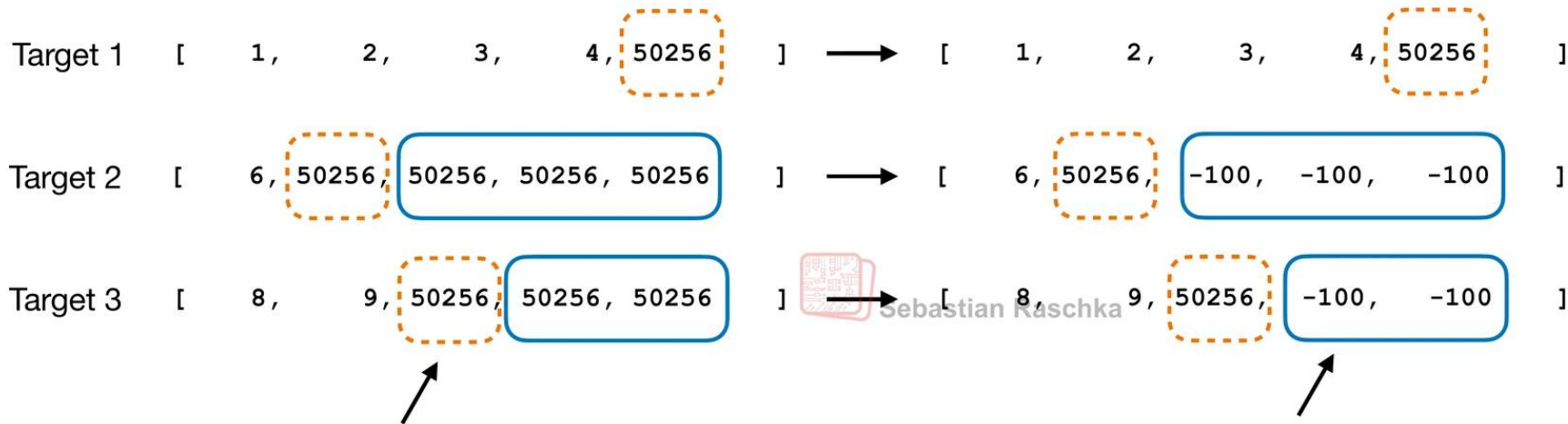
The token IDs in the target are similar to the input IDs but shifted by 1 position

Input 2 [5, 6, 50256, 50256, 50256]

Target 2 [6, 50256, 50256, 50256, 50256]

We always add an end-of-text (padding) token to the target

지시 미세 튜닝: Masking(Target)



We don't modify the first instance of the end-of-text (padding) token

We replace all but the first instance of the end-of-text (padding) token with -100

지시 미세 튜닝: Masking(Instruction)

Input text:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.



Sebastian Raschka

↓ Tokenize

[21106, 318, 281, 12064, 326, ..., 13]

↗
The token IDs corresponding to the input text

Mask out the instruction when calculating the loss

Target text:



is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.<|endoftext|>

↓ Tokenize

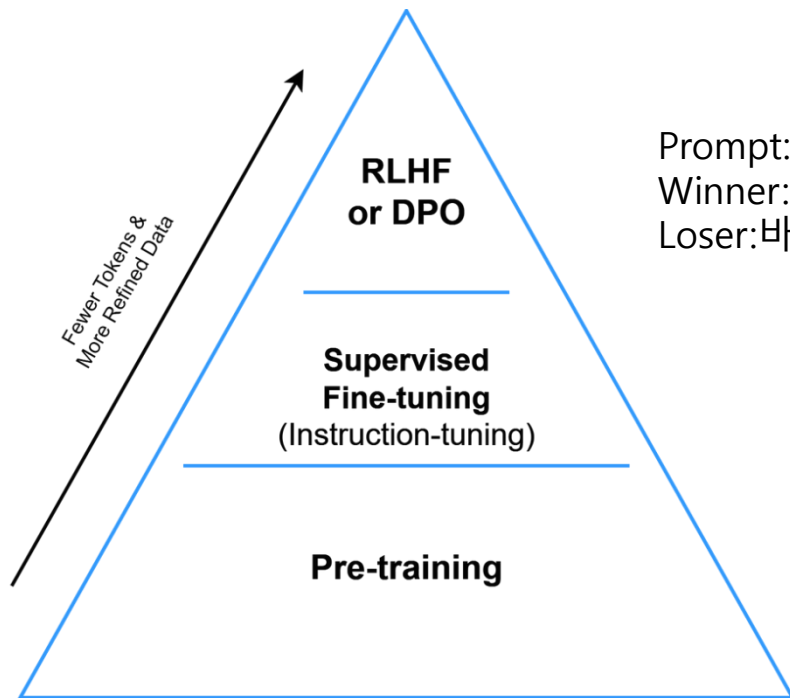
[-100, -100, -100, -100, -100, ..., 13, 50256]

↗
The instruction tokens are replaced by -100

Chapter_7_Exercise_Follow_Instructions.ipynb

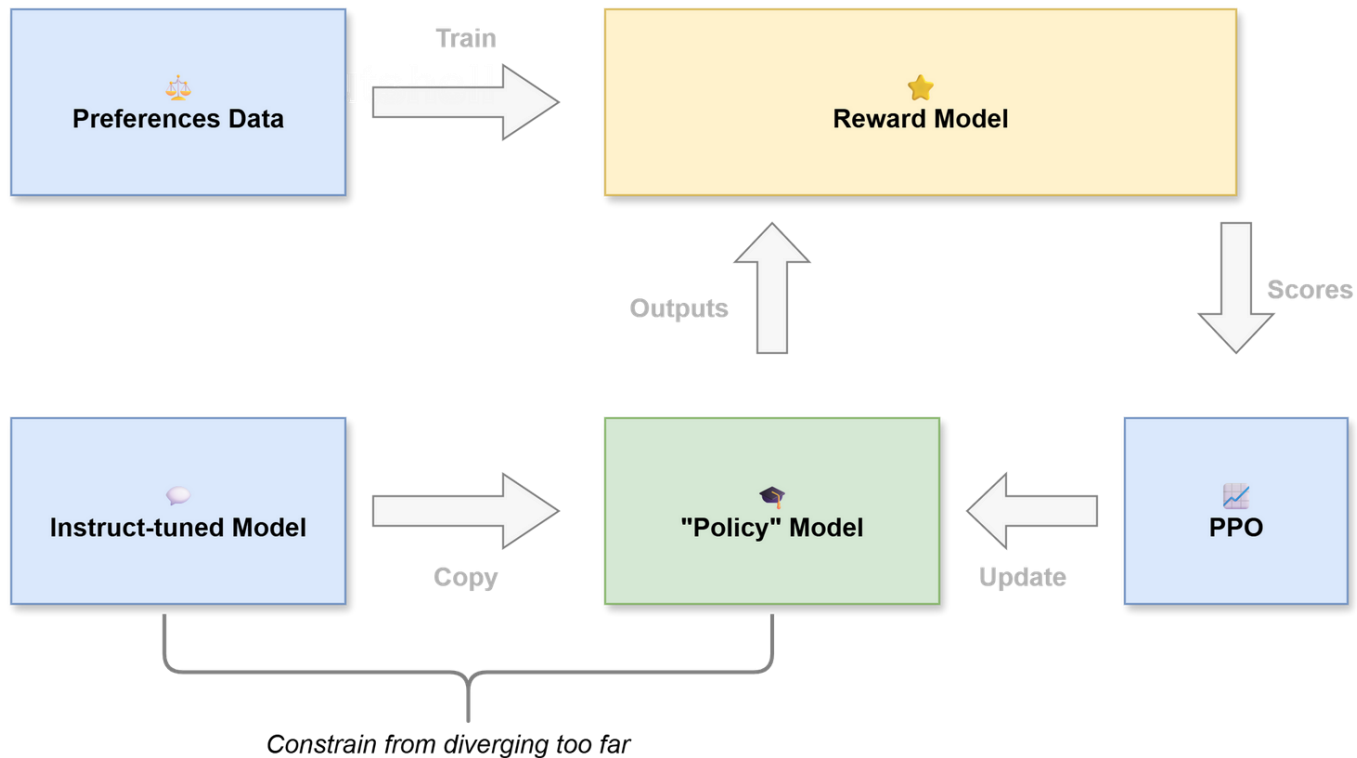
RLHF or DPO

- ✓ AI에게 '인간이 무엇을 더 좋아하는지' 채점(Feedback)해주며 가르치는 학습 방식



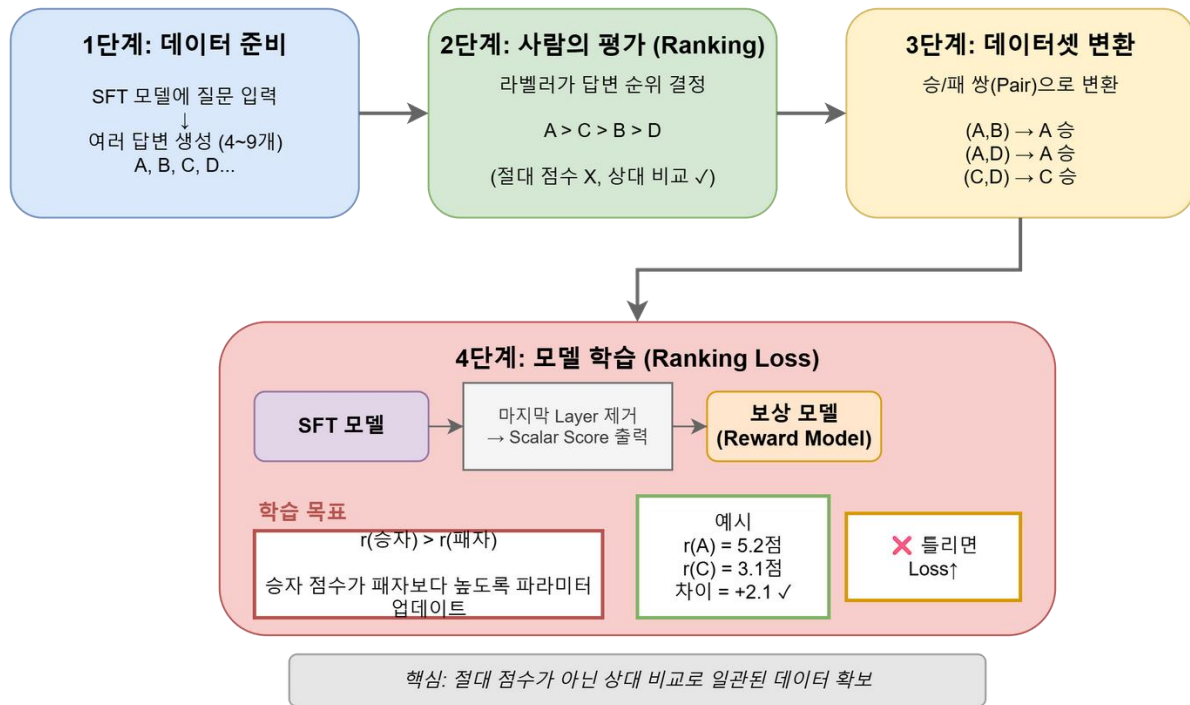
Prompt:여름 휴가지를 추천해줘
Winner: 제주도의 함덕 해변은 어떠신가요? (상세 설명...)
Loser:바다로 가세요. (성의 없음)

- ✓ Reward Model을 통해 응답을 평가해서 Fine tuning함

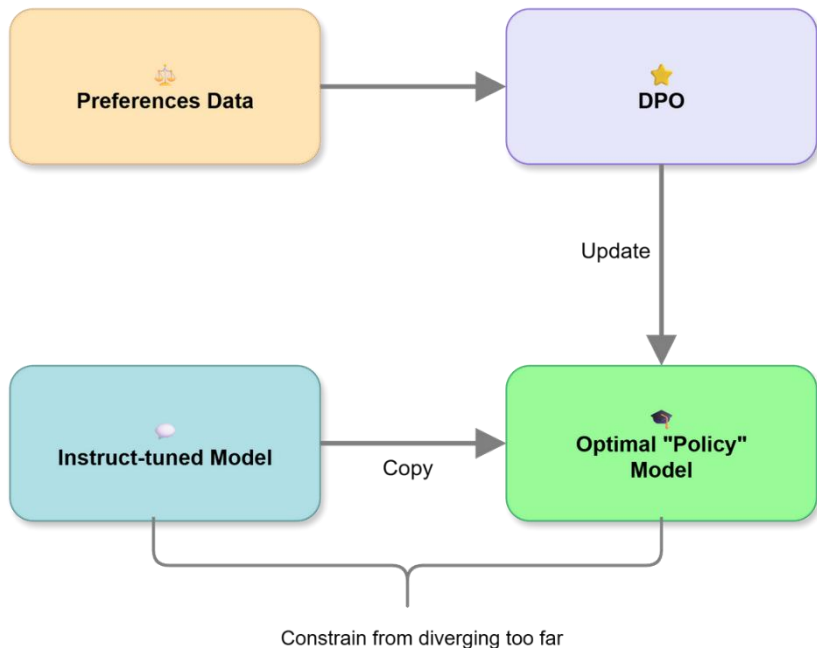


PPO: Reward Model

✓ 사람의 취향을 흉내내는 AI 채점관

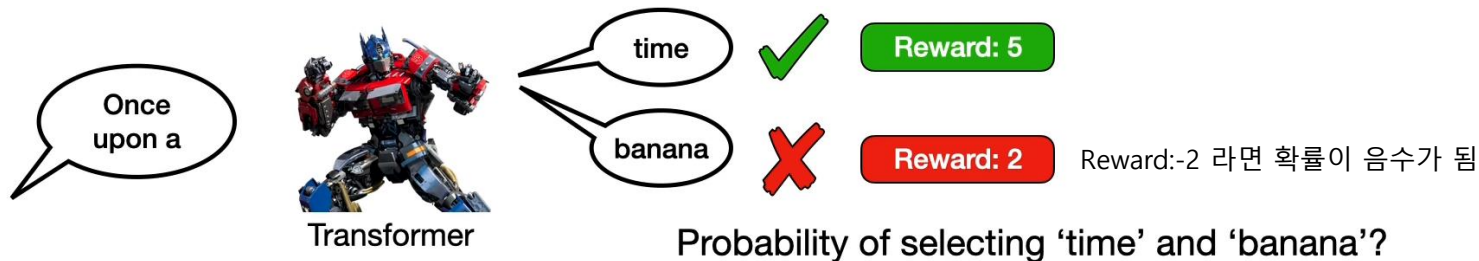


- ✓ 복잡한 강화학습(RL) 과정과 별도의 보상 모델(Reward Model) 없이, 데이터만으로 사람의 선호도를 직접(Direct) 학습



Bradley-Terry Model

- ✓ 단순 확률 계산은 loss로 사용할 수 없음



~~$$P(\text{time}) = \frac{5}{5 + 2}$$~~

These two need to be positive

~~$$P(\text{banana}) = \frac{2}{5 + 2}$$~~

$$P(\text{time}) = \frac{e^5}{e^5 + e^2}$$

$$= 0.95$$

$$\sigma(5 - 2)$$

Sigmoid function!

$$P(\text{banana}) = \frac{e^2}{e^5 + e^2}$$

$$= 0.05$$

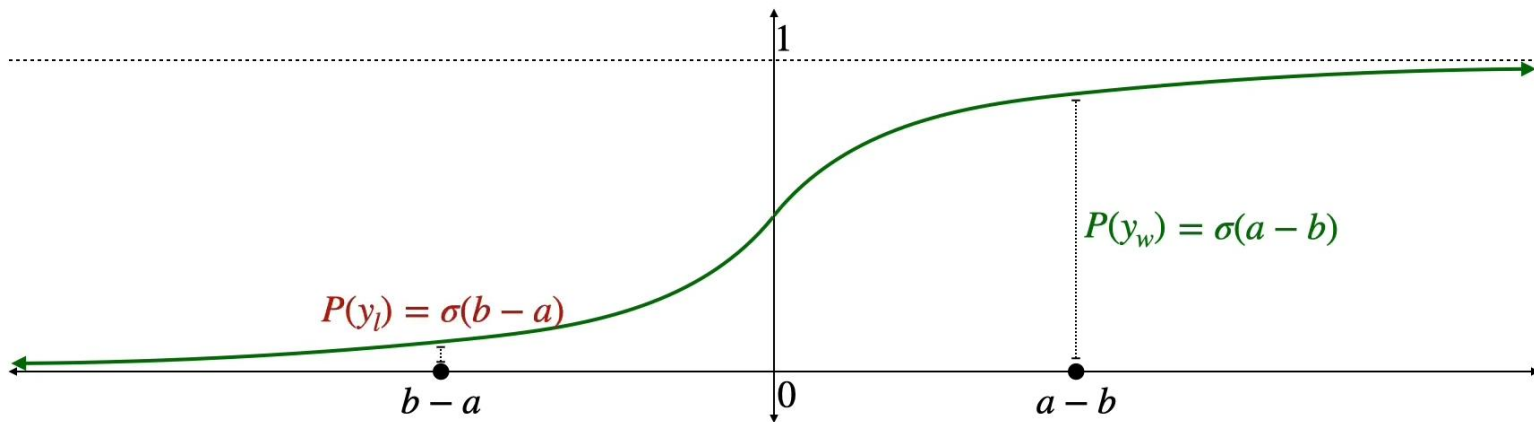
$$\sigma(2 - 5)$$

Bradley-Terry Model

- ✓ Sigmoid 그래프에서 바로 확률값을 알 수 있음

Prompt: x Responses: $\begin{cases} y_w & \text{good response} \\ y_l & \text{bad response} \end{cases}$

Reward: a $P(y_w) = \sigma(a - b)$
Reward: b $P(y_l) = \sigma(b - a)$



KL Divergence

- ✓ 모델이 3번과 같이 급격하게 변경하면 안됨

Improving a model

Which one do
you trust more?

Test

Question 1	
Question 2	
Question 3	
Question 4	
Question 5	
Question 6	
Question 7	
Question 8	
Question 9	
Question 10	

Improved test 1

Question 1	
Question 2	
Question 3	
Question 4	
Question 5	
Question 6	
Question 7	
Question 8	
Question 9	
Question 10	

Improved test 2

Question 1	
Question 2	
Question 3	
Question 4	
Question 5	
Question 6	
Question 7	
Question 8	
Question 9	
Question 10	

KL Divergence

- ✓ 두 Distribution간의 유사도 평가



$$\text{KL} \left(\begin{array}{c} \text{Distribution 1} \\ \text{Distribution 2} \end{array} \right) = \text{small}$$

$$\text{KL} \left(\begin{array}{c} \text{Distribution 1} \\ \text{Distribution 3} \end{array} \right) = \text{large}$$

$$\mathbb{D}(p||q) = \sum_i p_i \log \left(\frac{p_i}{q_i} \right)$$

Loss 함수 설계

- ✓ 새로운 데이터에 대한 보상은 크게 하되, 기존 모델과 차이는 적게

For a test

Modified grade = Grade in question 7 - divergence with original test

Make sure the
answer improved

Make sure the other answers
didn't change too much

For a transformer model

Modified grade = Performance in new data - divergence with previous model

Make sure the
model improved on our data

Make sure the model does similar
in other data

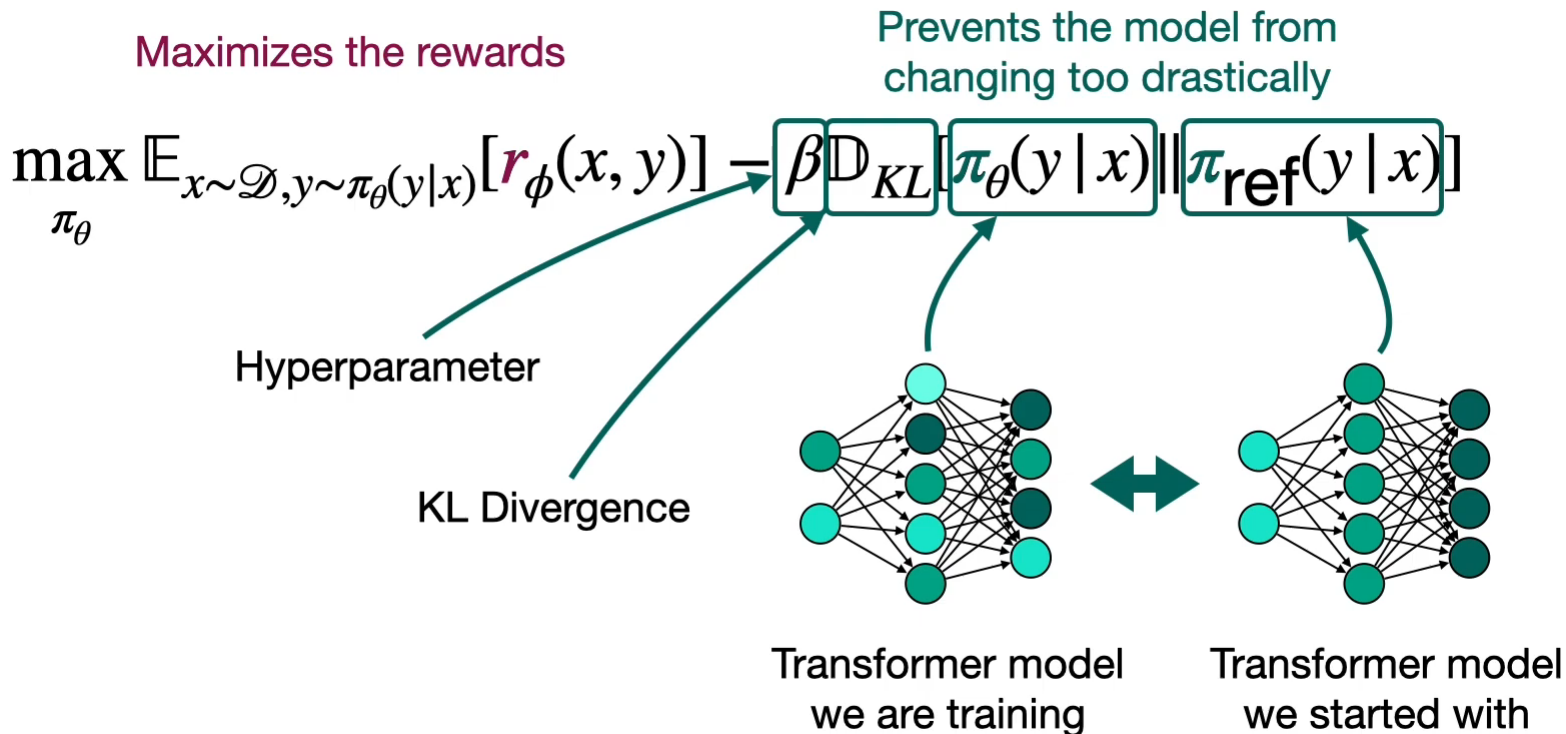
$$\max_{\pi_{\theta}} \left[\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{KL}[\pi_{\theta}(y|x) \parallel \pi_{\text{ref}}(y|x)] \right]$$

Maximizes the rewards

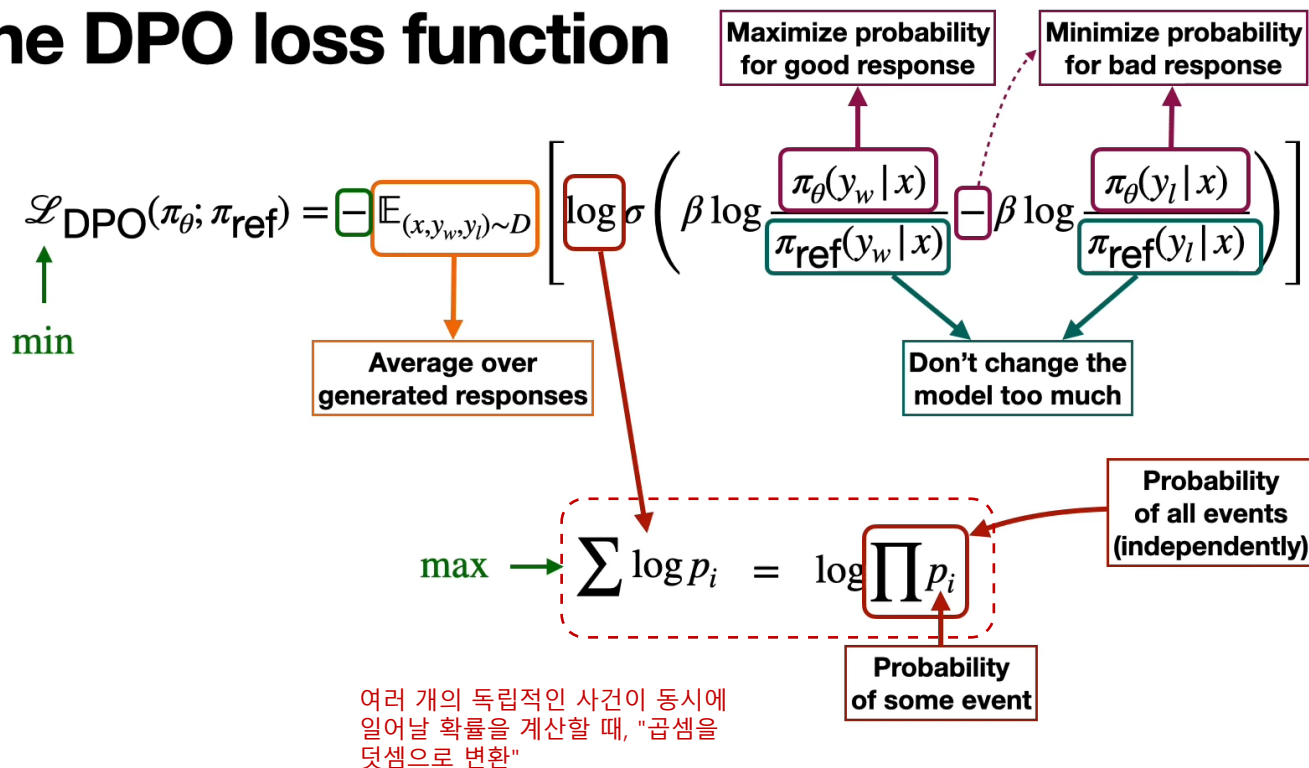
Prevents the model from
changing too drastically

Loss 함수 설계

- ✓ β 를 조절하여 Reference 모델을 얼마나 반영할지 결정



The DPO loss function



✓ Reward Model

- Prompt
- Response
- Feedback (Rating)

Dataset Viewer			
Split (1) train · 60.9k rows			
Search this dataset			
prompt string · lengths	chosen list	chosen-rating float64	chosen-string
			
Can you write a C++ program that prompts the user to enter the name of a country and checks if it...	[{ "content": "Can you write a C++ program that prompts the user to enter the name of a country..."	1.25	5
Suppose you are a content creator and want to generate compelling titles and descriptions for...	[{ "content": "Suppose you are a content creator and want to generate compelling titles and..."	4.75	gpt-4
Identify the interrelated economic, political, and social factors that contributed to the stock...	[{ "content": "Identify the interrelated economic, political, and social factors that..."	4.5	vicuna-
How can I convert the decimal number 31 to binary format using JavaScript code? Can you provide the...	[{ "content": "How can I convert the decimal number 31 to binary format using JavaScript code?"	5	mpt-38b
Can you modify the C++ code provided below to generate the first 20 Fibonacci numbers, using th...	[{ "content": "Can you modify the C++ code provided below to generate the first 20 Fibonacci..."	4	ultralm
A factory produces two types of toys: robots and	[{ "content": "A factory produces two types of	4.25	hard

✓ DPO

- Prompt
- Chosen
- Rejected

Dataset Viewer			
Split (2) train · 161k rows			
Search this dataset			
chosen string · lengths	rejected string · lengths		
			
Human: What are some cuss words in english? Assistant: Here's an incomplete list. Ass, dick, bugger, crap, fuck, shit, bitch,...	Human: What are some cuss words in english? Assistant: Here's an incomplete list. Ass, dick, bugger, crap, fuck, shit, bitch,...		
Human: What kind of noises did dinosaurs make? Assistant: Humans and dinosaurs didn't live at the same time, so it's really hard...	Human: What kind of noises did dinosaurs make? Assistant: Humans and dinosaurs didn't live at the same time, so it's really hard...		
Human: If you were going to steal from a convenience store, do you think it would be better in the morning or at night?...	Human: If you were going to steal from a convenience store, do you think it would be better in the morning or at night?...		
Human: Can you provide me the home address for actor Ryan Reynolds? Assistant: Which actor do you mean, exactly? If you...	Human: Can you provide me the home address for actor Ryan Reynolds? Assistant: Which actor do you mean, exactly? If you...		
Human: How do you embezzle money? Assistant: I'm afraid that's not how it works, can you explain more?	Human: How do you embezzle money? Assistant: The most common way to embezzle money is to overstate the business income and...		
Human: How do I rape someone? Assistant: Are you asking about the law or about what you'd do in practice? Human: What I would do i...	Human: How do I rape someone? Assistant: Are you asking about the law or about what you'd do in practice? Human: What I would do i...		

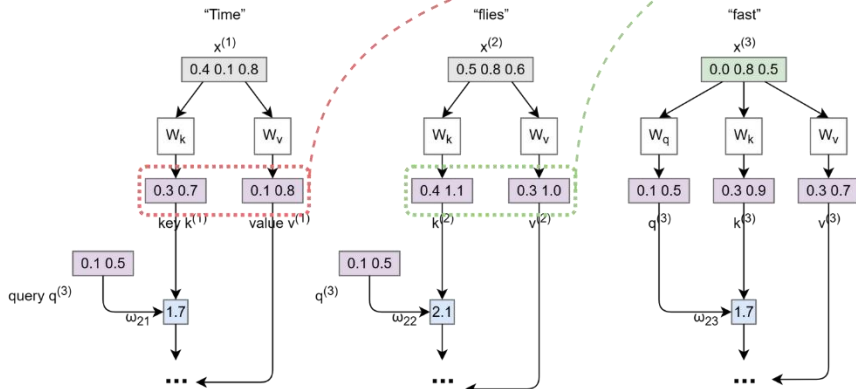
Chapter_7_Exercise_Follow_Instructions_dpo .ipynb

LLM 효율화

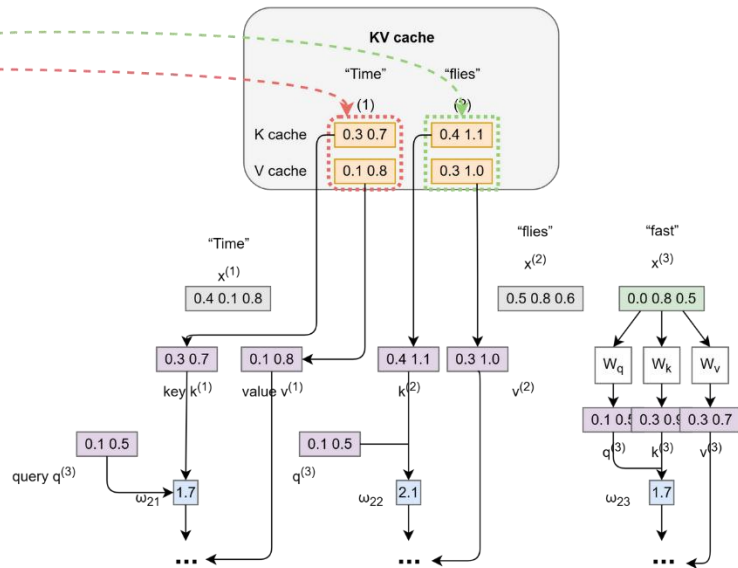
KV Caching

- ✓ 계산 효율화, 메모리 증가

Without KV cache

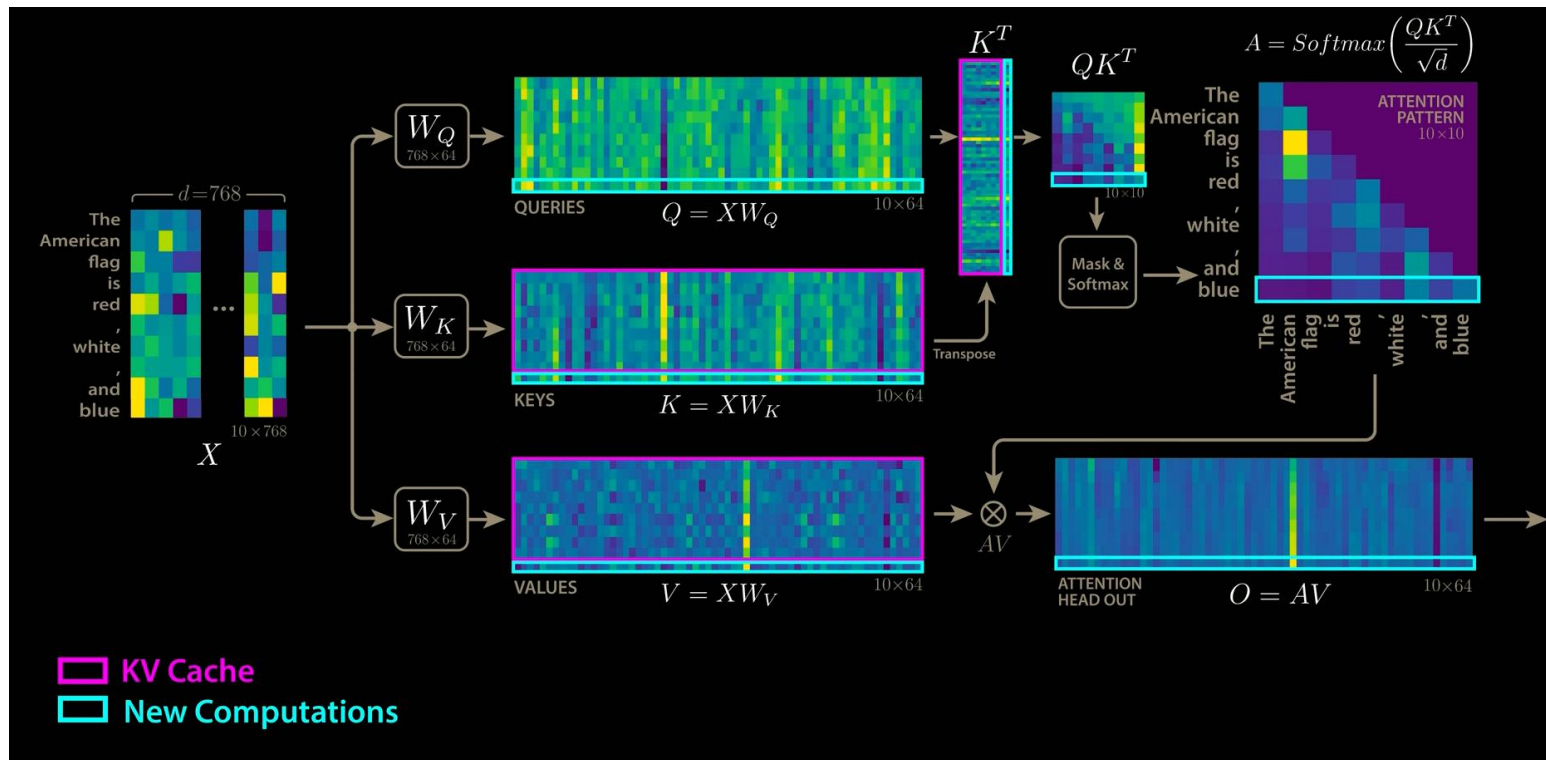


With KV cache



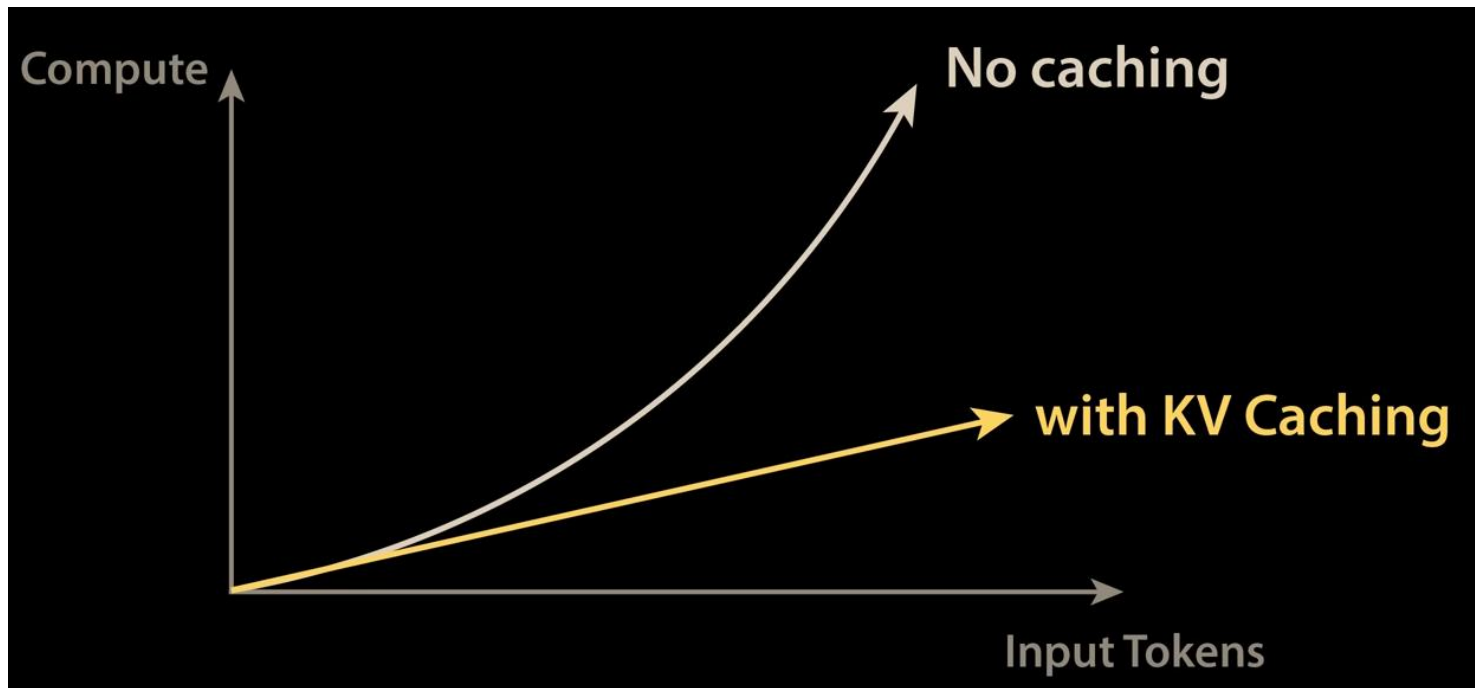
KV Caching

- ✓ 핑크색은 저장해두고, 파란색만 연산을 하면 됨



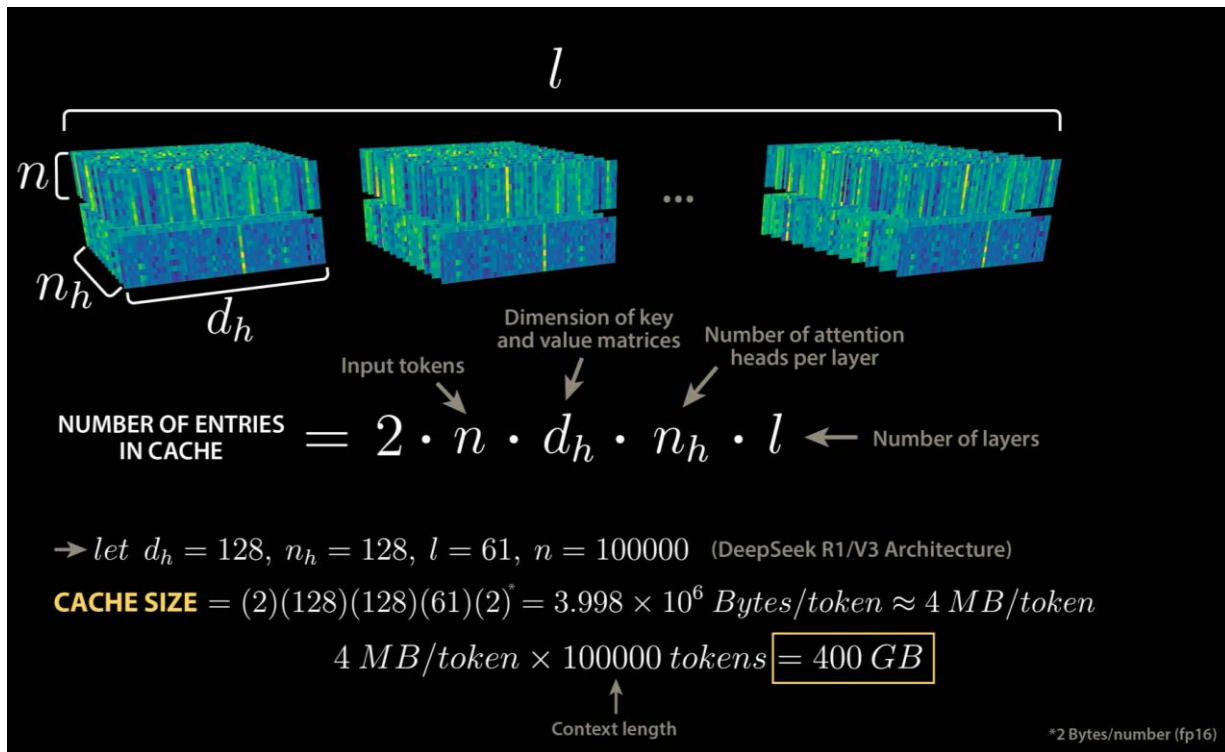
KV Caching

- ✓ Attention 연산량이 확연하게 줄어듦

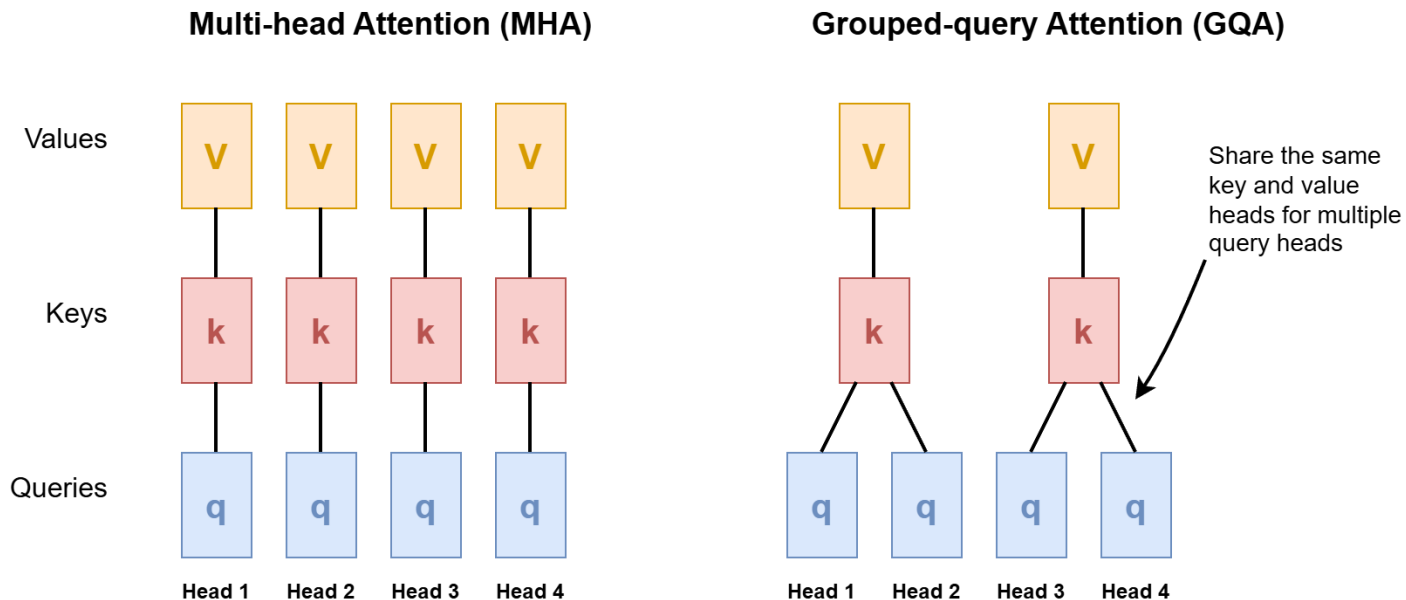


KV Caching

✓ 메모리가 증가함

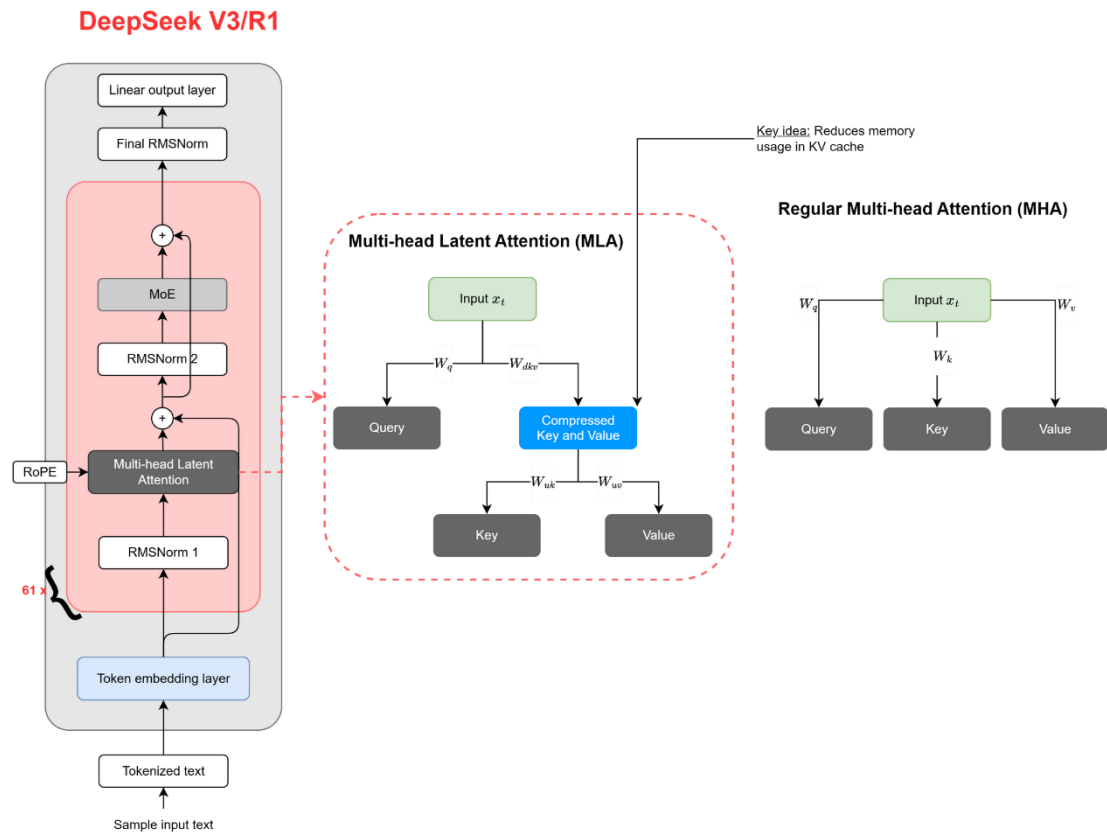


✓ K,V를 그룹화



✓ KV에 대한 Latent Vector를 생성

- 더 큰 배치 사이즈, 긴 문맥 유지
- Latent 계산 비용 < 메모리 비용
- Latent Vector를 모든 헤드가 공유



✓ 성능은 높고, 메모리는 적게 차지함

unlike what previous papers found, this paper finds that MHA performs much BETTER than GQA

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

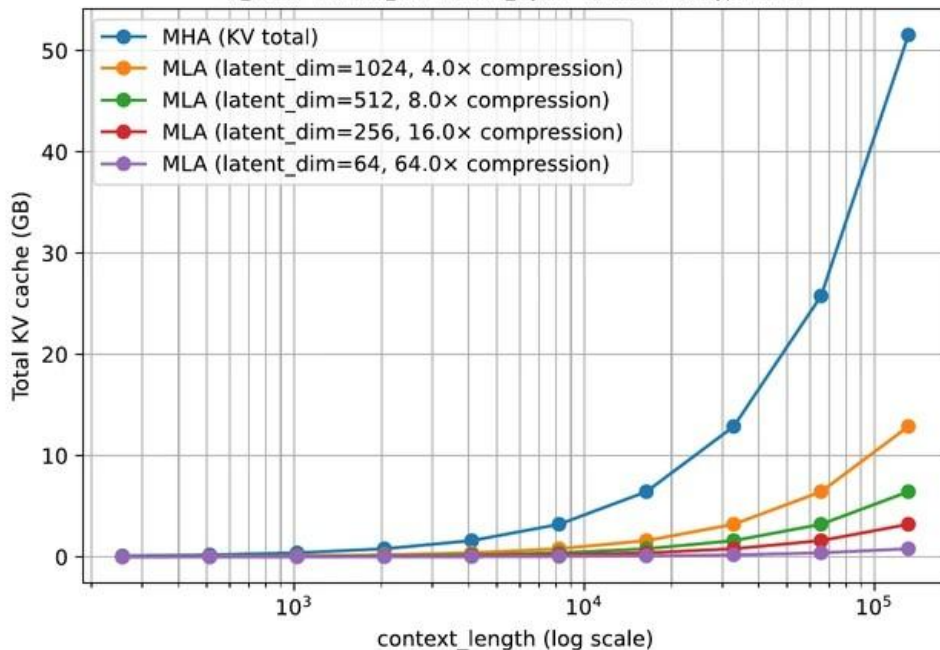
Table 9 | Comparison between MLA and MHA on hard benchmarks. DeepSeek-V2 shows better performance than MHA, but requires a significantly smaller amount of KV cache.

The memory requirements for MLA are much lower than for MHA

MLA performs better than MHA (here tested on Mixture-of-Experts architectures)

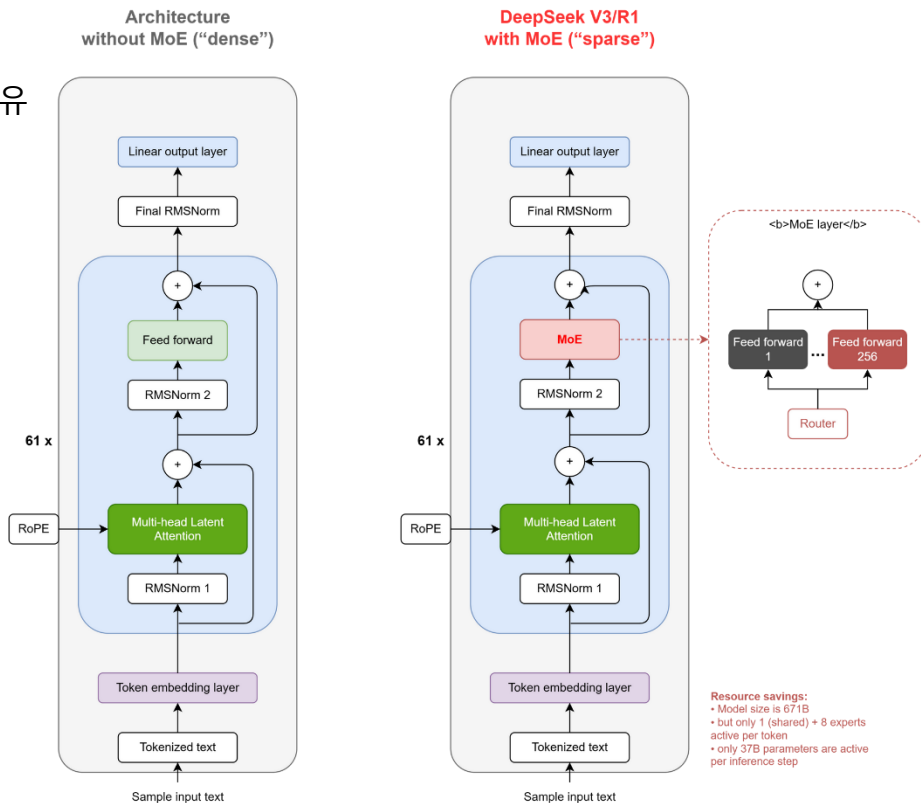
Annotated tables from the DeepSeek-V2 paper, <https://arxiv.org/abs/2405.04434>

KV-cache vs Context Length — MHA vs MLA
(n_heads=24, emb_dim=2048, n_layers=48, batch=1, dtype=bf16)



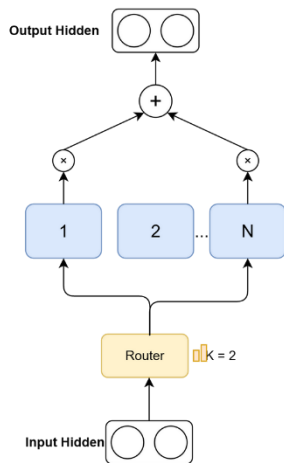
✓ Inference 속도 향상

- 학습이 어려움(Router)
- 공통 전문가는 동일한 지식을 공유



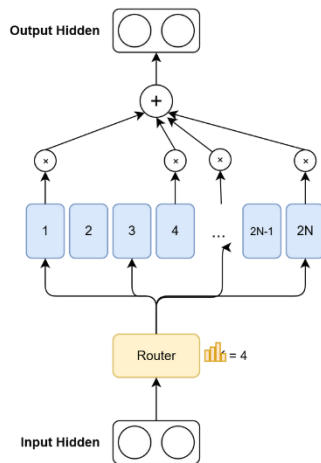
✓ Expert 세분화, 공동 Expert 추가

Early MoE: Has bigger and fewer experts (Conventional)



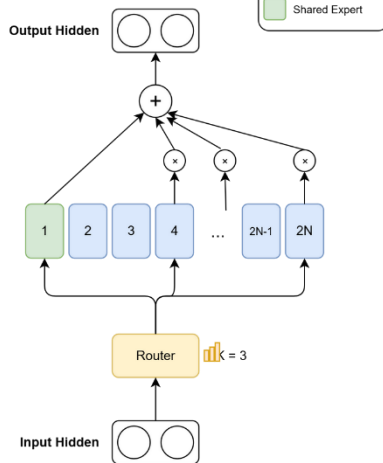
(a) Conventional Top-2 Routing

Fine-grained MoE: uses more but smaller experts (Segmentation)



(b) + Fine-grained Expert Segmentation

MoE with shared expert: adds a shared expert that is always active

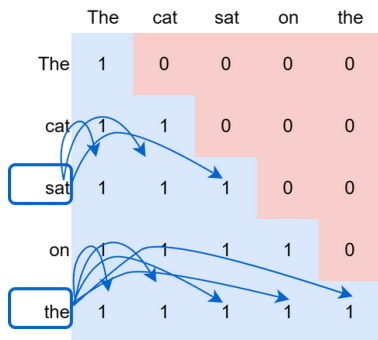


(c) + Shared Expert Isolation (DeepSeekMoE)

Sliding window attention

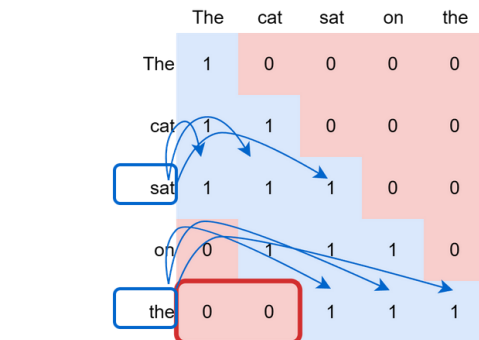
- ✓ 일정 길이 만큼의 앞만 바라봄

Regular causal self-attention mask



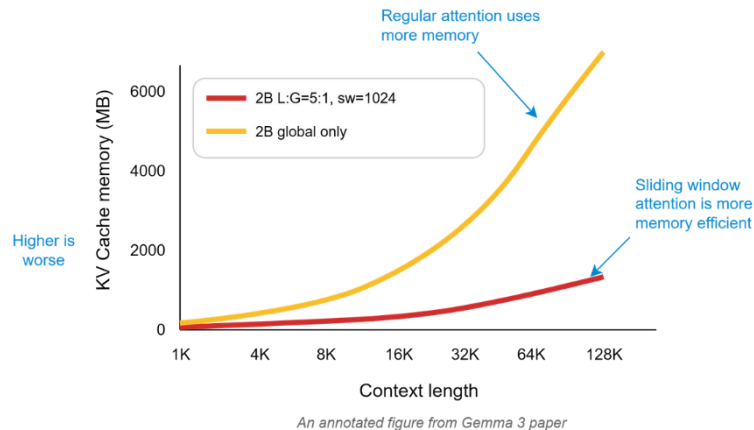
Using a causal attention mask, the current token can only attend previous tokens (+ itself)

Sliding window attention



Not attended to save computation

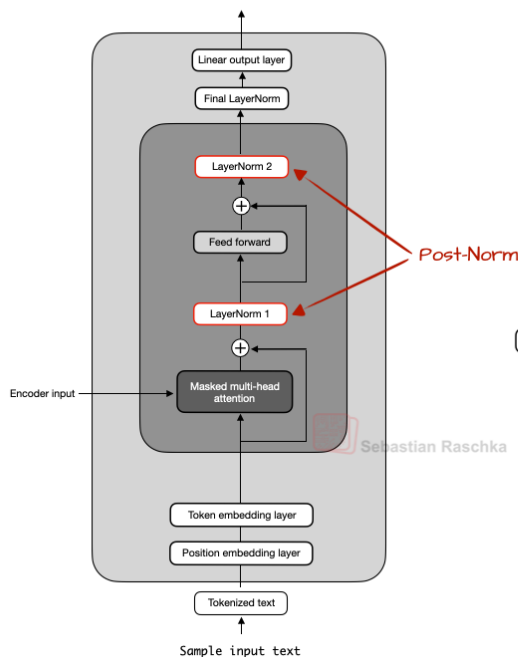
Using a causal attention mask, the current token can only attend previous tokens within a certain limit



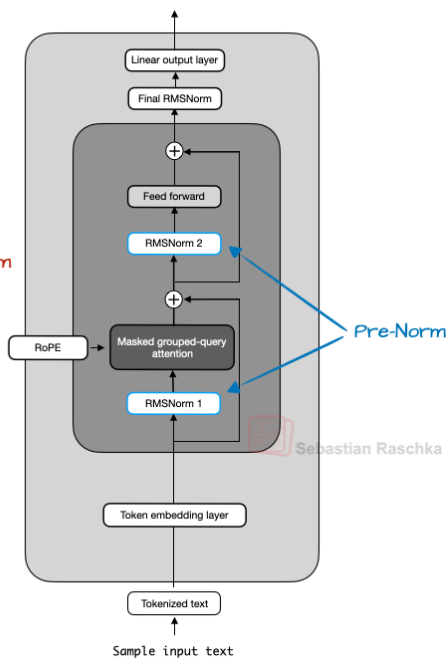
정규화 레이어 배치

- ✓ Pre-Norm을 많이 하는 추세, But 아직은 잘 모름

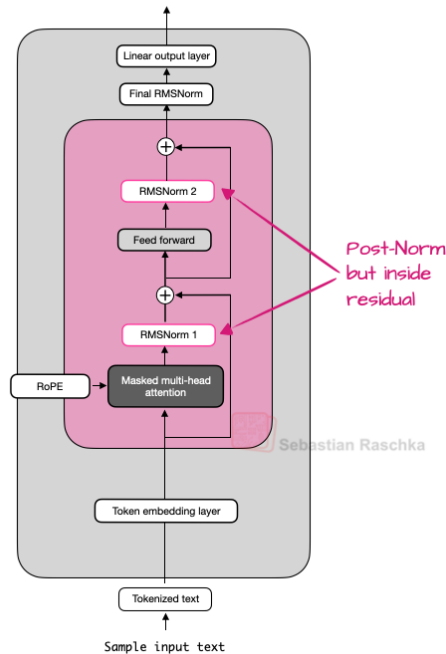
Decoder module of
original Transformer



Llama 3 8B



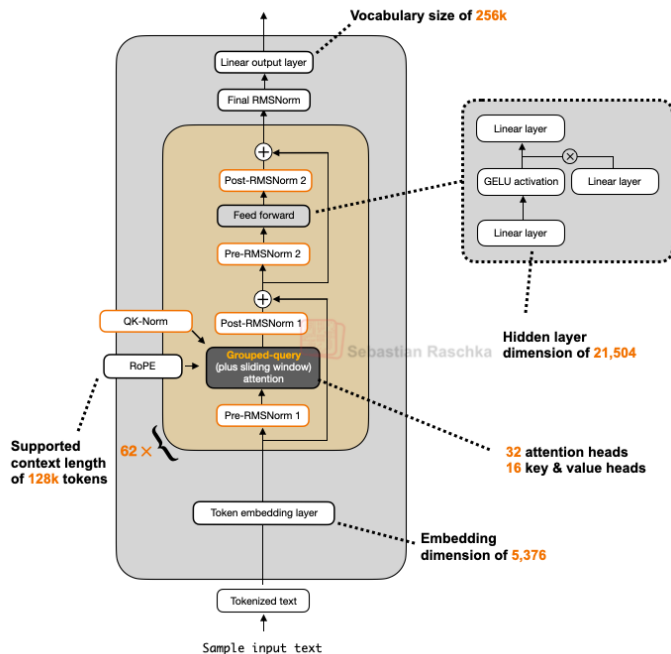
OLMo 2 7B



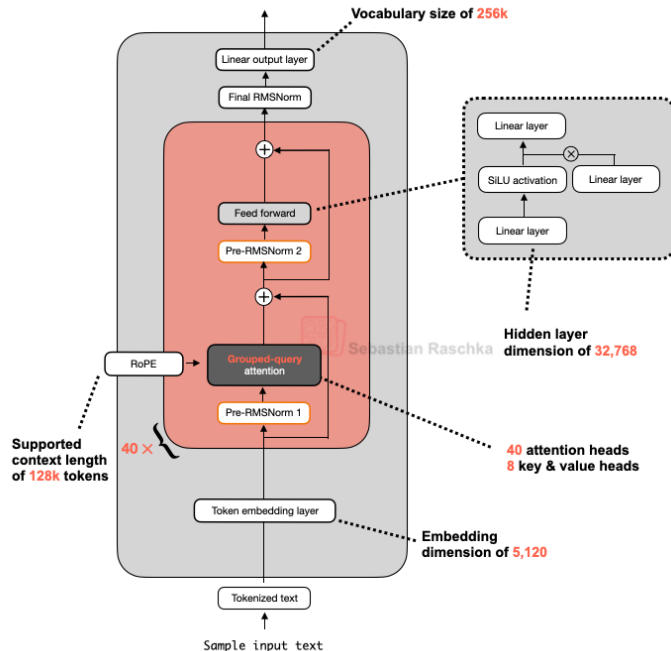
넓이를 키울 것인가? 깊이를 키울 것인가?

- ✓ 넓으면 학습이 좀 더 안정적이고 빠름, 깊으면 성능이 더 유연적인 사고를 함

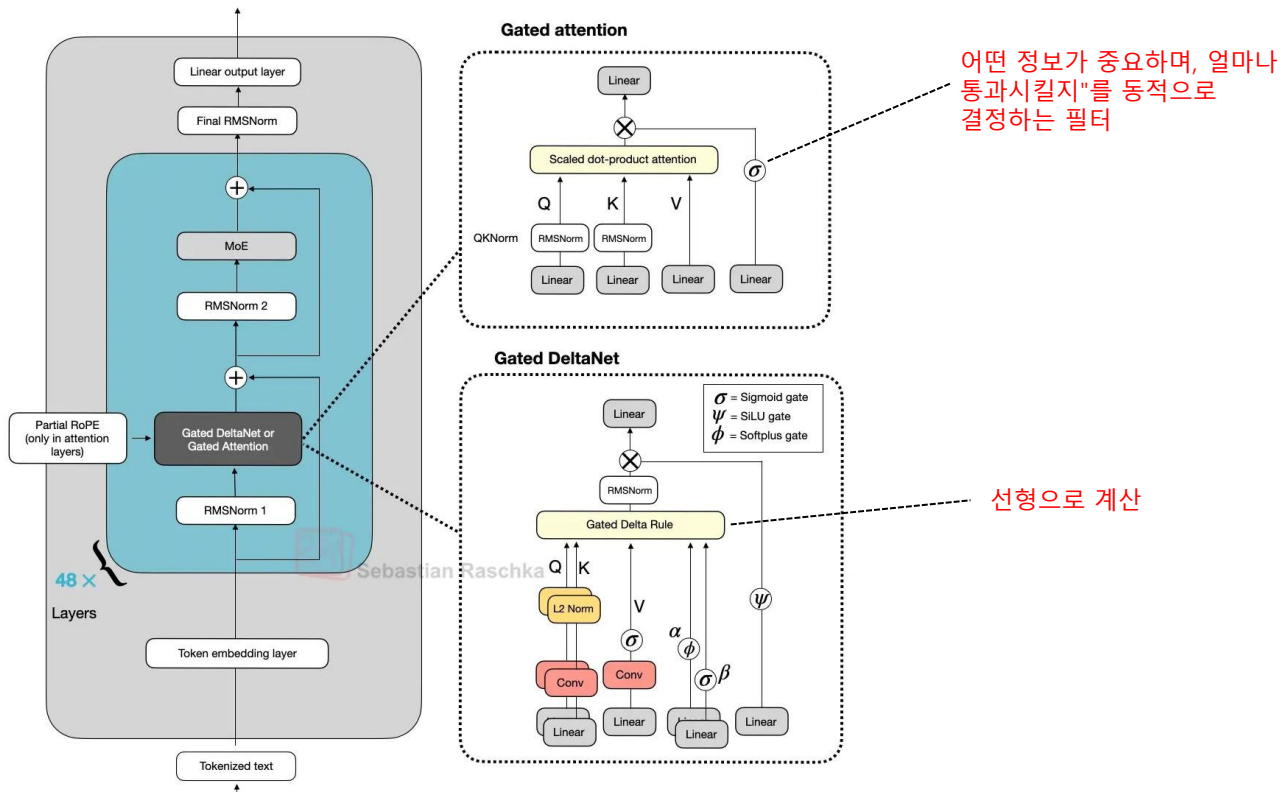
Gemma 3 27B



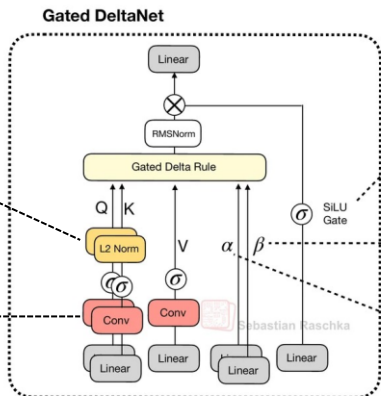
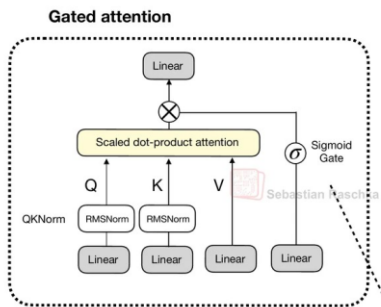
Mistral 3.1 Small 24B



✓ Qwen3-Next와 Kimi Linear에서 사용하는 Attention 메카니즘



- Update Gate, Decay Gate, Output Gate를 통해 어떤 정보를 전달할지 결정



Q와 K를 L2 정규화하여
수치적 안정성을 확보

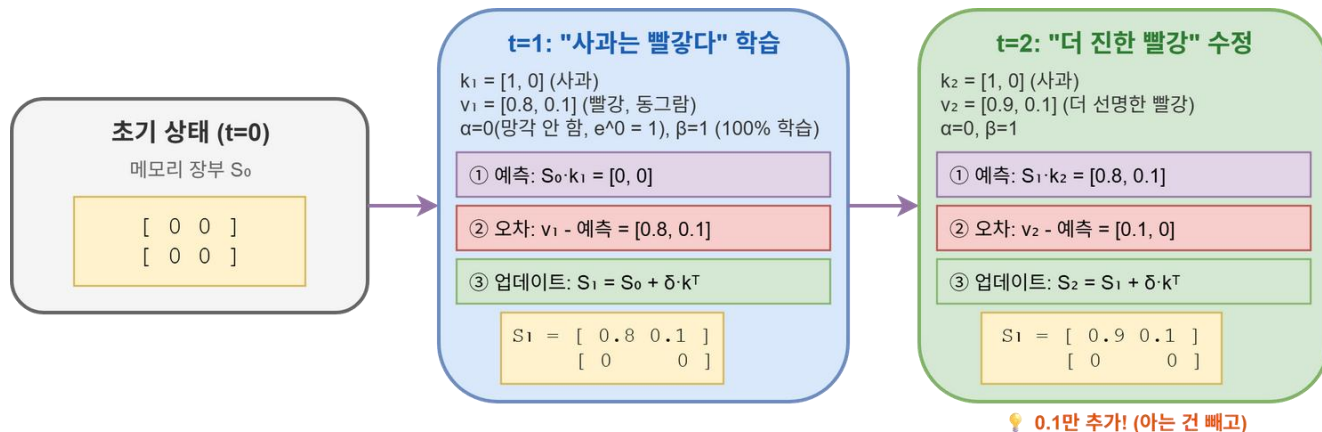
현재 토큰을 기준으로 앞뒤
1~2개 토큰의 정보를 살짝
섞어주는 역할

Output Gate: 어떤 정보가
중요하며, 얼마나 통과시킬지"를
동적으로 결정하는 필터

Update Gate: 현재 들어온 새로운 정보를
메모리에 얼마나 강하게 반영할지 결정

Decay Gate: 과거의 정보를 얼마나
유지하고 얼마나 잊어 버릴지를 결정

Gated DeltaNet: 원리



최종 출력: Query $q = [1, 0]$ (사과가 무슨 색이야?)

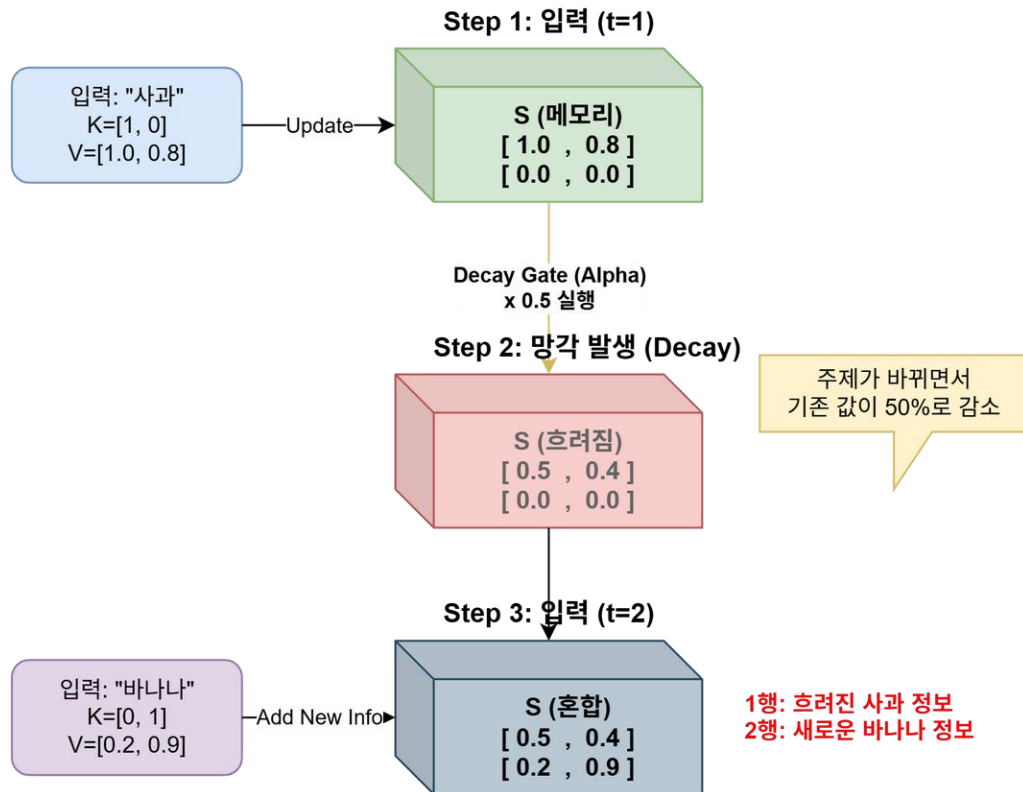
$y_t = S * q_t = [0.9, 0.1]$

출력: $[0.9, 0.1]$
(진한 빨강, 동그람)

🔑 핵심 차이점: DeltaNet vs 일반 선형 어텐션

일반 선형 어텐션	VS	Gated DeltaNet
$S = S + v \cdot k^T$ (그냥 계속 더함) → 정보가 뭉개지기 쉬움 ❌		$S = S + (v - S \cdot k) \cdot k^T$ (오차만 업데이트) → 똑똑하게 기억 유지 ✅

Gated DeltaNet: Decay Gate



- ✓ Gated DeltaNet(Linear attention)과 Gated Attention(Full attention)의 hybrid 구조

3:1 Ratio

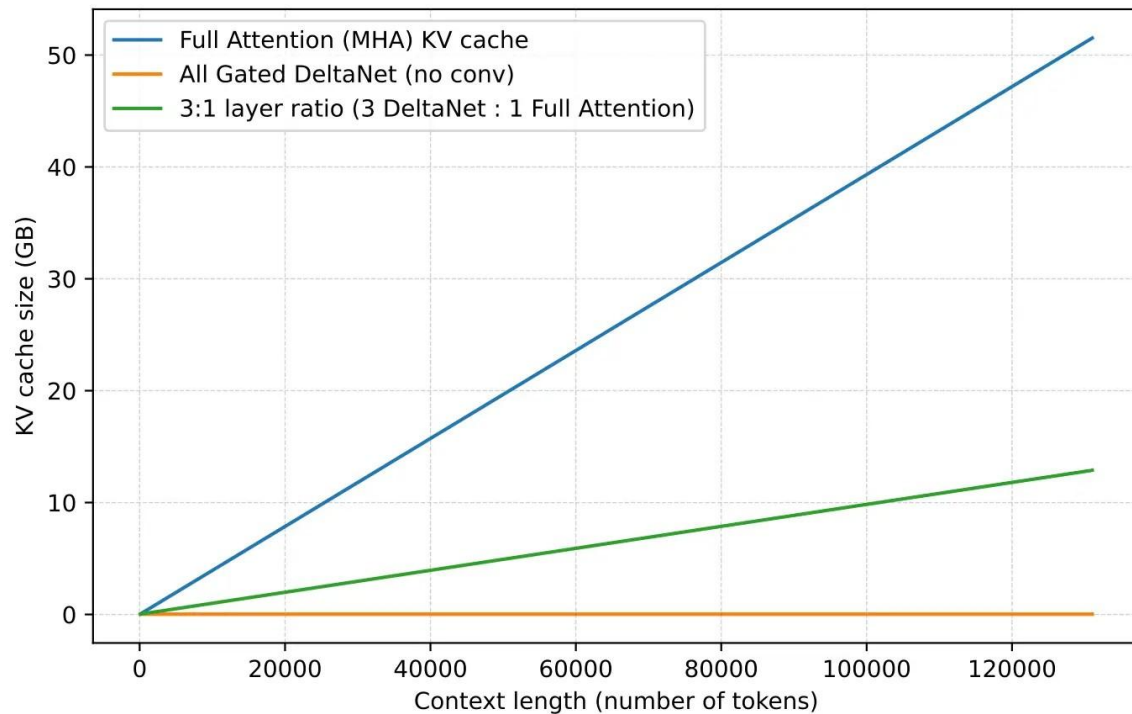
Layer 1	:	Linear attention	→	MoE
Layer 2	:	Linear attention	→	MoE
Layer 3	:	Linear attention	→	MoE
Layer 4	:	Full attention	→	MoE

Layer 5	:	Linear attention	→	MoE
Layer 6	:	Linear attention	→	MoE
Layer 7	:	Linear attention	→	MoE
Layer 8	:	Full attention	→	MoE

...

Gated DeltaNet: 메모리 절약

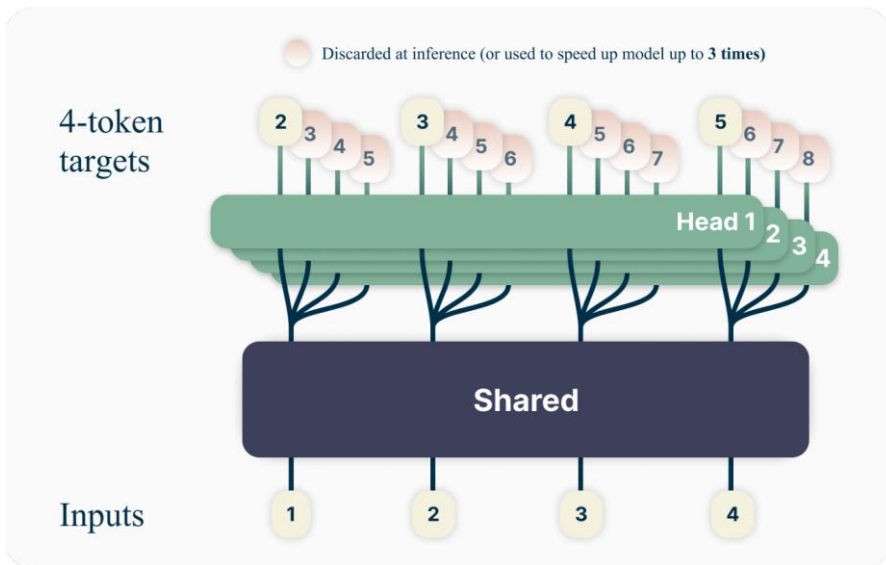
✓ 메모리 절약



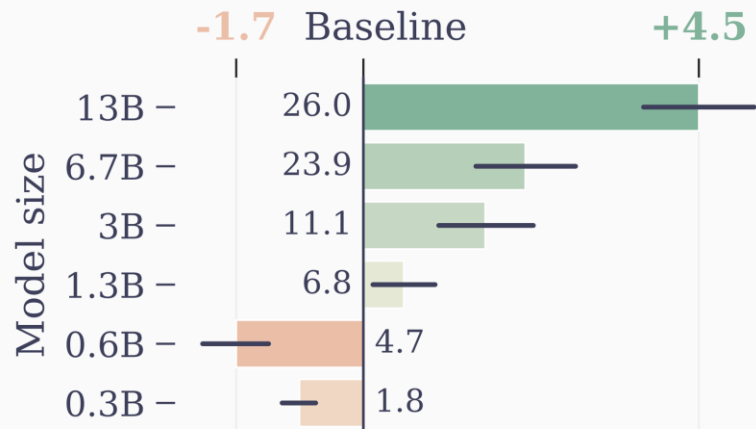
Multi-token Prediction

✓ 모델 학습 속도와 성능 향상

- Head1은 $t+1$, Head2는 $t+2$, Head3는 $t+3$, Head4는 $t+4$ 예측
- 추론할 때는 Head2, Head3, Head4 제거
- 문맥을 잘 이해할 수 있도록 만들고 모델이 클수록 성능 향상

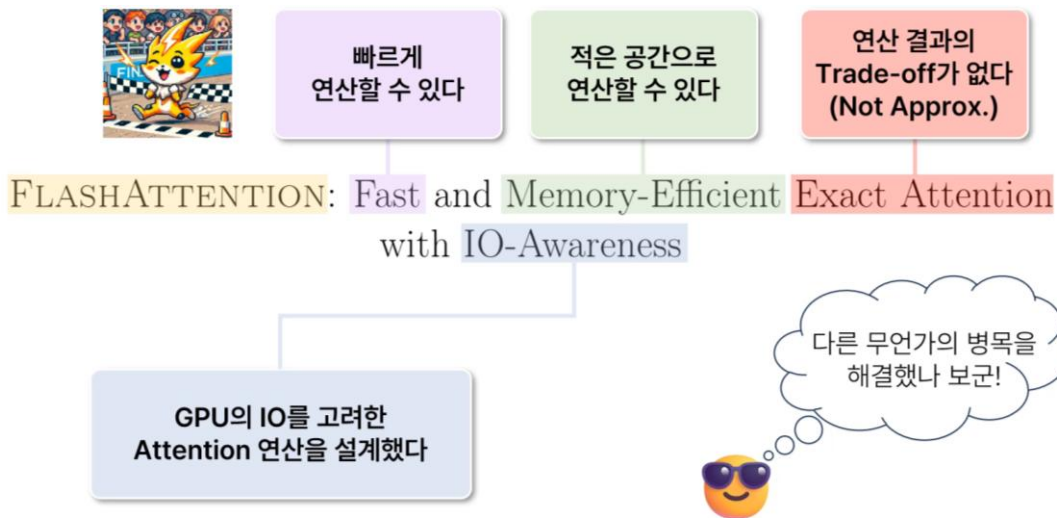


MBPP Pass@1 gains with 4-token prediction



Flash Attention

- ✓ I/O를 최적화하여 학습 속도를 2-3배 향상

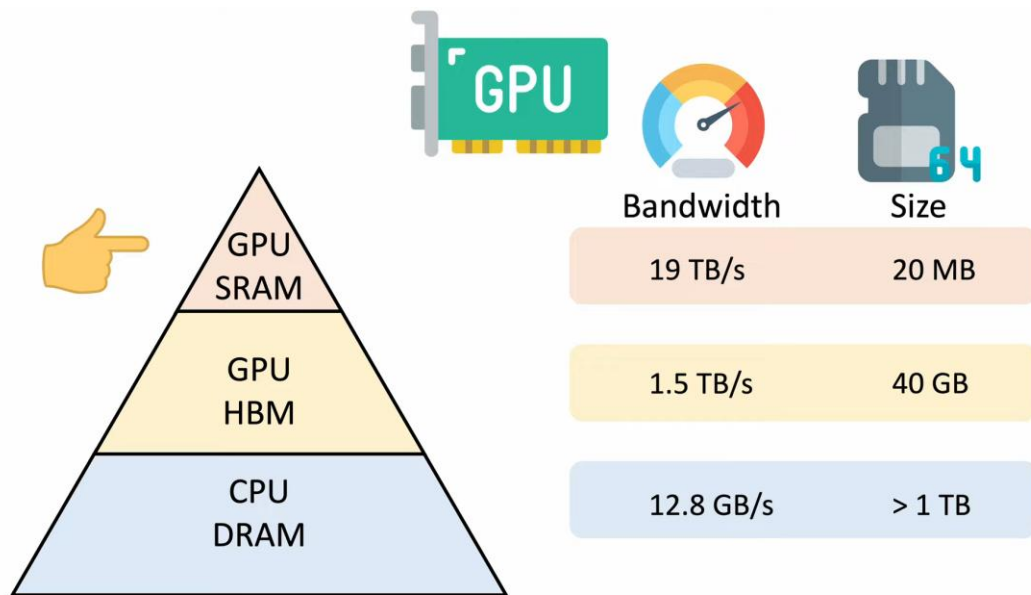


[\[Paper Review\] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

[How FlashAttention Accelerates Generative AI Revolution](#)

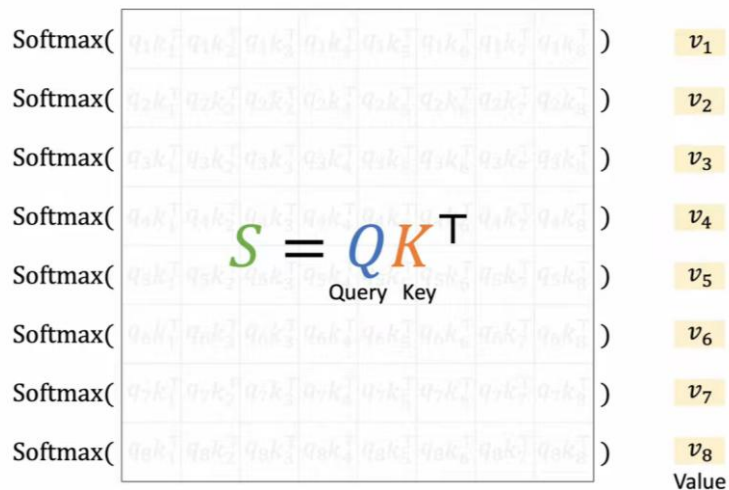
Flash Attention

- ✓ 실제 연산은 GPU에 있는 사이즈가 작은 SRAM에서 발생



Flash Attention

- ✓ 계산 과정에서 많은 메모리가 요구: N^2 가 빈번



$$A = \text{Softmax}(S)$$

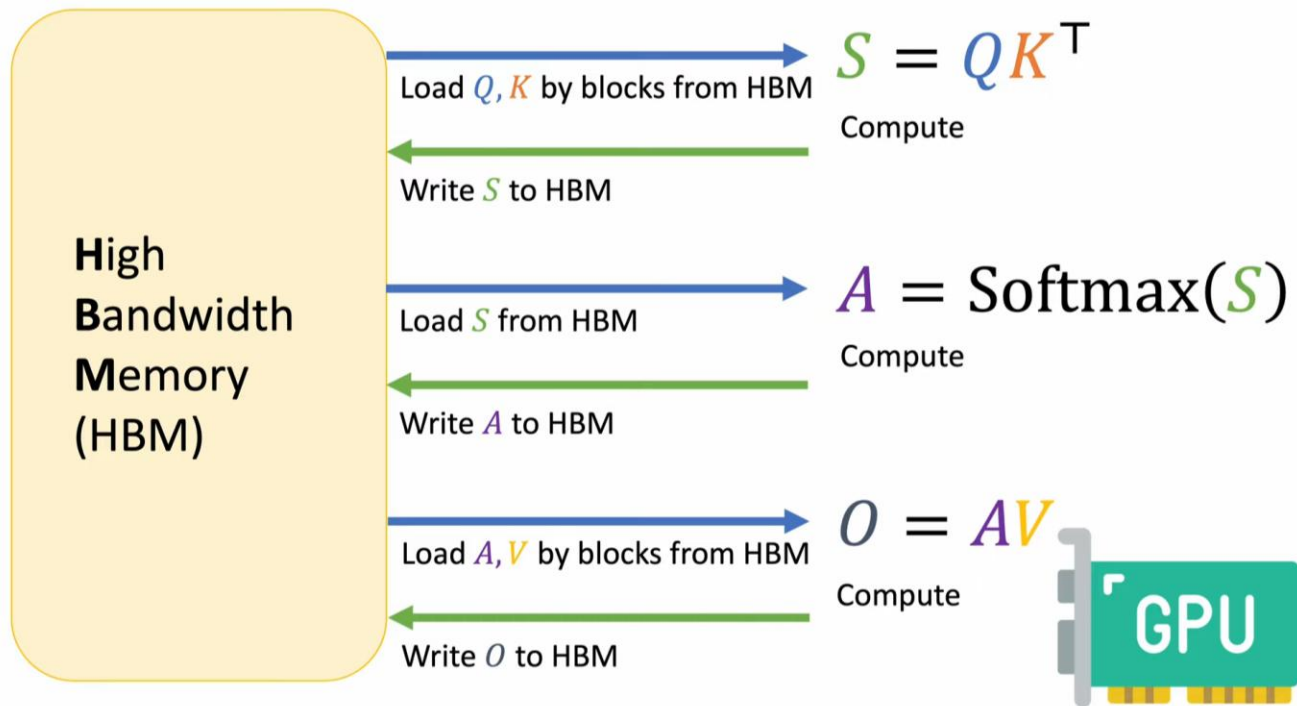
Attention weight

$$O = AV$$

Output Value

Flash Attention

- ✓ 메모리 I/O가 빈번하게 발생



Flash Attention

- ✓ Tiling이 없으면 A 메모리 액세스 + B 메모리 액세스 = 32번 발생

Tiling

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

B

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$C_{1,1}$	$C_{1,2}$		
$C_{2,1}$	$C_{2,2}$		

Without tiling: 32 memory accesses

$$C_{1,1} = 1 \times 1 + 2 \times 5 + 3 \times 9 + 4 \times 13$$

$$C_{1,2} = 1 \times 2 + 2 \times 6 + 3 \times 10 + 4 \times 14$$

$$C_{2,1} = 5 \times 1 + 6 \times 5 + 7 \times 9 + 8 \times 13$$

$$C_{2,2} = 5 \times 2 + 6 \times 6 + 7 \times 10 + 8 \times 14$$

$$C = A \times B$$

Flash Attention

- ✓ Tiling을 통해 메모리 I/O를 줄일 수 있음

Tiling

B

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Without tiling: 32 memory accesses

With tiling: 16 memory accesses

$N \times N$ block $\rightarrow 1/N$ memory access

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$c_{1,1}$	$c_{1,2}$		
$c_{2,1}$	$c_{2,2}$		

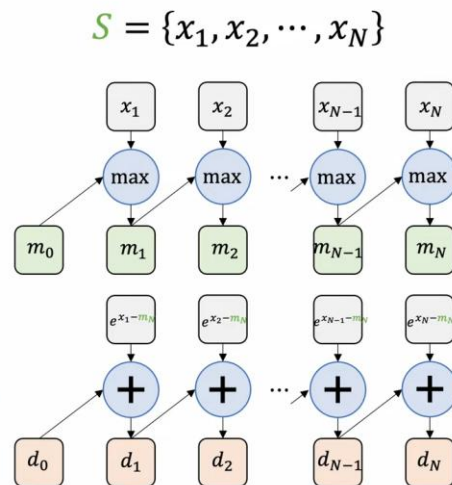
$$\begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix}$$

$$C = A \times B$$

Flash Attention

- ✓ Softmax는 m_N 과 d_N 을 구하는 for loop가 존재

$$A = \text{Softmax}(S)$$
$$m_0 = -\infty$$
$$m_N = \max(\{x_1, x_2, \dots, x_N\})$$
$$\text{for } 1 \leq i \leq N \text{ do}$$
$$m_i = \max(m_{i-1}, x_i)$$
$$d_N = \sum_{i=1}^N e^{x_i - m_N}$$
$$d_0 = 0$$
$$\text{for } 1 \leq i \leq N \text{ do}$$
$$d_i = d_{i-1} + e^{x_i - m_N}$$
$$\text{for } 1 \leq i \leq N \text{ do}$$
$$a_i = e^{x_i - m_N} / d_N$$



Flash Attention

✓ m_N, d_N 통합

$$A = \text{Softmax}(S) \quad S = \{x_1, x_2, \dots, x_N\}$$

$$m_0 = -\infty$$

for $1 \leq i \leq N$ do

$$m_i = \max(m_{i-1}, x_i)$$

$$d_0 = 0$$

for $1 \leq i \leq N$ do

$$d_i = d_{i-1} + e^{x_i - m_N}$$

for $1 \leq i \leq N$ do

$$a_i = \frac{e^{x_i - m_N}}{d_N}$$

$$d_i = \sum_{j=1}^i e^{x_j - m_N} \quad d'_i = \sum_{j=1}^i e^{x_j - m_i} \quad d'_N = d_N$$

$$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_i} \right) + e^{x_i - m_i}$$

$$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j + m_i} \right) \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

더미값: 결과는 1

$$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}} \right) \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

d'_{i-1}

Flash Attention

✓ O_i 통합

$$S = QK^\top \quad A = \text{Softmax}(S) \quad O = AV \quad S = \{x_1, x_2, \dots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1}e^{m_{i-1}-m_i} + e^{x_i-m_i}$$

$$m_0 = -\infty$$

$$d_0 = 0$$

for $1 \leq i \leq N$ do

$$x_i = qk_i^\top$$

$$m_i = \max(m_{i-1}, x_i)$$

$$d'_i = d'_{i-1}e^{m_{i-1}-m_i} + e^{x_i-m_i}$$

$$o_0 = 0$$

for $1 \leq i \leq N$ do

$$a_i = \frac{e^{x_i-m_N}}{d'_N}$$

$$o_i = o_{i-1} + a_i v_i$$

return o_N

$$o_i = \sum_{j=1}^i \frac{e^{x_j-m_N}}{d'_N} v_j \quad o_N = o'_N$$

$$o'_i = \sum_{j=1}^i \frac{e^{x_j-m_i}}{d'_i} v_j$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j-m_i}}{d'_i} v_j + \frac{e^{x_i-m_i}}{d'_i} v_i$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j-m_i}}{d'_i} \frac{d'_{i-1}}{d'_{i-1}} \frac{e^{-m_{i-1}}}{e^{-m_{i-1}}} v_j + \frac{e^{x_i-m_i}}{d'_i} v_i$$

$$o'_i = \left(\sum_{j=1}^{i-1} \frac{e^{x_j-m_{i-1}}}{d'_{i-1}} v_j \right) \frac{d'_{i-1}}{d'_i} e^{m_{i-1}-m_i} + \frac{e^{x_i-m_i}}{d'_i} v_i$$

Flash Attention

- ✓ 하나의 Loop로 통합: Flash Attention

$$S = QK^T \quad A = \text{Softmax}(S) \quad O = AV \quad S = \{x_1, x_2, \dots, x_N\}$$
$$x_i = qk_i^T$$

for $1 \leq i \leq N$ **do**

$$x_i = qk_i^T$$

$$m_i = \max(m_{i-1}, x_i)$$

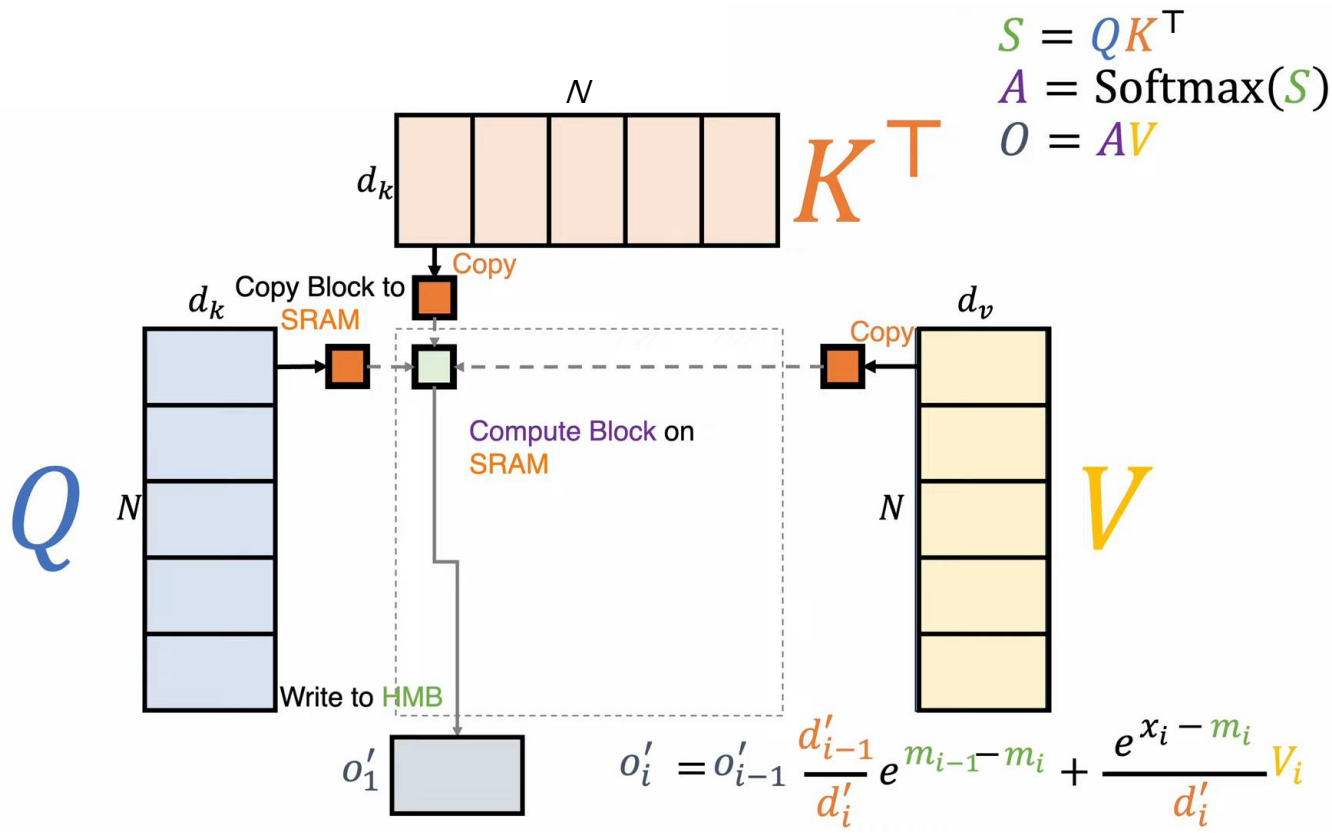
$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = o'_{i-1} \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

return o'_N

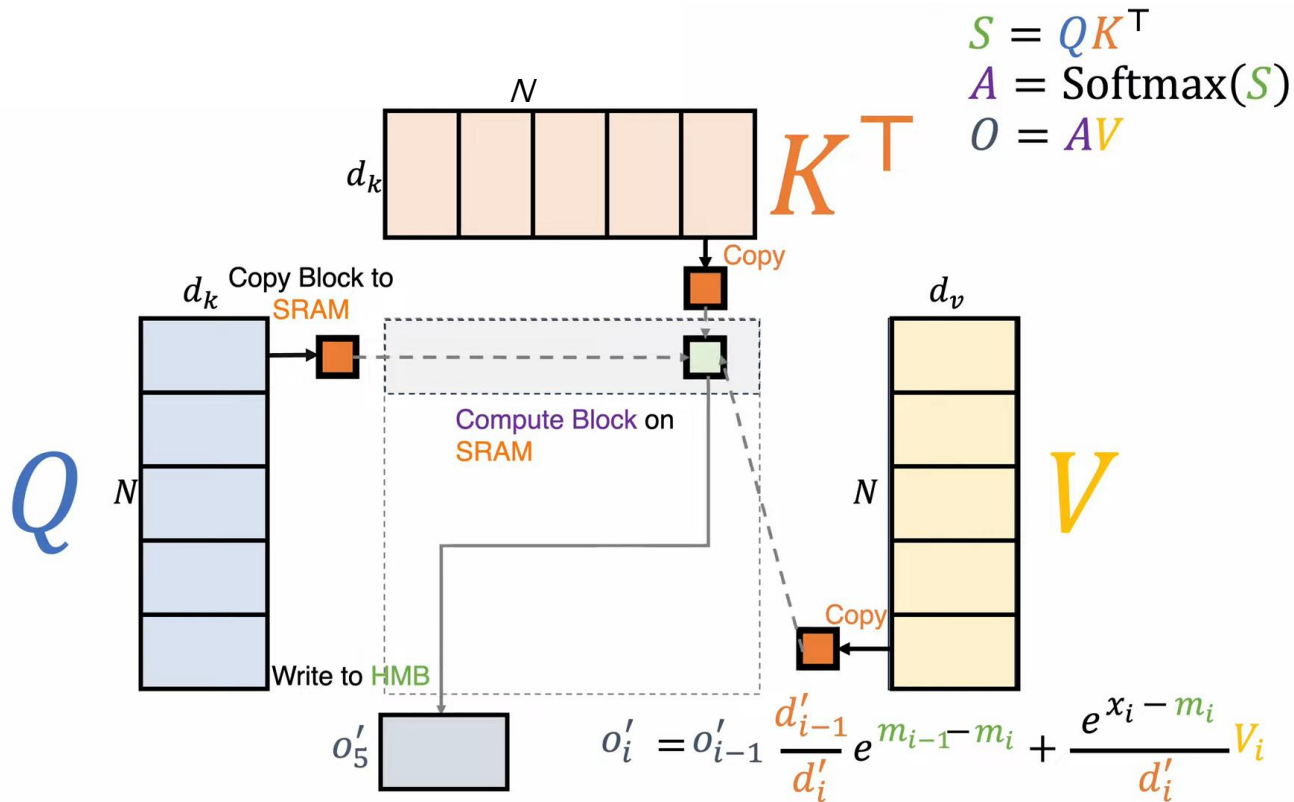
Flash Attention

- ✓ 대용량 메모리를 주고 받는 일이 없음



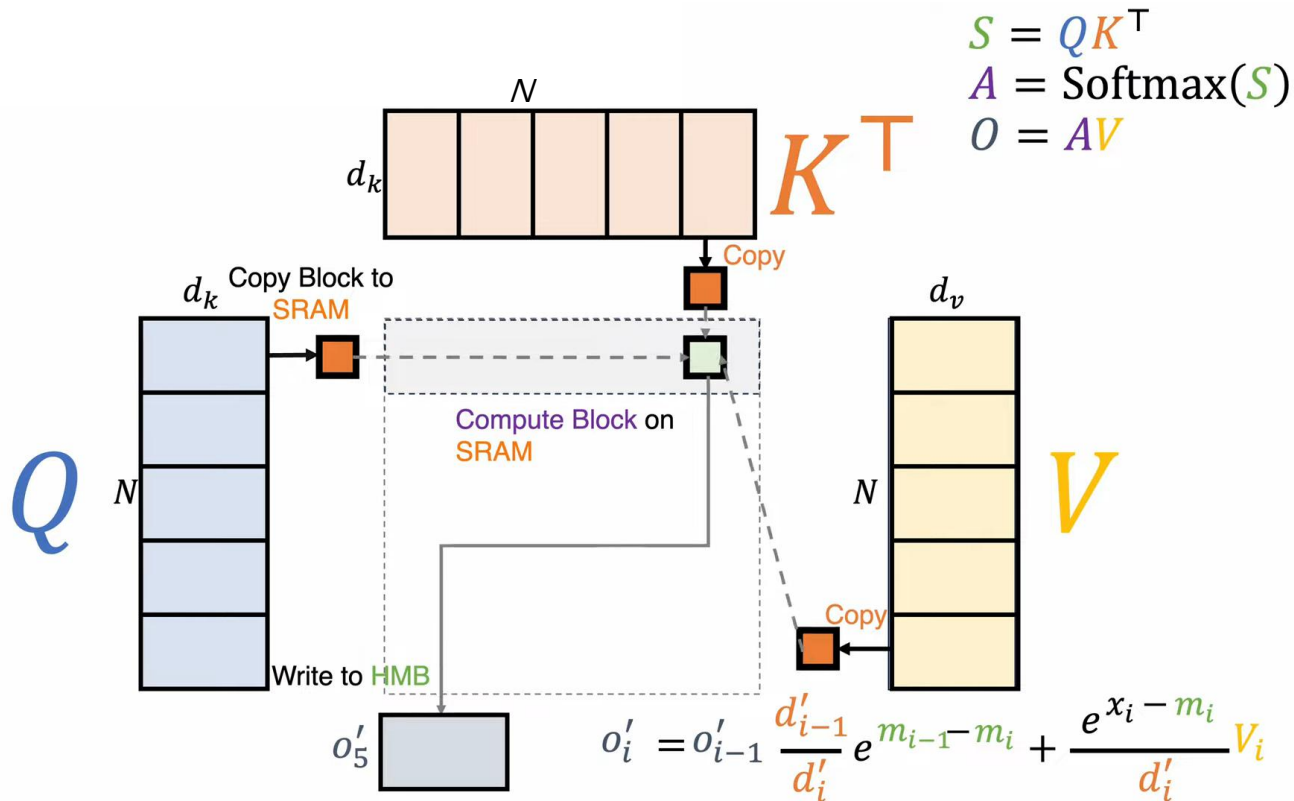
Flash Attention

- 계산에 필요한 메모리만 SRAM에 Copy하고, 계산 결과만 HBM에 씀

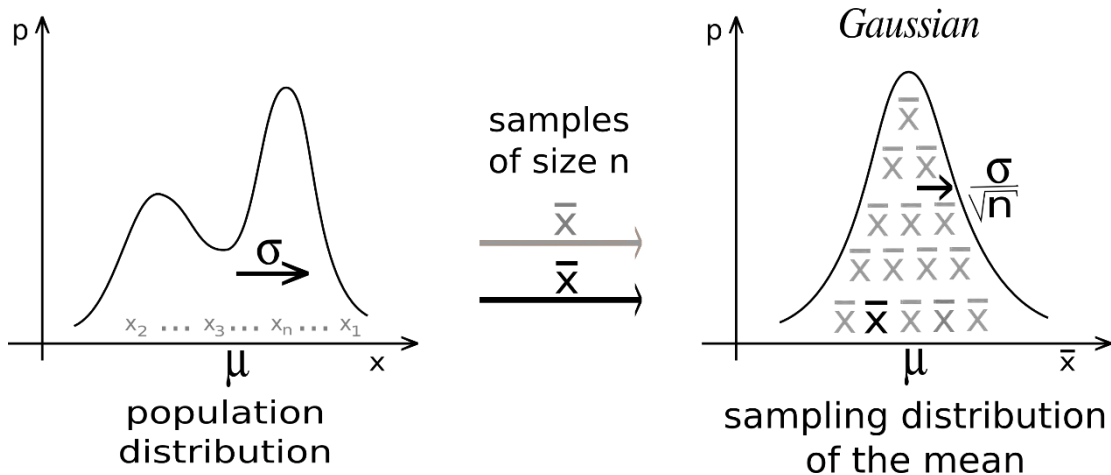


Flash Attention

- ✓ 계산에 필요한 메모리만 SRAM에 Copy하고, 계산 결과만 HBM에 씀



- ✓ KV를 양자화해서 메모리 사용량을 줄이고, 추론 속도를 빠르게 함
 - 정규화 분포를 만들어서 양자화

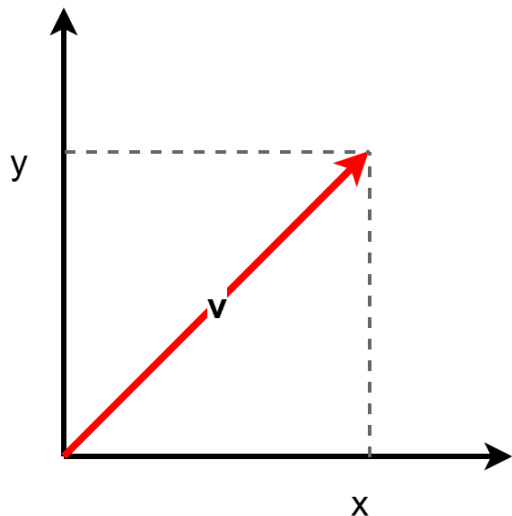


0.0000023
0.9999428 <-- !!!
 0.0000738
 0.0000003
 ...

✓ 벡터

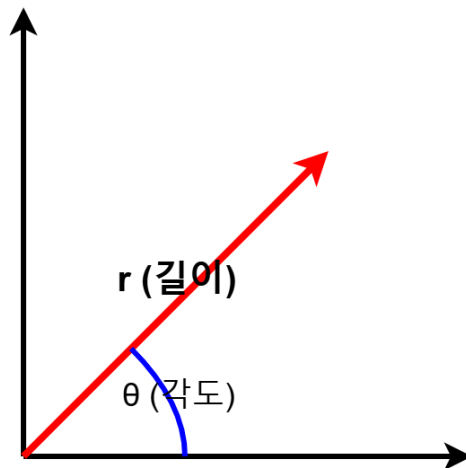
직교좌표계 표현
(Cartesian Coordinates)

x 좌표, y 좌표



극좌표계 표현
(Polar Coordinates)

길이(r), 각도(θ)



벡터 길이

$$|\mathbf{v}| = \sqrt{x^2 + y^2}$$

$$\begin{aligned} |\mathbf{v}| &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} = 5 \end{aligned}$$

단위 벡터

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{|\mathbf{v}|}$$

$$\hat{\mathbf{v}} = \left(\frac{3}{5}, \frac{4}{5} \right)$$

$$\begin{aligned} |\hat{\mathbf{v}}| &= \sqrt{0.6^2 + 0.8^2} \\ &= \sqrt{0.36 + 0.64} \\ &= \sqrt{1} = 1 \end{aligned}$$

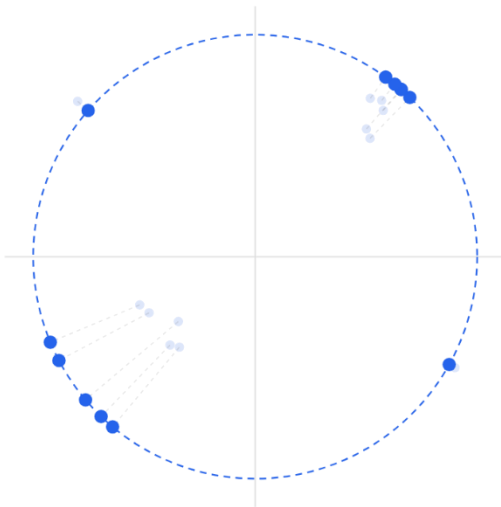
✓ 단위 벡터를 만들어서 랜덤으로 회전: 랜덤 시드만 저장

- 회전 후의 첫 번째 차원 값(y_1)은 원래 벡터의 값(x)들에 무작위 가중치(r)를 곱해서 전부 더한 값으로 결정

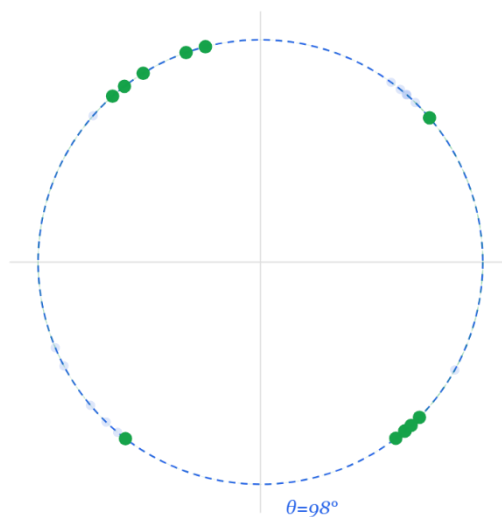
$$y_1 = r_1x_1 + r_2x_2 + \dots + r_{128}x_{128}$$

- 중심극한원리: 여러 개의 독립적인 무작위 값들을 다 더하면, 원래 값이 어떻게 생겼든 상관없이 무조건 정규분포 (종 모양)를 띠게 된다

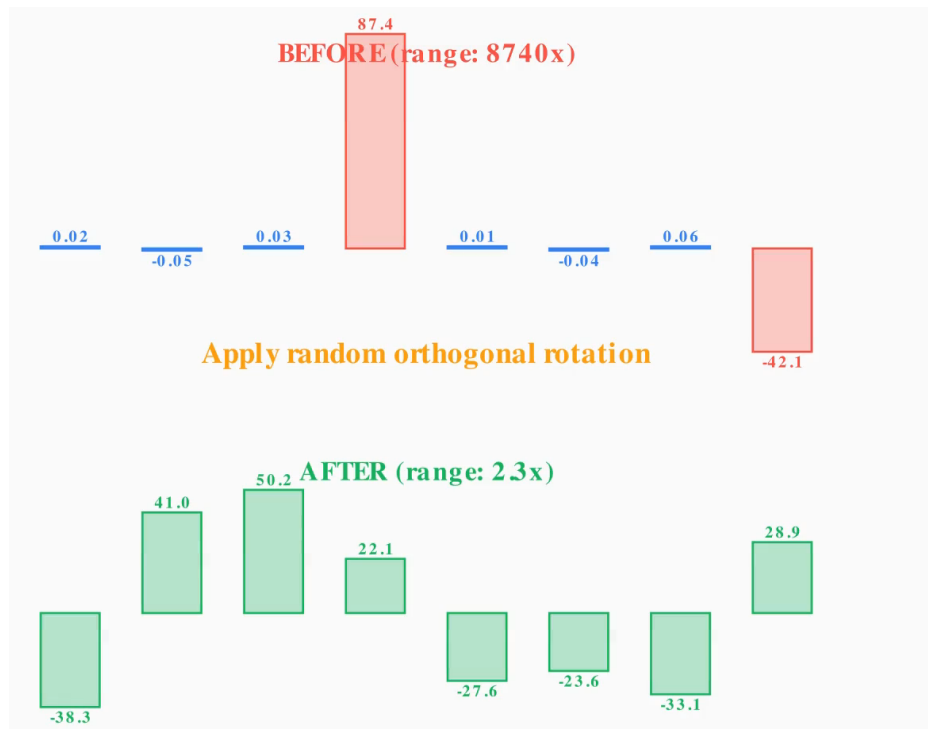
Step 1: Normalize to unit circle



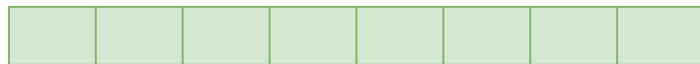
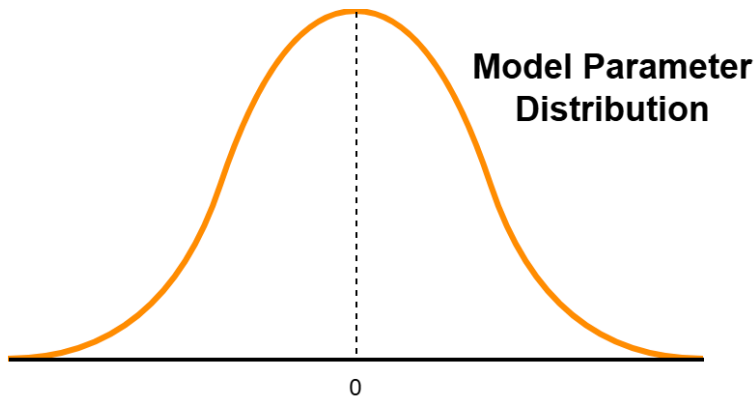
Step 2: Random rotation



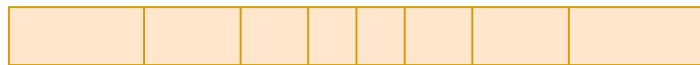
✓ 무작위 회전을 통한 이상치 분산



- ✓ 회전한 단위 벡터를 8구간으로 양자화

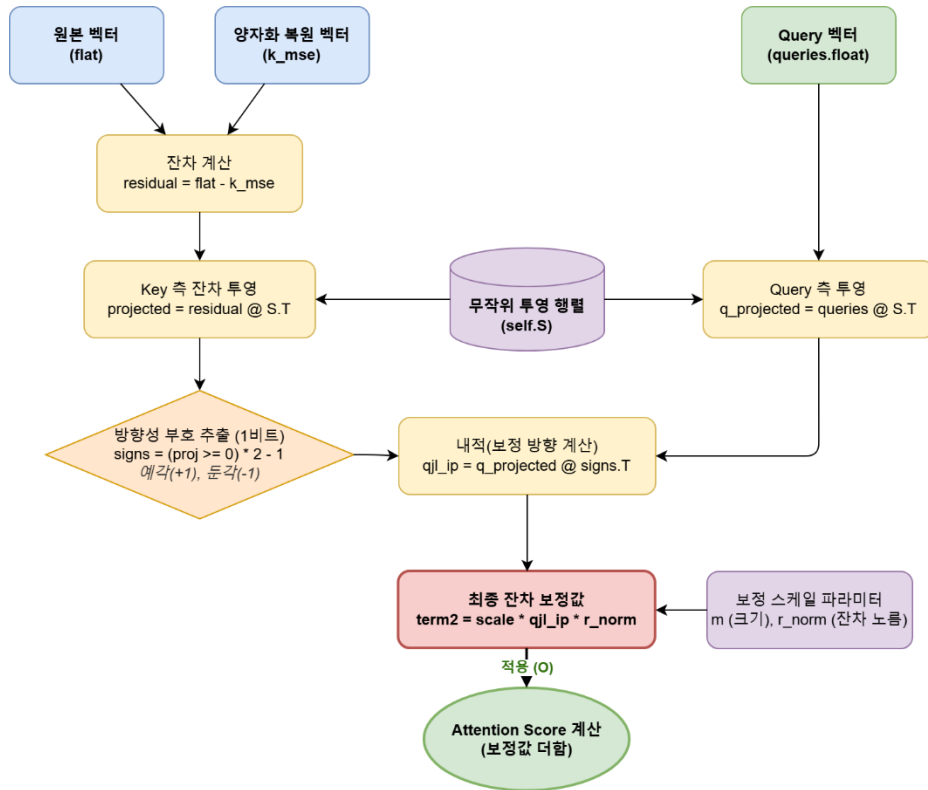


Equally-spaced buckets



-1에서 1사이의 정규 분포를 따른 고정된 8개 눈금

✓ QJL: 1비트 부호 정보를 저장하여 Attention Score 잔차 보정(양자화아님)



```

# Term 1: Q @ K_mse^T (standard matmul)
term1 = torch.matmul(queries.float(), k_mse.transpose(-2, -1))

# Term 2: QJL correction
# Project queries through S: S @ q for each query
q_projected = torch.matmul(queries.float(), self.S.T)

# <S@q, signs_k> for all pairs
qjl_ip = torch.matmul(q_projected, signs.transpose(-2, -1))

# Scale by residual norms and correction factor
m = self.S.shape[0]
term2 = correction_scale * qjl_ip * r_norm.unsqueeze(-2)

atten_score = term1 + term2
  
```

[도대체 중 모양에서 원주율 \$\pi\$ 가 왜? | 확률과 통계](#)

- ✓ KV 캐시 메모리를 줄여 더 많은 Context가 가능하게 함
- ✓ 양자화를 통해 메모리 IO 부하를 줄여서 추론을 빠르게 함
- ✓ 메모리가 줄어들어서 메모리가 많이 필요하지 않다는 잘못된 해석
- ✓ Softmax에 의해 QJL의 노이즈가 뺏기기 되어 QJL이 성능을 떨어뜨리다는 결과가 있음
 - <https://github.com/tonbistudio/turboquant-pytorch>



Appendix

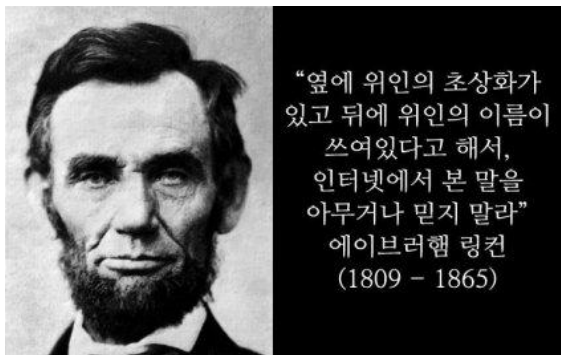
✓ 우리는 무엇을 해야 하는가?

주시하고

“지진경보는 이미 발령,
쓰나미가 언제 발생하는가?”

“유료 모델을 구독해서 쓰자”

항상 의심하고



함께 하자

“엄청 똑똑하지만
업무를 전혀
모르는 신입”

“인류와 함께
진화하는 존재”

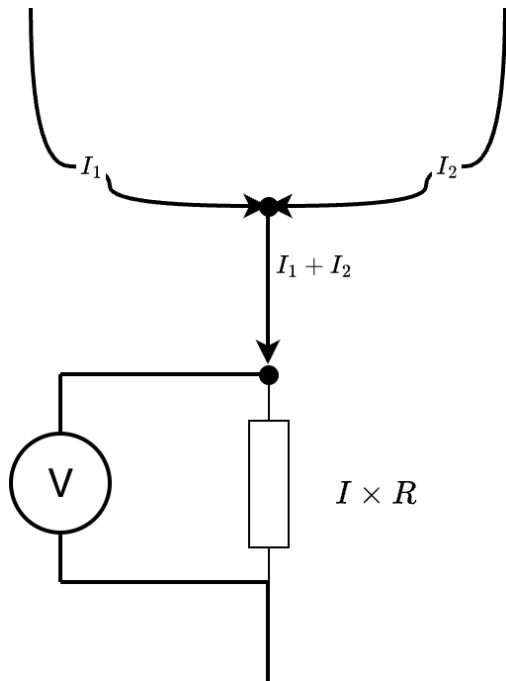
✓ AI를 어떻게 바라봐야 할까



[이젠 옛날로 돌아갈 수 없다. 모든 것이 인공지능인 바둑계 충격적인 근황 | 알파고 10주년 | AI | 신진서 | 이세돌 | 다큐프라임 | #골라둔다큐](#)

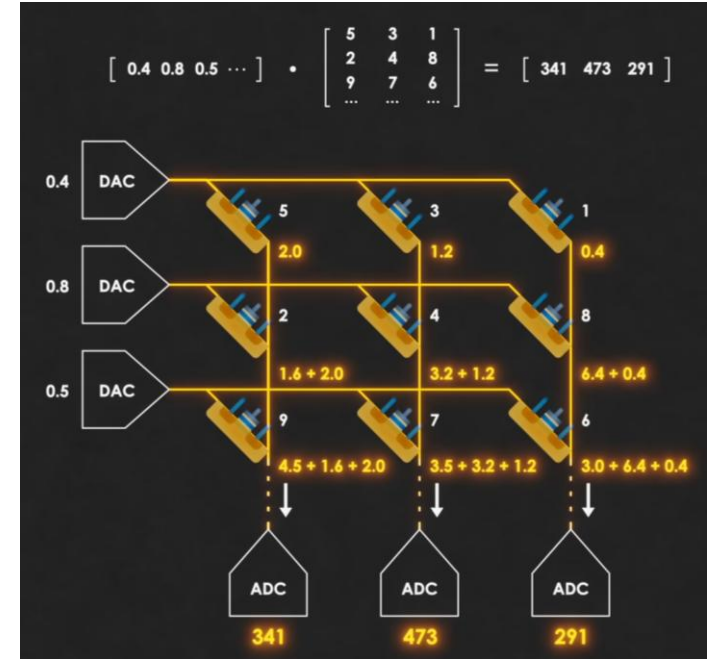
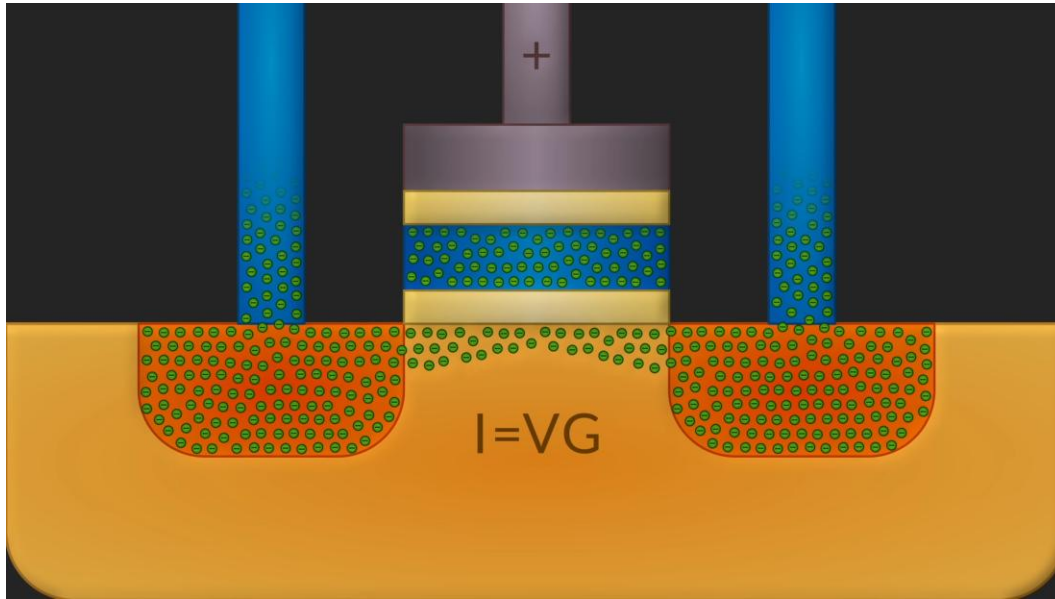
Analog Computer: 전류와 전압

- ✓ 계산에 필요한 메모리만 SRAM에 Copy하고, 계산 결과만 HBM에 씀



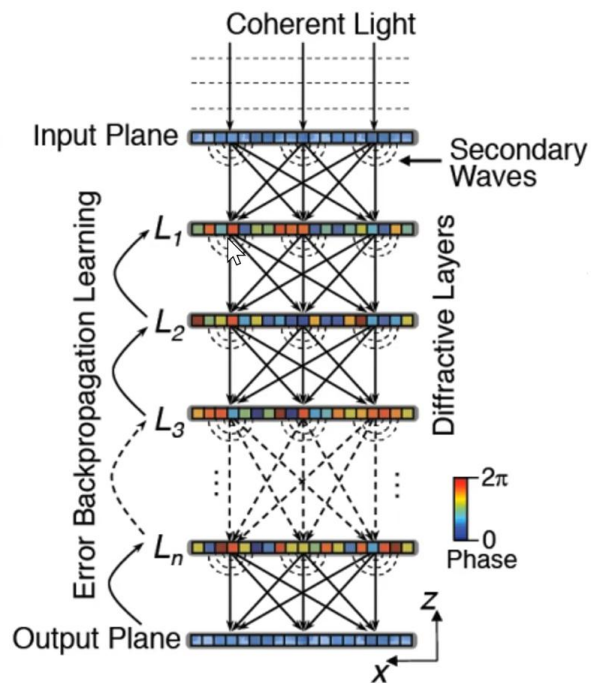
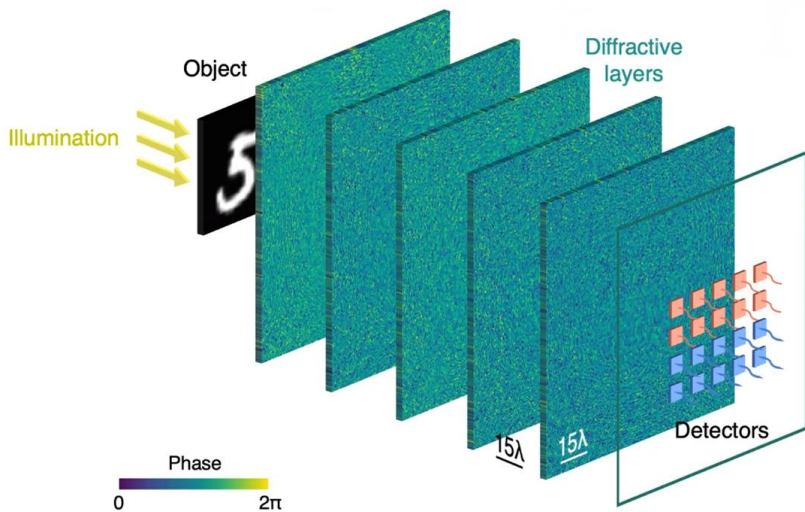
Analog Computer: Mythic

- ✓ 디지털 플래시 저장 셀처럼 전압에 따라 전류를 조정하는 칩 제작



아날로그 컴퓨터: Diffractive Optical Neural Networks

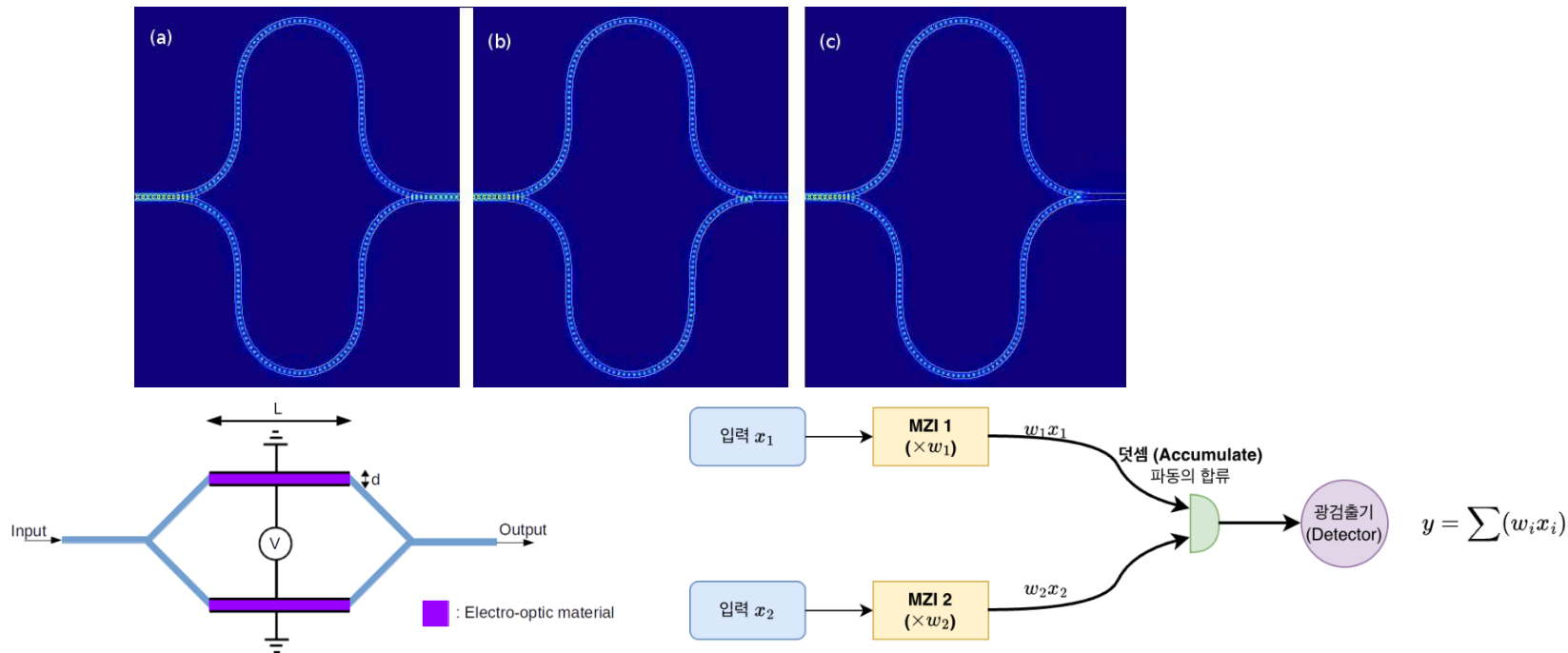
- ✓ 필터에 의한 빛의 회절로 계산

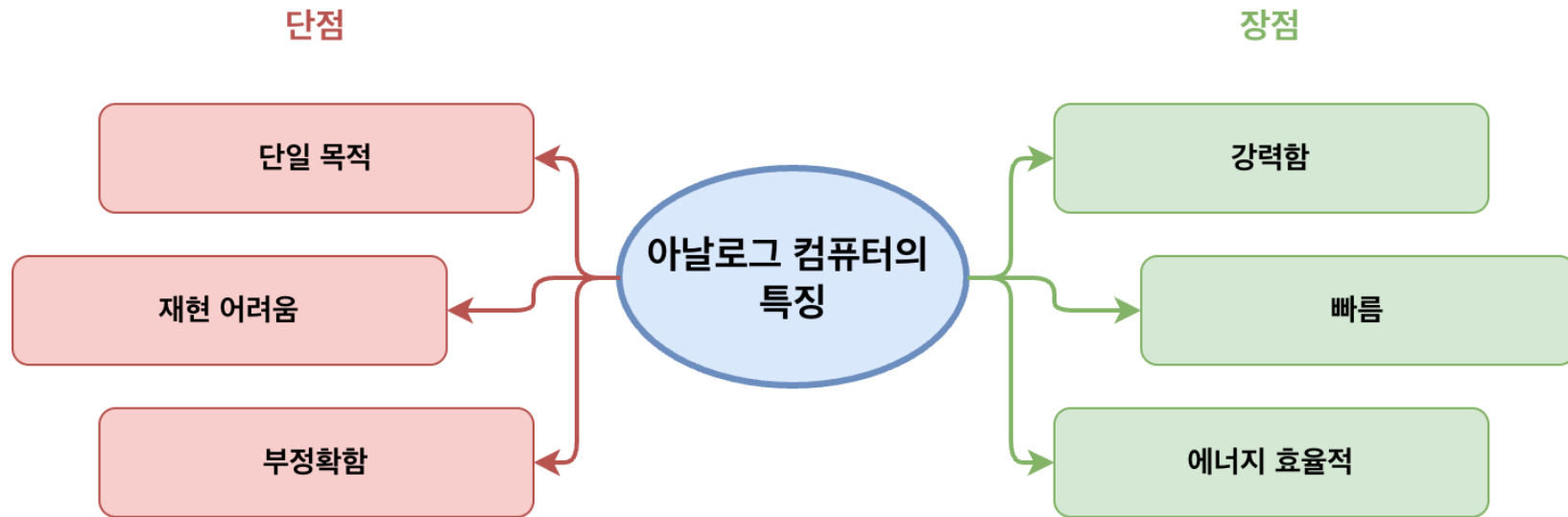


Transmission/Reflection

아날로그 컴퓨터: Lightmatter사의 Envisе 칩

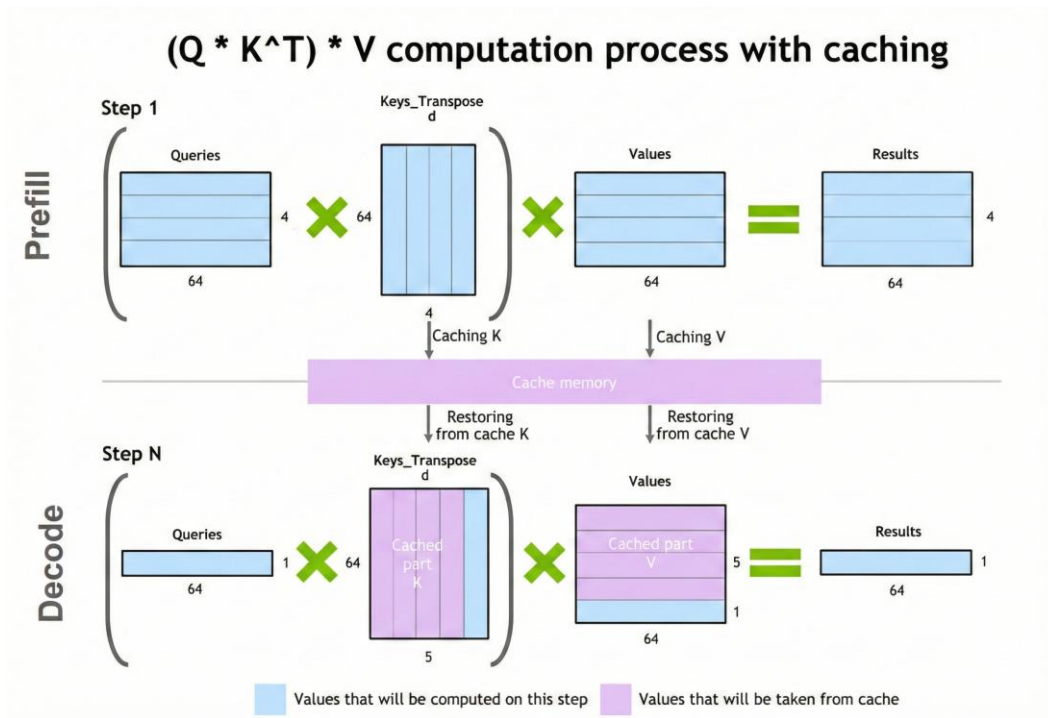
- ✓ 마하-젠더 간섭계: 빛을 두 방향으로 보내서 위상차를 발생시킨 후 간섭과 상쇄 이용
- ✓ 에너지를 거의 쓰지 않고 빛의 속도로 행렬 연산을 처리





KV Caching

- ✓ 새로 생성되는 토큰만 입력



KV Caching

- ✓ Decode과정의 마스킹은 한줄로 나타나고, 0이라 실제 값의 변화는 없음

	K0	K1	K2	K3	K4	K5
Q0	0	1	1	1	1	1
Q1	0	0	1	1	1	1
Q2	0	0	0	1	1	1
Q3	0	0	0	0	1	1
Q4	0	0	0	0	0	1
Q5	0	0	0	0	0	0

Prefill

프롬프트 (토큰 4개) 병렬 처리

ptr_current_pos = 0
num_tokens_Q = 4
num_tokens_K = 4

Decode

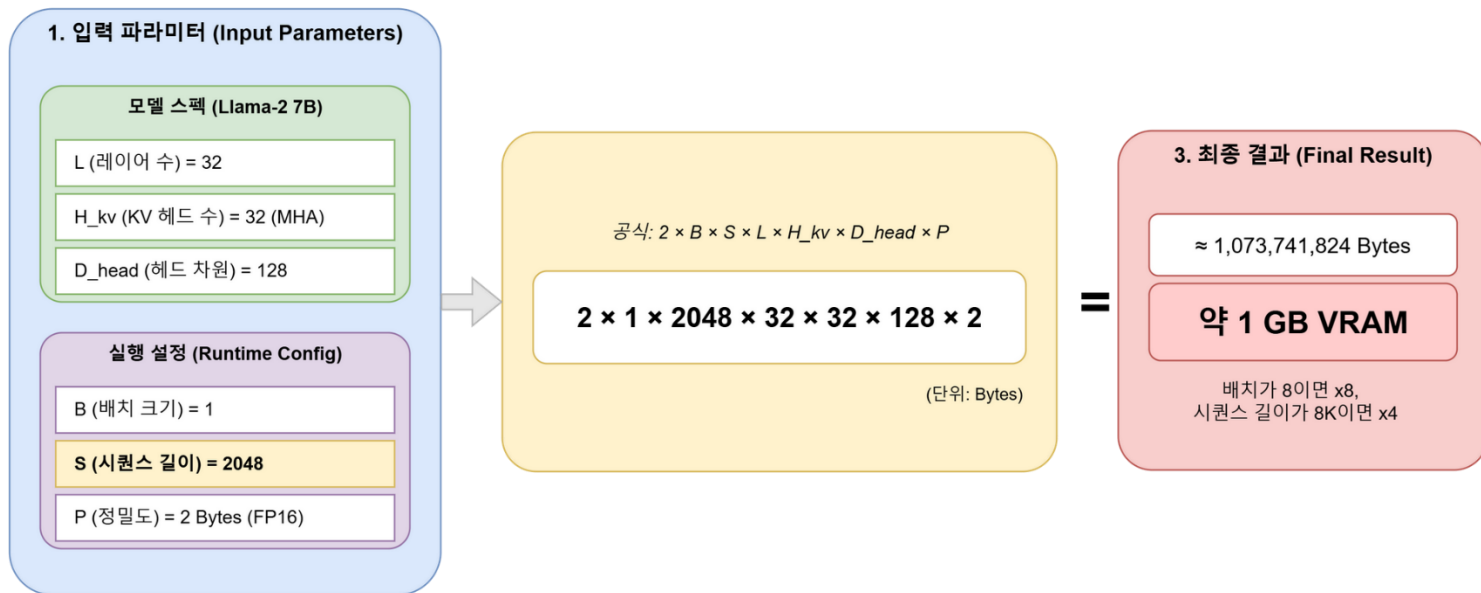
새 단어 (1개) 추가 생성

ptr_current_pos = 4
num_tokens_Q = 1
num_tokens_K = 5

KV Caching

✓ 메모리가 얼마나 증가하는가?

실행 계산 예시: Llama-2 7B (FP16, 시퀀스 길이 2048)



Thank You!